HOUSEHOLD ELECTRICITY CONSUMPTION PREDICTION SYSTEM

A PROJECT REPORT

Submitted by

Kunal Mittal (24BCS12354)
Ansh Kumar (24BCS12097)
Vishal Kumar Shrivastava (24BCS12629)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

November 2026

BONAFIDE CERTIFICATE

Certified that this project report "Household Electricity Consumption Prediction System" is the bonafide work of "Kunal Mittal, Ansh Kumar and Vishal Kumar Shrivastava" who carried out the project work under my/our supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT **SUPERVISOR**

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

ABSTRACT 10
GRAPHICAL ABSTRACT 12
ABBREVIATIONS 13
SYMBOLS 14
CHAPTER 1. INTRODUCTION 15
1.1. Identification of Client / Need / Relevant Contemporary issue 15
1.2. Identification of Problem 15
1.3. Identification of Tasks 16
1.4. Timeline 16
1.5. Organization of the Report 17
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 18
2.1. Timeline of the reported problem 18
2.2. Existing solutions 18
2.3. Bibliometric analysis 21
2.4. Review Summary 21
2.5. Problem Definition 21
2.6. Goals/Objectives 22
CHAPTER 3. DESIGN FLOW/PROCESS 24
3.1. Evaluation & Selection of Specifications/Features 24
3.2. Design Constraints 28
3.3. Analysis of Features and finalization subject to constraints 29
3.4. Design Flow 30
3.5. Design selection 30
3.6. Implementation plan/methodology 31
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 43
4.1. Implementation of solution 43
CHAPTER 5. CONCLUSION AND FUTURE WORK 83
5.1. Conclusion 83
5.2. Future work 83
REFERENCES 85
APPENDIX 87

1. Plagiarism Report 87
2. Design Checklist 87
3. Syllabus experiments performed in the project 87
4. Source code 88
USER MANUAL 133

List of Figures

Figure 0.1: *Graphical abstract: end-to-end MLOps architecture of VoltCast* 12
Figure 1.1: *Gantt chart of the project timeline* 17
Figure 3.1: *Daily mean active power with 30-day rolling mean (strong yearly seasonality)* 24
Figure 3.2: *Average consumption by weekday and hour (morning and evening peaks)* 24
Figure 3.3: *Monthly pattern and share of energy per sub-meter* 25
Figure 3.4: *Share of missing one-minute readings per month* 25
Figure 3.5: *Distribution of each hourly variable* 26
Figure 3.6: *Autocorrelation of hourly consumption up to eight days* 26
Figure 3.7: *Average weekly load profile* 26
Figure 3.8: *Average energy by sub-meter and hour of day* 27
Figure 3.9: *Hourly consumption by month (box plots)* 27
Figure 3.10: *Monthly average consumption, year by year* 27
Figure 3.11: *Box plots of hourly readings; points beyond the whiskers are IQR outliers* 28
Figure 3.12: *Alternative designs considered* 30
Figure 3.13: *Flowchart of the training and deployment algorithm* 31
Figure 3.14: *Detailed block diagram of the MLOps architecture* 32
Figure 3.15: *Data-flow diagram, level 0 (context)* 32
Figure 3.16: *Data-flow diagram, level 1* 33
Figure 3.17: *Use-case diagram* 33
Figure 3.18: *Sequence diagram: serving a forecast request* 34
Figure 3.19: *Deployment diagram: Docker Compose, Vercel and Kubernetes* 34
Figure 3.20: *CI/CD pipeline* 35
Figure 3.21: *DVC pipeline graph* 35
Figure 3.22: *Flowchart of the preprocessing stage* 36
Figure 3.23: *Flowchart of the training and tuning stage* 37
Figure 3.24: *Flowchart of the peak-hour classifier stage* 38
Figure 3.25: *Flowchart of serving a prediction* 39
Figure 3.26: *Flowchart of drift monitoring and the retraining decision* 40
Figure 4.1: *Correlation matrix of the measured variables* 43
Figure 4.2: *Validation MAE and R^2 of all models (deployed model in orange, baselines in grey)* 44
Figure 4.3: *MLflow tracking UI: runs of every model, tuned variant and final model* 45
Figure 4.4: *Training vs validation MAE of each model (default hyper-parameters)* 46
Figure 4.5: *Validation curve of the decision tree: error against max_depth* 46
Figure 4.6: *Learning curves with time-series cross-validation* 46
Figure 4.7: *Validation MAE before and after GridSearchCV tuning* 47
Figure 4.8: *Hyper-parameter response: decision-tree grid, KNN neighbours, ridge alpha* 47
Figure 4.9: *MAE in each time-series cross-validation fold* 48
Figure 4.10: *Test MAE, RMSE, R^2 and accuracy ($1 - WAPE$) of every model* 50
Figure 4.11: *Forecasts of every model over the last seven days of the test set* 51

Figure 4.12*: Predicted vs actual for every model (test set)* 52
Figure 4.13*: Residual distribution of every model (test set)* 53
Figure 4.14*: Next-hour forecast vs actual consumption, last 14 days of the test set* 53
Figure 4.15*: Predicted vs actual scatter and residual distribution* 54
Figure 4.16*: Top 15 features by permutation importance* 54
Figure 4.17*: Forecast MAE by hours ahead, compared with the seasonal-naive baseline* 55
Figure 4.18*: Peak-hour classifier on the test set: confusion matrix, ROC and precision-recall curves* 56
Figure 4.19*: Confusion matrices of the three classifiers (test set)* 56
Figure 4.20*: ROC and precision-recall curves of the three classifiers (test set)* 56
Figure 4.21*: Precision, recall, F1 and accuracy against the decision threshold* 57
Figure 4.22*: VoltCast web application (React) with a live 24-hour forecast and peak-hour alerts* 59
Figure 4.23*: Prometheus target page: the VoltCast API is being scraped (state UP)* 60
Figure 4.24*: Grafana dashboard "VoltCast – Model Monitoring" fed by Prometheus* 61
Figure 4.25*: Population Stability Index of monitored features (train vs test)* 61
Figure 4.26*: PSI against the training data and test MAE, month by month* 62
Figure 4.27*: Test error across sub-groups (dashed line: overall MAE)* 63
Figure 4.28*: Responsible-AI section of the web application* 63
Figure 4.29*: Python and library versions used, run 2026-11-03 18:51:54* 64
Figure 4.30*: Git history of the project repository, run 2026-11-03 19:21:38* 64
Figure 4.31*: dvc dag: the pipeline stages and their dependencies, run 2026-11-03 18:51:56* 65
Figure 4.32*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 1 of 3), run 2026-11-03 18:51:58* 66
Figure 4.33*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 2 of 3), run 2026-11-03 18:51:58* 67
Figure 4.34*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 3 of 3), run 2026-11-03 18:51:58* 68
Figure 4.35*: dvc status after the run, run 2026-11-03 19:03:29* 68
Figure 4.36*: dvc metrics show: every tracked metric (part 1 of 2), run 2026-11-03 19:03:32* 69
Figure 4.37*: dvc metrics show: every tracked metric (part 2 of 2), run 2026-11-03 19:03:32* 69
Figure 4.38*: pytest -v: the automated test suite, run 2026-11-03 19:03:34* 70
Figure 4.39*: MLflow: tracked runs of the forecasting experiment (part 1 of 3), run 2026-11-03 19:03:41* 71
Figure 4.40*: MLflow: tracked runs of the forecasting experiment (part 2 of 3), run 2026-11-03 19:03:41* 72
Figure 4.41*: MLflow: tracked runs of the forecasting experiment (part 3 of 3), run 2026-11-03 19:03:41* 73

Figure 4.42*: MLflow model registry: versions of voltcast-forecaster, run 2026-11-03 19:03:46* 73
Figure 4.43*: Building the React web application, run 2026-11-03 19:03:50* 73
Figure 4.44*: docker compose up: building the image and starting the five services, run 2026-11-03 19:03:55* 74
Figure 4.45*: Docker image and Compose services, run 2026-11-03 19:04:13* 74
Figure 4.46*: Flask API: health check and model information, run 2026-11-03 19:04:16* . 75
Figure 4.47*: Flask API: 4-hour forecast with peak alerts, run 2026-11-03 19:04:16* 76
Figure 4.48*: Flask API: POST /api/predict with one week of readings, run 2026-11-03 19:04:17* 76
Figure 4.49*: Flask API: invalid requests are rejected with HTTP 422, run 2026-11-03 19:04:18* 77
Figure 4.50*: Prometheus metrics exposed by the API at /api/metrics, run 2026-11-03 19:04:50* 77
Figure 4.51*: Prometheus: scrape target health and a PromQL query, run 2026-11-03 19:04:50* 78
Figure 4.52*: Prediction log (logs/predictions.jsonl), run 2026-11-03 19:04:51* 78
Figure 4.53*: Grafana: provisioned dashboard and data source, run 2026-11-03 19:04:51* 79
Figure 4.54*: Kubernetes manifest and Docker Compose configuration validation, run 2026-11-03 19:05:11* 79
Figure 4.55*: GitHub Actions: recent CI/CD runs, run 2026-11-03 19:21:39* 80
Figure 4.56*: GitHub Actions: steps of the latest run, run 2026-11-03 19:21:40* 80
Figure 4.57*: Vercel: production deployments created from GitHub pushes, run 2026-11-03 19:21:44* 81
Figure 4.58*: The live deployment answering requests, run 2026-11-03 19:21:50* 81
Figure M.1*: Web application – live forecast* 134
Figure M.2*: Web application – model performance, over-fitting and tuning* 135
Figure M.3*: Web application – peak-hour classifier metrics* 136
Figure M.4*: Grafana monitoring dashboard* 136
Figure M.5*: Prometheus targets page* 137
Figure M.6*: Streamlit dashboard (Docker / local) – forecast tab* 137

List of Tables

Table 0.1*: Standards referred to in the project* 9
Table 0.2*: List of abbreviations* 13
Table 0.3*: List of symbols* 14
Table 1.1*: Identified tasks* 16
Table 2.1*: Timeline of the reported problem* 18
Table 2.2*: Bibliometric comparison of existing solutions* 21
Table 2.3*: Mapping of project objectives to course outcomes* 22
Table 3.1*: Descriptive statistics of the hourly data* 27
Table 3.2*: Selected feature specification (31 features)* 28
Table 3.3*: Design constraints* 28
Table 3.4*: Comparison of alternative designs* 30
Table 3.5*: Software modules of the implementation* 41
Table 4.1*: Training and validation performance of all models (default hyper-parameters)* 44
Table 4.2*: Hyper-parameter tuning results* 47
Table 4.3*: Cross-validation MAE (kW) per time-series fold, tuned models* 48
Table 4.4*: Held-out test performance (Jun–Nov 2010)* 49
Table 4.5*: Test-set error metrics of every model (kW, kW²)* 49
Table 4.6*: Test-set accuracy-style metrics, training time and prediction time of every model* 49
Table 4.7*: Forecast error by horizon* 55
Table 4.8*: Peak-hour classifiers on the validation set* 55
Table 4.9*: Classification reports on the test set* 57
Table 4.10*: Effect of the probability threshold (tuned decision tree, test set)* 57
Table 4.11*: REST API endpoints* 58
Table 4.12*: Automated test suite results* 60
Table 4.13*: Monthly PSI of monitored features and monthly test MAE* 62
Table 4.14*: Achievement of objectives* 81
Table A.1*: Design checklist* 87
Table A.2*: Syllabus experiments mapped to the project* 87

List of Standards

Table 0.1*: Standards referred to in the project*

Standard	Publishing Agency	About the standard	Where used
ISO/IEC 23053:2022	ISO / IEC	Framework for Artificial Intelligence systems using Machine Learning; defines the ML pipeline components (data acquisition, pre-processing, training, evaluation, deployment).	Ch. 3.2, 3.6
ISO/IEC 25010:2023	ISO / IEC	Systems and software quality model (functional suitability, reliability, maintainability, portability).	Ch. 3.2, 4.1
IEC 62056 (DLMS/COSEM)	IEC	Electricity metering data exchange; defines how smart meters expose load-profile readings such as those used as model inputs.	Ch. 3.1
IS 16444	Bureau of Indian Standards	A.C. static direct-connected smart energy meters; relevant for deploying the system on Indian household smart meters.	Ch. 3.2
ISO 50001:2018	ISO	Energy management systems: requirements for measuring, analysing and improving energy performance.	Ch. 1.1, 3.2
OpenMetrics (Prometheus exposition format)	CNCF	Text format for exposing service metrics; the API's /api/metrics endpoint follows it so Prometheus can scrape it.	Ch. 3.6, 4.1
Digital Personal Data Protection Act, 2023	Government of India	Rules for processing digital personal data; applies when real household meter data is used.	Ch. 3.2, 4.1

ABSTRACT

Residential buildings account for a large share of national electricity demand, yet most households receive only a monthly bill and have no forward-looking view of their consumption. Utilities, in turn, need short-term load forecasts at household level to plan demand response as smart meters are rolled out. This project develops **VoltCast**, a Household Electricity Consumption Prediction System that forecasts a household's hourly active power for the next 1–48 hours, flags likely peak-consumption hours, and is built end to end with Machine Learning Operations (MLOps) practice.

The UCI "Individual Household Electric Power Consumption" dataset (2,075,259 one-minute readings over almost four years) is cleaned of 1.252% missing values, checked for outliers, resampled to 34,589 hourly records and turned into 31 leakage-free features. Nine regression models covering the algorithms of the course (linear regression, decision tree, k-nearest neighbours, support vector machine, random forest) plus ridge regression, gradient boosting and two naive baselines are trained on a chronological split, tuned with GridSearchCV using time-series cross-validation, and tracked in MLflow. Only syllabus algorithms are eligible for deployment, and the best of them, random forest, is deployed: test MAE 0.295 kW, MSE 0.188, R^2 0.620, accuracy (1 − WAPE) 67.9%, a 21.9% MAE reduction over the persistence baseline and within 0.001 kW of the best reference model (gradient boosting). A tuned decision-tree classifier detects peak hours with 80.3% accuracy, 77.9% recall and ROC-AUC 0.873. The experiments also show over-fitting clearly: the default decision tree reaches zero training error but the worst validation error, which tuning reduces from 0.506 to 0.380 kW.

The models are versioned with DVC and the MLflow model registry, and served by a Flask REST API with input validation. They run in Docker (a five-service Docker Compose stack, plus Kubernetes manifests) and on the Vercel cloud behind a React web application. A GitHub Actions pipeline reproduces the DVC pipeline, applies quality gates, runs 11 automated tests and builds the container on every change and weekly. Prometheus collects service metrics that Grafana displays, every prediction is logged, and a drift monitor based on the Population Stability Index correctly detected seasonal drift and recommended retraining. Fairness, transparency, privacy and security are analysed, including error across groups of hours.

Keywords: load forecasting, smart meter, random forest, decision tree, GridSearchCV, MLOps, MLflow, DVC, Flask, Docker, Prometheus, Grafana, responsible AI.

GRAPHICAL ABSTRACT

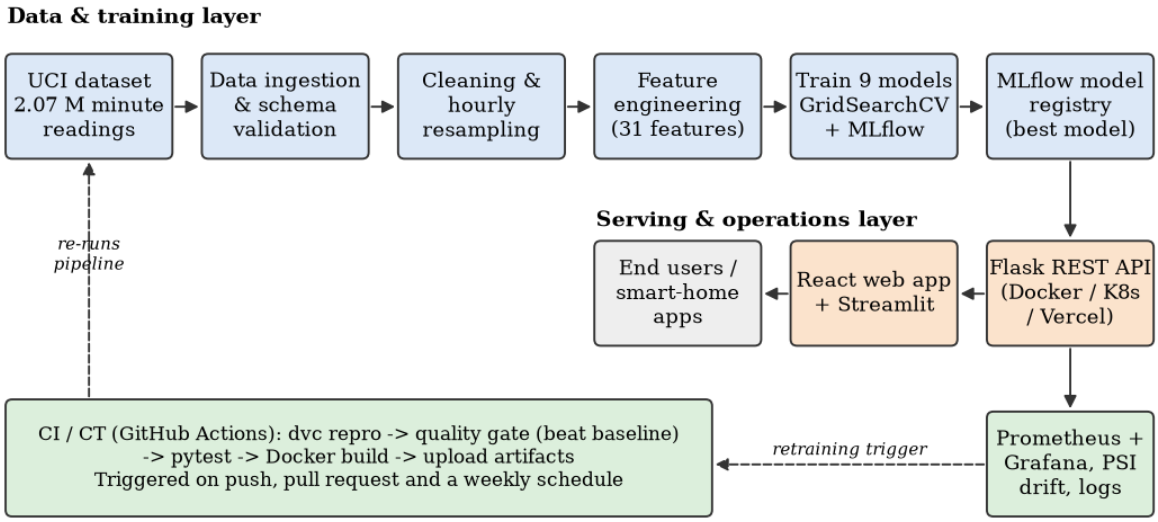


Figure 0.1*: Graphical abstract: end-to-end MLOps architecture of VoltCast*

ABBREVIATIONS

Table 0.2*: List of abbreviations*

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
AUC	Area Under the (ROC) Curve
CI/CD	Continuous Integration / Continuous Deployment
CO	Course Outcome
CT	Continuous Training
CV	Cross-Validation
CNN	Convolutional Neural Network
DVC	Data Version Control
EDA	Exploratory Data Analysis
FN / FP / TN / TP	False negative / false positive / true negative / true positive
HGB	Histogram-based Gradient Boosting
IEA	International Energy Agency
IQR	Inter-Quartile Range
K8s	Kubernetes
KNN	K-Nearest Neighbours
kW / kWh	Kilowatt / Kilowatt-hour
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MLOps	Machine Learning Operations
MSE	Mean Squared Error
NILM	Non-Intrusive Load Monitoring
PSI	Population Stability Index
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
R^2	Coefficient of determination
SVM / SVR	Support Vector Machine / Support Vector Regression
UCI	University of California, Irvine (ML Repository)

SYMBOLS

Table 0.3*: List of symbols*

Symbol	Meaning
y_t	Hourly mean global active power at hour t (kW)
$\hat{y}_t$	Predicted active power at hour t (kW)
lag_k	Value of the target k hours earlier, y_{t-k}
μ_w, σ_w	Rolling mean and standard deviation over the previous w hours
n	Number of evaluation samples
α	Ridge regularisation strength
η	Learning rate of gradient boosting
k	Number of neighbours in KNN
C, ε	SVR penalty and epsilon-insensitive margin
σ_r	Standard deviation of test residuals (used for prediction intervals)
h	Forecast horizon in hours
τ	Peak-hour threshold (75th percentile of training consumption, kW)
p_i, q_i	Share of reference and current data in bin i (PSI)

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary issue

Electricity use in buildings is one of the largest and fastest-growing components of energy demand. The International Energy Agency reports that buildings account for roughly 30% of global final energy consumption, and space cooling, water heating and appliances in homes are major drivers of evening peak demand [1]. In India, the domestic (household) sector consumes about one quarter of all electricity sold, and its share keeps rising with appliance ownership and air-conditioner adoption [2].

Three groups of stakeholders need short-term household consumption forecasts:

- **Households** want to know how much energy and money they will use tomorrow, so they can shift flexible loads (washing machines, water heaters) and avoid bill shock; time-of-day tariffs make this more valuable.
- **Distribution utilities** run demand-response and peak-shaving programmes and must forecast load at fine granularity. India's Revamped Distribution Sector Scheme targets the installation of about 250 million prepaid smart meters, which will generate exactly the kind of high-frequency data this project uses [3].
- **Smart-home and energy-management vendors** need a deployable, monitored prediction service rather than a one-off notebook model, which is the gap that MLOps practice addresses [4], [5].

Studies of production ML systems show that the model code is a small part of a real ML system and that most failures come from data dependencies, configuration and lack of monitoring [4], [23]. A consumption predictor that silently degrades when seasons or occupants change is a contemporary, documented issue.

1.2. Identification of Problem

Households and utilities lack a reliable, continuously maintained way to anticipate a single home's electricity demand over the next hours and to know in advance which hours will be peak hours. Household load is highly volatile, depends on human routines, weekday, season and appliance usage, and drifts over time. Meter readings contain gaps, noise and

extreme values. Models built once and left unattended become stale, and there is no feedback that tells the operator when this happens.

1.3. Identification of Tasks

The work was divided into the following tasks (Table 1.1), which also define the structure of this report:

Table 1.1*: Identified tasks*

#	Task	Output	Chapter
T1	Survey of the problem, existing forecasting methods and MLOps literature	Problem definition, objectives	1, 2
T2	Selection of a public, realistic smart-meter dataset and EDA	Data understanding, figures	3.1
T3	Design of features, constraints (incl. ethics) and alternative designs	Selected design	3
T4	Data pipeline: ingestion, cleaning, outliers, resampling, features, versioning with DVC	Versioned datasets	3.6, 4.1
T5	Model development: 9 algorithms, GridSearchCV tuning, over-fitting analysis, MLflow registry	Registered forecaster	4.1
T6	Peak-hour classifier with accuracy, precision, recall, F1, ROC	Peak alert model	4.1
T7	Deployment: Flask REST API, Docker, Kubernetes, Vercel cloud, React web app	Running services	4.1
T8	Automation and operations: CI/CD, tests, Prometheus, Grafana, logging, drift monitoring	CI pipeline, dashboards	4.1
T9	Responsible-AI analysis: fairness, transparency, privacy, security	Model card, bias analysis	3.2, 4.1
T10	Validation, documentation, report and user manual	This report	4, 5, Manual

1.4. Timeline

The project was executed over twelve weeks. Figure 1.1 shows the Gantt chart; tasks overlap where the pipeline allowed parallel work (for example, the web application was built while CI was configured).

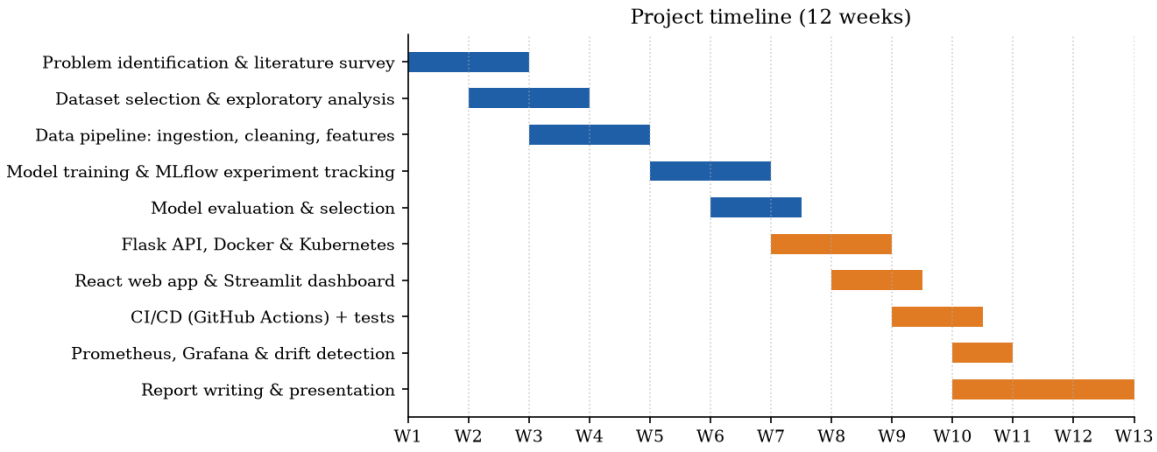


Figure 1.1*: Gantt chart of the project timeline*

1.5. Organization of the Report

- **Chapter 1** introduces the need, the problem, the tasks and the timeline.
- **Chapter 2** reviews the history of load forecasting and existing statistical, machine-learning and deep-learning solutions, compares them, and defines the problem and objectives mapped to the course outcomes.
- **Chapter 3** evaluates features and constraints (including ethical ones), compares two alternative designs, selects one and presents the implementation methodology.
- **Chapter 4** presents the implementation and validates the results: model comparison, tuning, classification metrics, deployment, CI/CD, monitoring and the responsible-AI analysis.
- **Chapter 5** concludes, discusses deviations from expectations and lists future work.
- References, Appendix and a step-by-step User Manual follow.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Table 2.1*: Timeline of the reported problem*

Period	Milestone
1970s	Box–Jenkins ARIMA methodology becomes the standard for utility-level load forecasting [6].
1992	Hart introduces Non-Intrusive Load Monitoring, showing household appliance loads can be inferred from aggregate meter data [7].
2000s	Machine-learning regressors (SVR, Random Forest [8], Gradient Boosting [9]) applied to building energy prediction.
2012	UCI publishes the Individual Household Electric Power Consumption dataset, which becomes a benchmark for household-level forecasting [10].
2015	Sculley et al. document "hidden technical debt" in production ML systems [4].
2016–2019	Deep learning (LSTM, CNN-LSTM) applied to household load forecasting [11], [12]; reviews of data-driven building energy prediction published [13].
2018	MLflow released to manage the ML lifecycle (tracking, projects, registry) [14].
2016–present	Probabilistic forecasting becomes the norm for load forecasting [15]; national smart-meter roll-outs (e.g. India's RDSS) make household data abundant [3].
2023	MLOps formally defined and its architecture surveyed in IEEE Access [5]; India enacts the Digital Personal Data Protection Act, relevant to smart-meter data [25].

2.2. Existing solutions

Statistical time-series models. ARIMA and exponential smoothing [6], [16] model linear autocorrelation and seasonality. They are interpretable and cheap but assume stationarity and struggle with the abrupt, appliance-driven spikes of a single household.

Classical machine learning. The standard supervised-learning toolkit, namely linear regression, decision trees, k-nearest neighbours, support vector machines and ensembles [21], [22], has been applied to building energy. Fan et al. combined data-mining techniques to predict next-day building consumption and peak demand [17]. Tree ensembles such as random forest [8] and gradient boosting [9], [18] learn non-linear interactions from engineered lag and calendar features and are robust to scale and outliers.

Deep learning. Marino et al. applied sequence-to-sequence LSTMs to the same UCI dataset [11]; Kim and Cho combined CNN and LSTM layers and reported lower error than

linear and LSTM baselines [12]. These models capture long dependencies but require more data, GPU training and careful tuning.

MLOps platforms. Kreuzberger et al. describe the components of a production ML system: versioned data and code, experiment tracking, model registry, automated pipelines, serving and monitoring [5]. Ameisen frames the same lifecycle from a product perspective, from prototype to monitored deployment [23]. MLflow [14], DVC and drift-detection methods [19] are widely used building blocks.

The following paragraphs summarise the theory of every algorithm used in this project, following the course textbooks [21]–[23]. Throughout, x_t is the feature vector of hour t and y_t the consumption to predict.

Linear regression. Models $\hat{y} = w \cdot x + b$ and chooses w, b to minimise the sum of squared errors $\Sigma(y_t - \hat{y}_t)^2$ (ordinary least squares). It is fast and interpretable, since each weight is the change in consumption per unit of a feature, but it can only represent linear relationships, so interactions such as "evening AND winter" must be engineered by hand. Inputs are standardised so the weights are comparable.

Ridge regression (regularisation). Adds a penalty $\alpha \cdot \Sigma w^2$ to the least-squares loss. Large α shrinks the weights towards zero, which reduces variance and the effect of correlated features (the lag and rolling features here are strongly correlated) at the cost of some bias. α is chosen by cross-validation.

Decision tree. Recursively splits the data on the feature and threshold that most reduce the mean squared error of the target in the two child nodes (the CART algorithm); a leaf predicts the mean of its training rows. Trees capture non-linear rules and need no scaling, but an unrestricted tree keeps splitting until every leaf is pure and memorises noise. max_depth and min_samples_leaf limit this and act as regularisation. For classification, splits minimise Gini impurity instead.

K-nearest neighbours (KNN). Predicts the average target of the k training rows closest to the new row in Euclidean distance (optionally weighted by 1/distance). It makes no assumptions about the shape of the relationship, but it is sensitive to feature scale (so inputs are standardised), slow to predict on large data, and small k over-fits while large k under-fits.

Support vector machine for regression (SVR). Fits a function that stays within ε of as many targets as possible while keeping the model as flat as possible; only points outside

the ε-tube (support vectors) influence it. The RBF kernel lets it learn non-linear functions. C trades flatness against errors outside the tube. Training cost grows roughly quadratically with the number of rows, which is why SVR is the slowest model here.

Random forest. Averages many decision trees, each grown on a bootstrap sample of the rows and choosing each split from a random subset of features (bagging) [8]. Averaging de-correlated trees reduces variance sharply, so the forest generalises far better than a single tree while keeping its non-linearity.

Gradient boosting. Builds trees sequentially; each new tree is fitted to the residual errors (the negative gradient of the loss) of the current ensemble, and its prediction is added with a small learning rate η [9]. The histogram variant bins each feature into at most 255 values, which makes split finding very fast on tens of thousands of rows [18]. Boosting reduces bias step by step, while the learning rate, number of leaves and L2 penalty control over-fitting.

Logistic regression. For the peak-hour classifier, models the probability of the positive class as $\sigma(w \cdot x + b)$, where σ is the sigmoid function, and is fitted by maximising the log-likelihood. It serves as the linear reference for the decision-tree classifier.

Model evaluation. Because the data is a time series, ordinary k-fold cross-validation would train on the future and test on the past. TimeSeriesSplit instead creates expanding windows: fold 1 trains on the first sixth of the training period and validates on the next sixth, fold 5 trains on five sixths and validates on the last. GridSearchCV evaluates every hyper-parameter combination on these folds and keeps the one with the best mean score [21], [22].

MLOps building blocks. Version control (Git) tracks code; data and pipeline versioning (DVC) tracks datasets, parameters and outputs by content hash, so any result can be reproduced from a commit. Experiment tracking and model registry (MLflow) records every run's parameters, metrics and model artifacts and numbers model versions. Containerisation (Docker) packages the code with its exact dependencies, and orchestration (Docker Compose, Kubernetes) runs and scales several containers as microservices. CI/CD (GitHub Actions, Vercel) automatically tests, builds and deploys on every change; continuous training re-runs the pipeline on a schedule. Monitoring (Prometheus collects time-series metrics by scraping HTTP endpoints; Grafana visualises

them) and drift detection close the loop by telling the team when the model needs retraining [5], [23].

2.3. Bibliometric analysis

Table 2.2*: Bibliometric comparison of existing solutions*

Work	Approach	Key features	Effectiveness	Drawbacks
Box & Jenkins [6]	ARIMA	Linear autoregression, differencing	Good for aggregate, smooth loads	Poor on volatile single-house data; manual order selection
Fan et al. [17]	Ensemble data mining	Feature selection + multiple regressors	Accurate next-day building forecasts	Commercial buildings; no deployment
Marino et al. [11]	LSTM seq2seq	Learns temporal patterns directly	Competitive on UCI data at hourly scale	Overfits at minute scale; heavy training
Kim & Cho [12]	CNN-LSTM	Convolutional feature extraction + LSTM	Low error on UCI dataset	Black box; GPU cost; no monitoring
Amasyali & El-Gohary [13]	Review	Survey of 60+ studies	Shows ML methods outperform statistical ones in most cases	Notes lack of deployed, maintained systems
Kreuzberger et al. [5]	MLOps architecture	CI/CD/CT, registry, monitoring	Reference architecture	Not specific to energy forecasting

2.4. Review Summary

The literature shows that (i) machine-learning models with good lag and calendar features consistently beat purely statistical models on household data; (ii) deep networks can be marginally more accurate but at much higher training and operational cost; and (iii) almost all published household forecasting work stops at an offline accuracy number, with no serving, automation, monitoring or ethical review. This project therefore compares the standard algorithms under one protocol, tunes them, and wraps the best one in a complete MLOps pipeline, evaluated honestly against naive baselines on a chronological hold-out.

2.5. Problem Definition

What is to be done: build a system that, given the most recent week of hourly smart-meter readings for a household, forecasts the household's mean active power (kW) for each of the

next 1–48 hours with a prediction interval, flags likely peak hours, and can be retrained, tested, deployed and monitored automatically.

How it is to be done: using the UCI household dataset, a reproducible DVC pipeline, supervised regression and classification models compared and tuned under MLflow tracking, a Flask REST API packaged with Docker and deployed to the cloud, a GitHub Actions CI/CD workflow, and Prometheus/Grafana and PSI-based monitoring.

What is not to be done: appliance-level disaggregation (NILM), control of household devices, billing integration, or collection of new personal data. Forecasts are for a single household and are not intended for grid-level dispatch or decisions about individuals.

2.6. Goals/Objectives

1. Clean and resample the 2.07 M-row dataset into a gap-free hourly series with an auditable data report, including missing-value and outlier analysis.
2. Engineer at least 25 features that use only past information (verified by an automated leakage test).
3. Train, tune (GridSearchCV) and compare the course algorithms, namely linear regression, decision tree, KNN, SVM and random forest, under experiment tracking on a strict chronological split.
4. Beat the persistence baseline on the held-out test period by at least 15% in MAE.
5. Build a peak-hour classifier and report accuracy, precision, recall, F1, the confusion matrix and ROC.
6. Serve predictions through a Flask REST API with input validation and response time under 1 s, containerised with Docker and deployed to a cloud platform.
7. Automate pipeline, quality gates, tests and container build with CI/CD; version data and models with DVC.
8. Monitor the service with Prometheus and Grafana, log predictions, and detect drift (PSI > 0.2).
9. Analyse fairness, transparency, privacy and security of the system.

Table 2.3 maps these objectives to the course outcomes of MLOps-1 (25CSH-320).

Table 2.3*: Mapping of project objectives to course outcomes*

Course outcome	How the project addresses it	Section
CO1: ML fundamentals, types, applications	Supervised regression and classification applied to a real energy-sector problem; algorithm survey	1.1, 2.2
CO2: Preprocessing,	Missing values, IQR outliers, resampling, scaling, 31	3.1, 3.3, 4.1

Course outcome	How the project addresses it	Section
feature engineering, data handling	engineered features, chronological split	
CO3: Develop and evaluate models with various algorithms	9 regressors and 3 classifiers, GridSearchCV with time-series CV, over/under-fitting analysis, MSE/MAE/R^2 and accuracy/precision/recall/F1/ROC	3.6, 4.1
CO4: Deploy with APIs, microservices and cloud	Flask REST API, Docker Compose microservices, Kubernetes manifests, Vercel cloud deployment	4.1
CO5: CI/CD and ethical considerations	GitHub Actions + DVC + Vercel CD; fairness, transparency, privacy and security analysis; model card	3.2, 4.1

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Dataset. The UCI dataset [10] contains 2,075,259 one-minute measurements from 2006-12-16 to 2010-11-26 for one house in Sceaux, France: global active and reactive power, voltage, current intensity and three sub-meters (kitchen; laundry room; electric water heater and air conditioner). It was chosen because it is public, long enough to cover four winters, contains real gaps, and mirrors the load-profile channels exposed by IEC 62056 smart meters. It is stored as a single CSV file versioned with DVC. Figures 3.1–3.3 summarise the exploratory analysis.

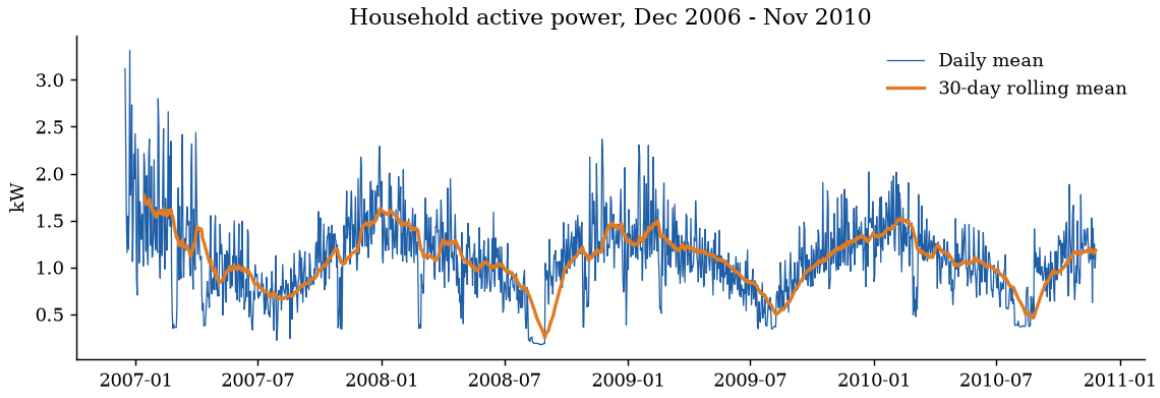


Figure 3.1*: Daily mean active power with 30-day rolling mean (strong yearly seasonality)*

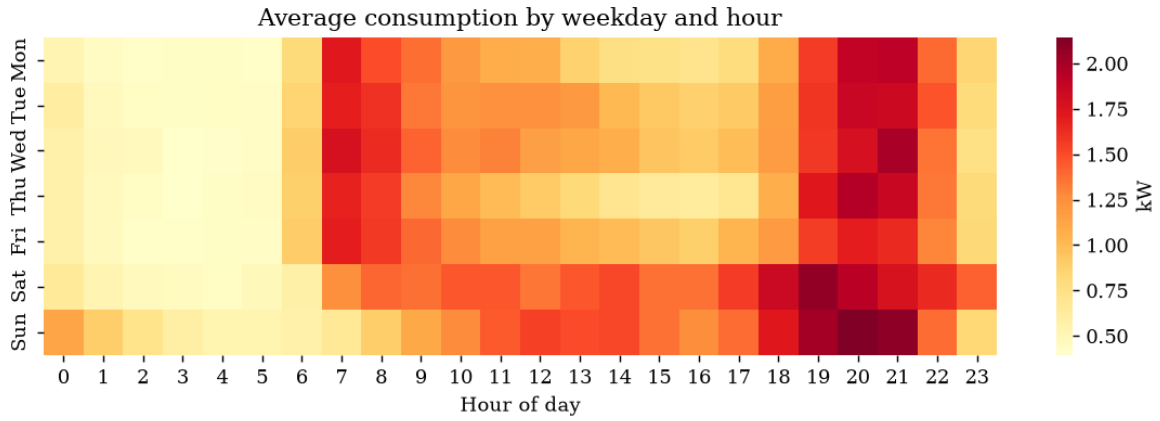


Figure 3.2*: Average consumption by weekday and hour (morning and evening peaks)*

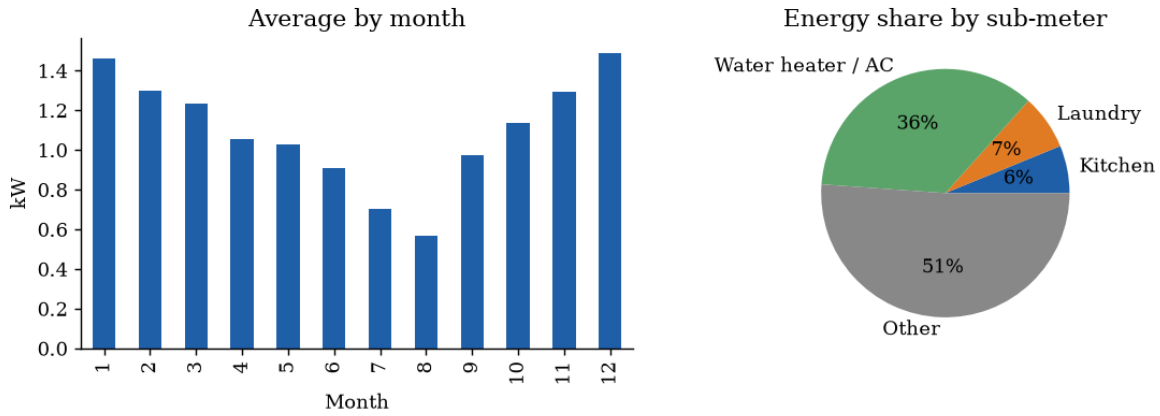


Figure 3.3*: Monthly pattern and share of energy per sub-meter*

The EDA shows a strong winter peak (consumption roughly doubles from August to December), two daily peaks around 07:00 and 20:00–21:00, higher daytime use at weekends, and an August holiday dip.

Figures 3.4–3.10 extend the analysis. Missing readings are concentrated in a few months, the worst being 2010-08 with 16.19% missing (long outages), which justified the two-step gap filling below. All variables are right-skewed, with many low-consumption hours and a long tail of peaks. The autocorrelation of hourly consumption is 0.713 at one hour, with repeating peaks every 24 hours (0.436 at 24 h, 0.466 at one week), which is why the lag features at 1, 24 and 168 hours were chosen.

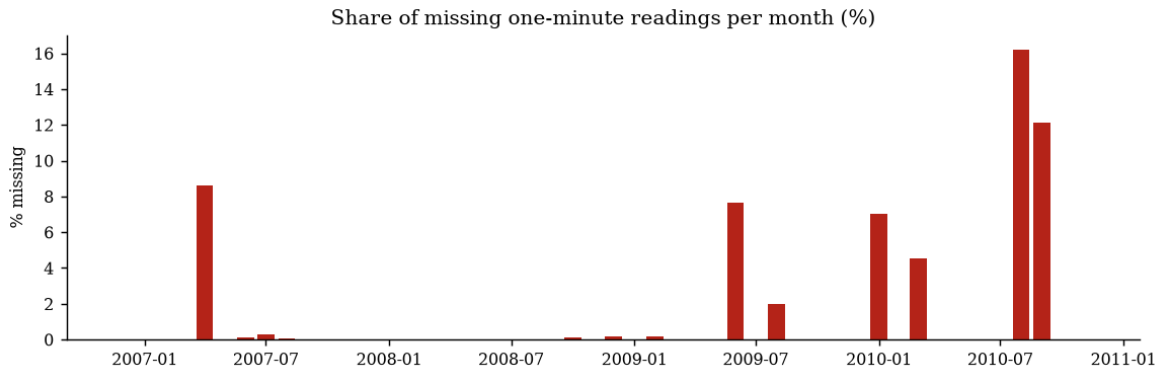


Figure 3.4*: Share of missing one-minute readings per month*

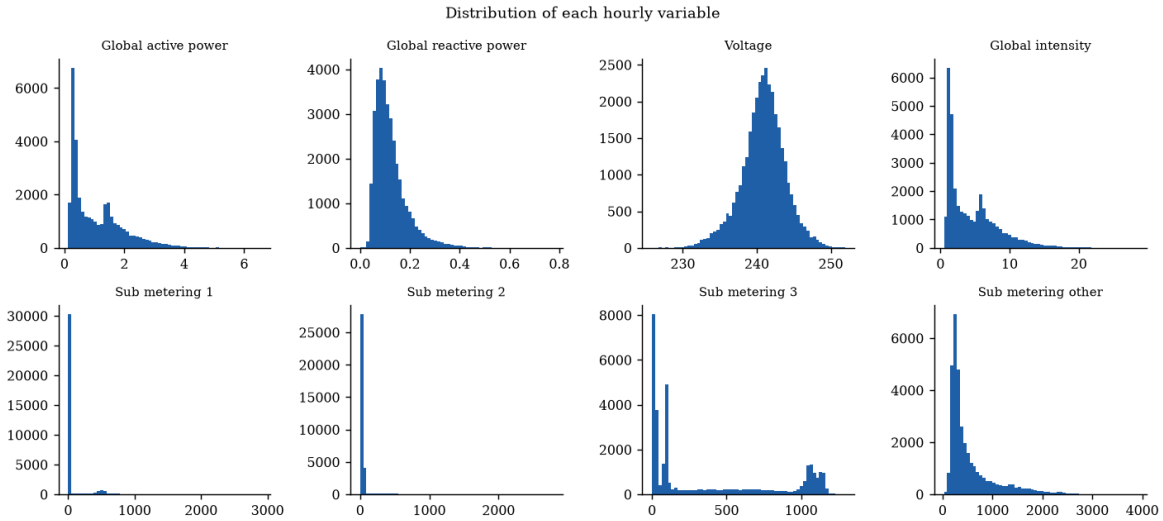


Figure 3.5*: Distribution of each hourly variable*

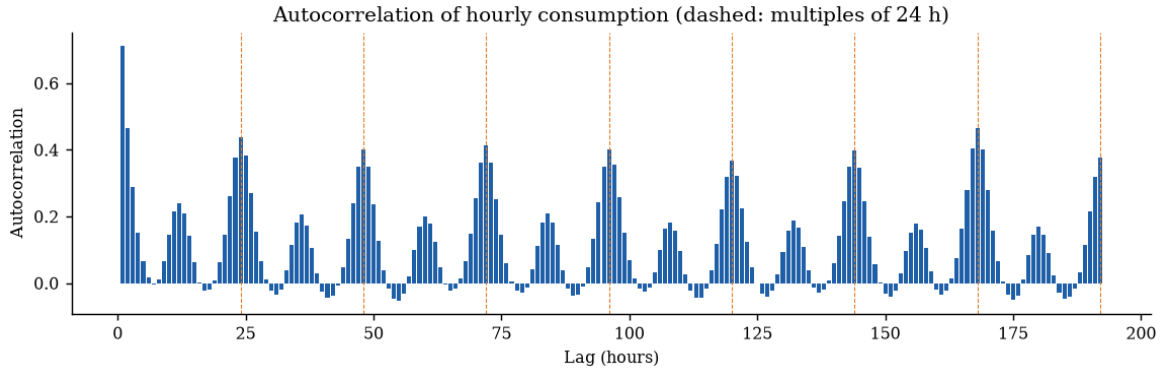


Figure 3.6*: Autocorrelation of hourly consumption up to eight days*

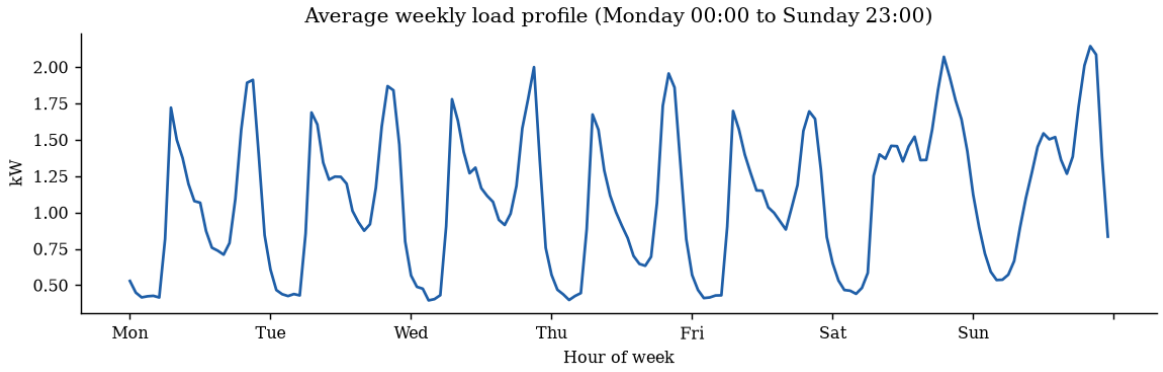


Figure 3.7*: Average weekly load profile*

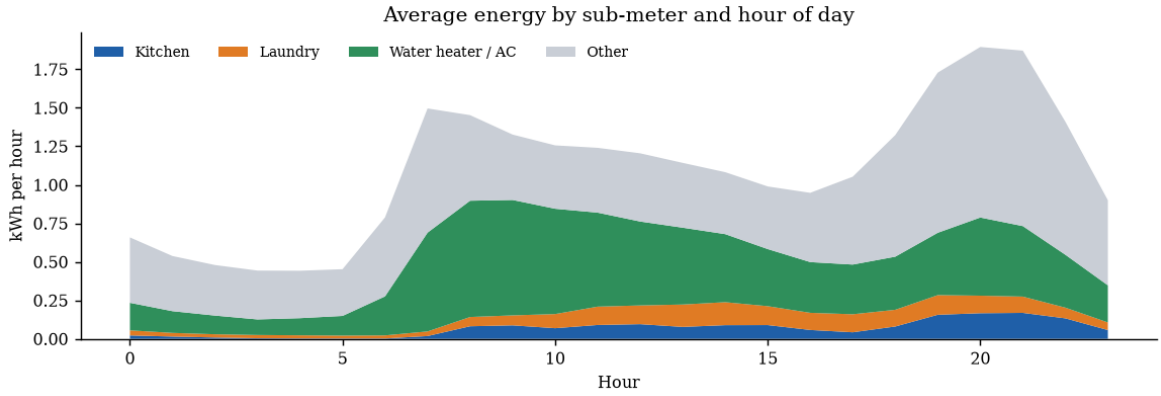


Figure 3.8*: Average energy by sub-meter and hour of day*

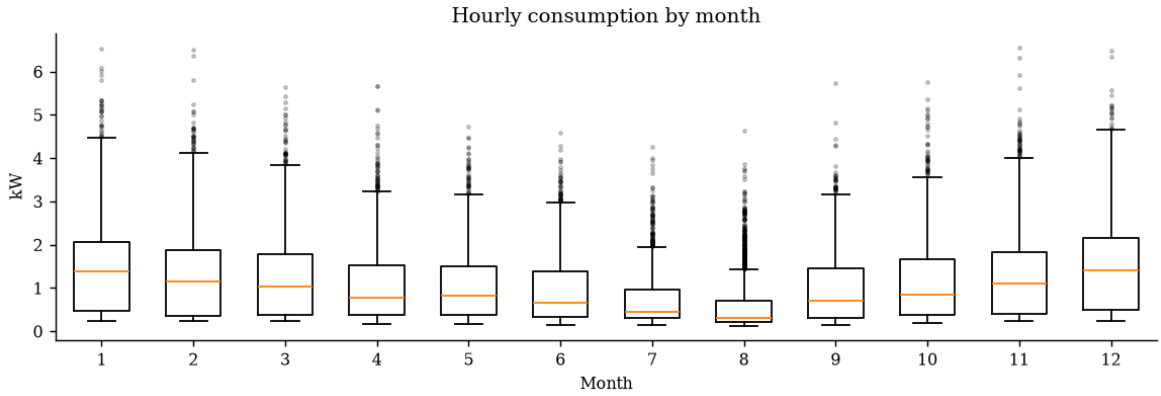


Figure 3.9*: Hourly consumption by month (box plots)*

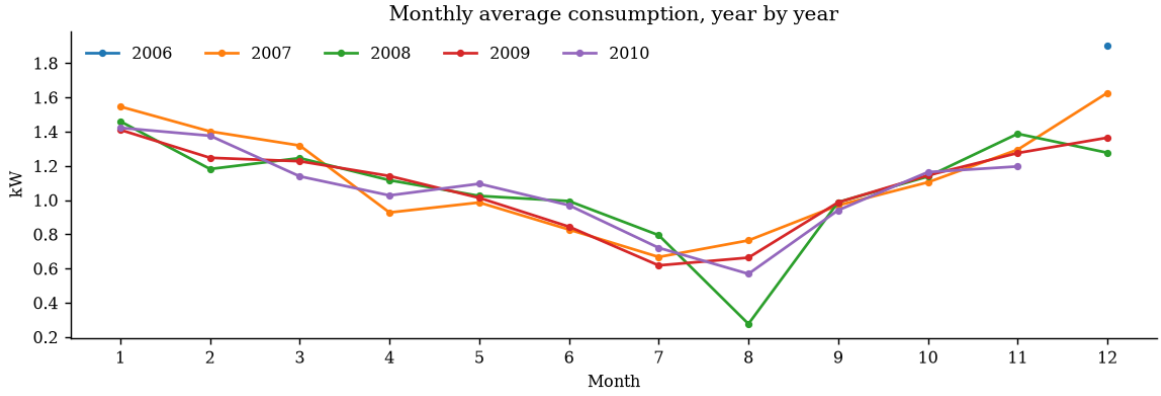


Figure 3.10*: Monthly average consumption, year by year*

Table 3.1*: Descriptive statistics of the hourly data*

Variable	Mean	Std	Min	25%	Median	75%	Max
Global active power	1.089	0.896	0.124	0.341	0.800	1.577	6.561
Global reactive power	0.124	0.067	0.000	0.078	0.107	0.149	0.774
Voltage	240.845	2.997	225.835	239.197	240.985	242.694	251.902
Global intensity	4.618	3.752	0.503	1.510	3.410	6.590	28.383
Sub metering 1	67.337	212.318	0.000	0.000	0.000	0.000	2902.000
Sub metering 2	77.757	251.090	0.000	0.000	20.000	39.000	2786.000
Sub metering 3	386.775	440.533	0.000	39.000	99.000	848.000	1293.000

Variable	Mean	Std	Min	25%	Median	75%	Max
Sub metering other	556.394	494.912	27.600	260.667	343.700	662.567	3890.233

Outliers. With the inter-quartile-range rule (outside Q1 − 1.5·IQR … Q3 + 1.5·IQR), 738 hourly values of active power (2.13%) lie above 3.431 kW (Figure 3.11). These are genuine peak-usage hours (heaters, ovens), exactly the hours the forecaster must learn, so they are kept rather than removed. Instead, distance- and margin-based models (KNN, SVM) receive standardised inputs so that large-valued columns do not dominate.

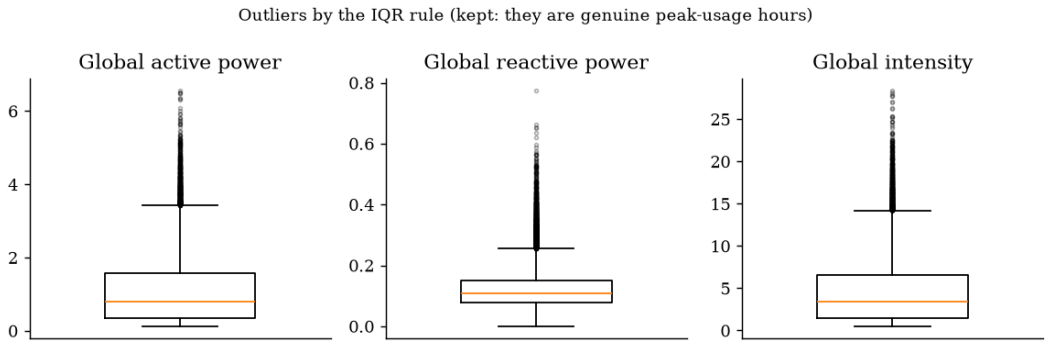


Figure 3.11*: Box plots of hourly readings; points beyond the whiskers are IQR outliers*

These findings drove the feature specification in Table 3.2.

Table 3.2*: Selected feature specification (31 features)*

Feature group	Features	Rationale
Autoregressive lags	lag 1, 2, 3, 6, 12, 24, 48, 168 h	Short-term persistence, same hour yesterday and last week
Rolling statistics	mean and std over 3, 6, 24, 168 h	Recent level and volatility
Calendar	hour, weekday, month, weekend, French public holiday	Routines and seasonality
Cyclical encodings	sin/cos of hour and month	23:00 and 00:00 are neighbours
Electrical context (t−1)	reactive power, voltage, intensity, 3 sub-meters	Which appliances were active

3.2. Design Constraints

Table 3.3*: Design constraints*

Constraint	Consideration in the design
Standards	ML life-cycle stages follow ISO/IEC 23053; software quality attributes of ISO/IEC 25010 (reliability, maintainability) drive testing and modularity; metrics exposed in the OpenMetrics format.
Economic / Cost	Must run on a student laptop, free CI runners and a free cloud tier: no GPU, only open-source tools (scikit-learn, MLflow, DVC, Flask, Docker, Prometheus, Grafana).
Environmental	Low-compute models reduce the energy footprint of retraining; the purpose of the system is itself to support energy saving (ISO 50001

Constraint	Consideration in the design
	spirit).
Health & Safety	Advisory system only: it never switches appliances, so incorrect forecasts cannot cause physical harm.
Professional / Ethical	Honest evaluation against naive baselines on unseen future data; no data leakage; limitations and error differences between groups of hours are disclosed (model card).
Fairness & bias	The data comes from one French household (2006–2010); quality of service is measured across day type, time of day, season and usage level; the model is not claimed to generalise to other homes or climates without re-validation.
Privacy	Fine-grained meter data can reveal occupancy and routines. Only a public anonymised dataset is used; the prediction log stores forecasts, never submitted readings; deployment on real data must follow India's Digital Personal Data Protection Act, 2023 [25].
Security	All API input is validated (types, ranges, minimum history, duplicates); request size is limited; only models produced by this pipeline are loaded; rejected requests are monitored.
Manufacturability / Deployability	Must be reproducible on any machine: pinned configuration, DVC pipeline and lock file, Docker image, single-command reproduction.
Regulatory	Indian smart meters follow IS 16444; the API input schema maps to the hourly load profile such meters provide.

3.3. Analysis of Features and finalization subject to constraints

- **Removed:** minute-level forecasting. At one-minute resolution the series is dominated by appliance switching noise; hourly resolution matches billing and demand-response use and reduces data 60×.
- **Removed:** weather features. They would improve accuracy but no free, historical, hourly weather feed for Sceaux 2006–2010 could be integrated within the cost constraint.
- **Removed:** deep LSTM model as the production candidate, due to the no-GPU and explainability constraints (kept as future work).
- **Modified:** missing data handling. Gaps up to 6 h are time-interpolated; longer outages are filled with the same hour of the previous week to preserve the daily profile.
- **Modified:** outliers are reported and kept (they are real peaks); feature scaling is applied only where the algorithm needs it (linear models, KNN, SVM).
- **Added:** a peak-hour classifier, because demand response needs a yes/no alert as well as a number.
- **Added:** prediction intervals ($\pm 1.96\sigma_r\sqrt{h}$) so users can see forecast uncertainty.
- **Added:** automated leakage test, quality gates in CI, and a bias analysis across groups of hours to satisfy the professional/ethical constraints.

3.4. Design Flow

Two alternative designs were developed to meet the specification (Figure 3.12).

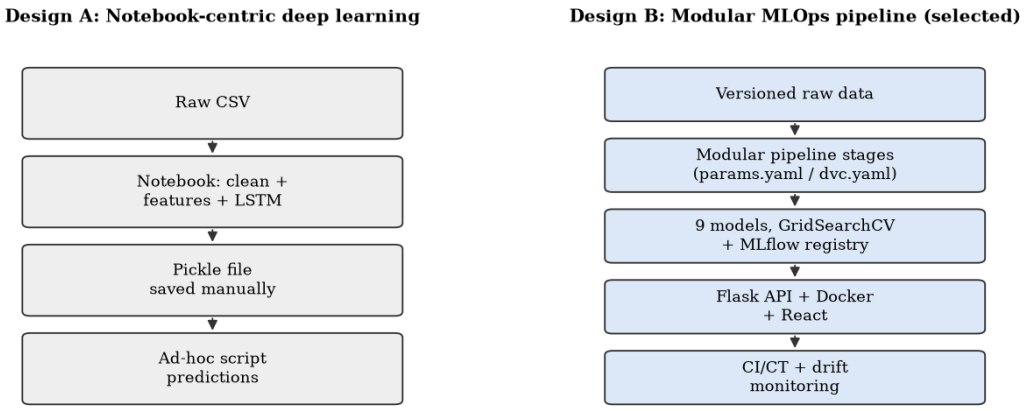


Figure 3.12*: Alternative designs considered*

Design A – Notebook-centric deep learning. All steps (cleaning, features, an LSTM model) live in a single Jupyter notebook. The trained network is pickled manually and predictions are produced with an ad-hoc script. This is the typical approach in the literature [11], [12].

Design B – Modular MLOps pipeline. Each stage is an independent, tested Python module driven by one configuration file (params.yaml) and declared as a DVC pipeline. All course algorithms are compared and tuned under MLflow tracking; the best is registered and served by a Flask REST API in Docker, deployed to the cloud behind a React web app; GitHub Actions automates the pipeline, and Prometheus, Grafana and a drift monitor close the loop.

3.5. Design selection

Table 3.4*: Comparison of alternative designs*

Criterion	Design A	Design B
Expected accuracy	High (with tuning)	High (random forest, a syllabus algorithm)
Training cost / hardware	GPU preferred, minutes–hours	CPU, about 4 min including tuning
Reproducibility	Low: hidden notebook state	High: params.yaml + DVC + Docker
Experiment tracking	Manual	Automatic (MLflow)
Deployment	Manual script	Flask API + container + cloud + web app
Testing / CI	None	11 automated tests, CI quality gates
Monitoring & retraining	None	Prometheus, Grafana, PSI

Criterion	Design A	Design B
		drift, weekly CT
Explainability & ethics	Low	Feature importance, bias analysis, model card
Fit with constraints	Violates cost & reproducibility	Satisfies all

Design B is selected. It meets every constraint in Section 3.2, its accuracy is competitive, and it delivers the operational capabilities (automation, deployment, monitoring, responsible AI) that are the core outcomes of the course and that the literature identifies as missing.

3.6. Implementation plan/methodology

Figure 3.13 shows the algorithm executed by the pipeline and Figure 3.14 the detailed block diagram.

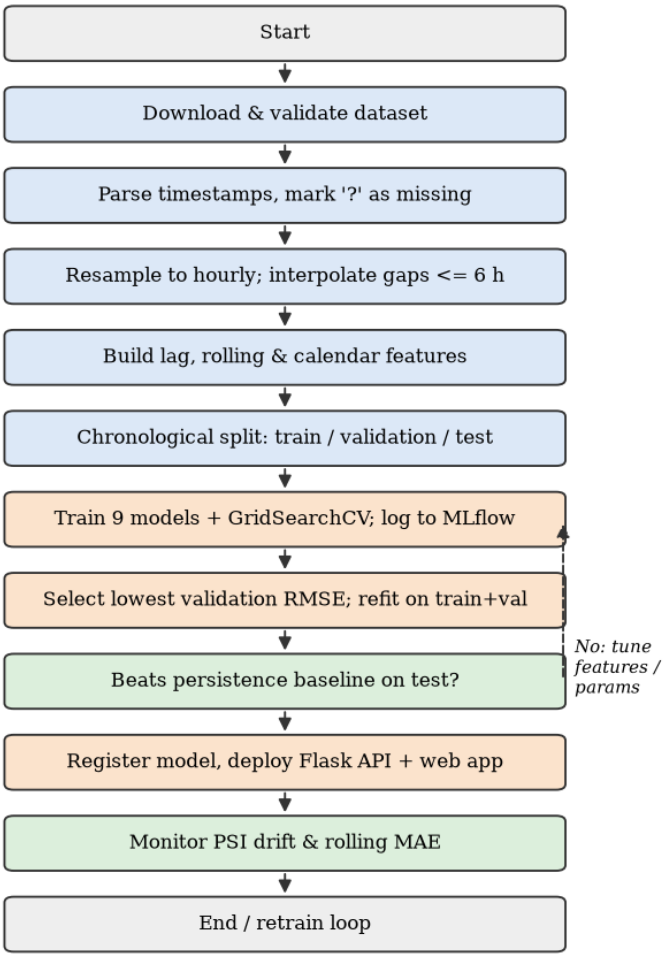


Figure 3.13*: Flowchart of the training and deployment algorithm*

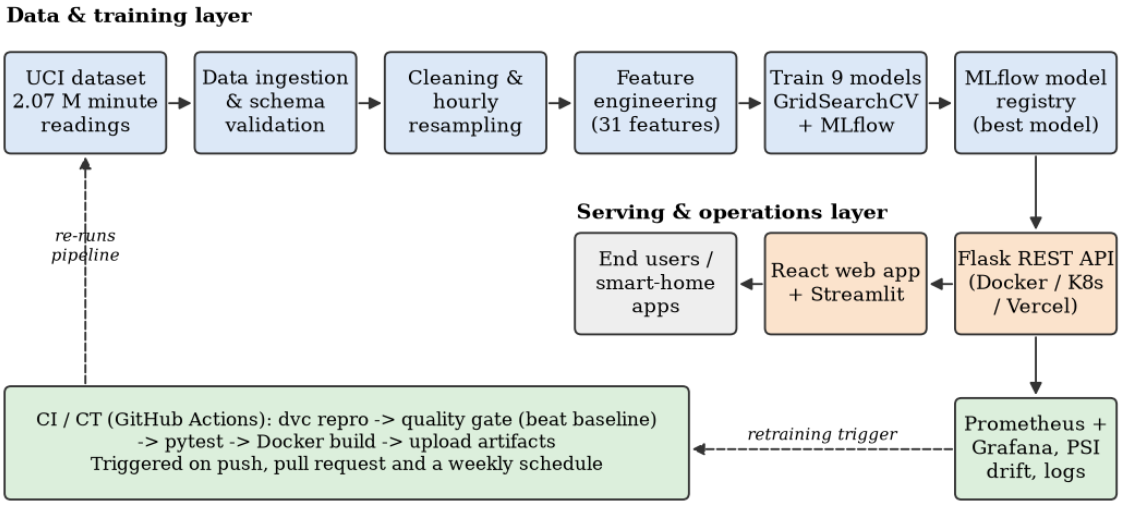


Figure 3.14*: Detailed block diagram of the MLOps architecture*

The system is further described by a context data-flow diagram (Figure 3.15), a level-1 data-flow diagram showing processes and data stores (Figure 3.16), a use-case diagram (Figure 3.17), a sequence diagram of a forecast request (Figure 3.18), the deployment diagram (Figure 3.19), the CI/CD pipeline (Figure 3.20) and the DVC stage graph (Figure 3.21).

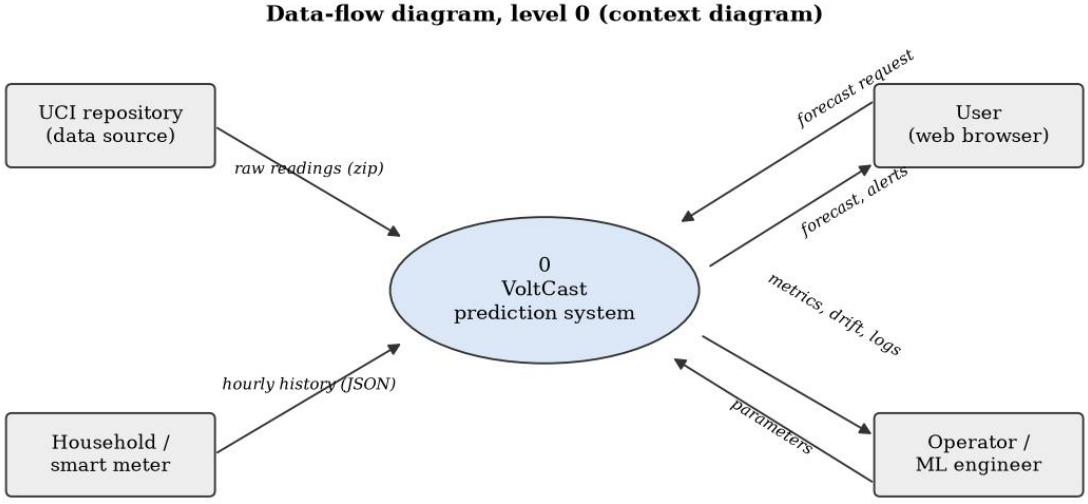


Figure 3.15*: Data-flow diagram, level 0 (context)*

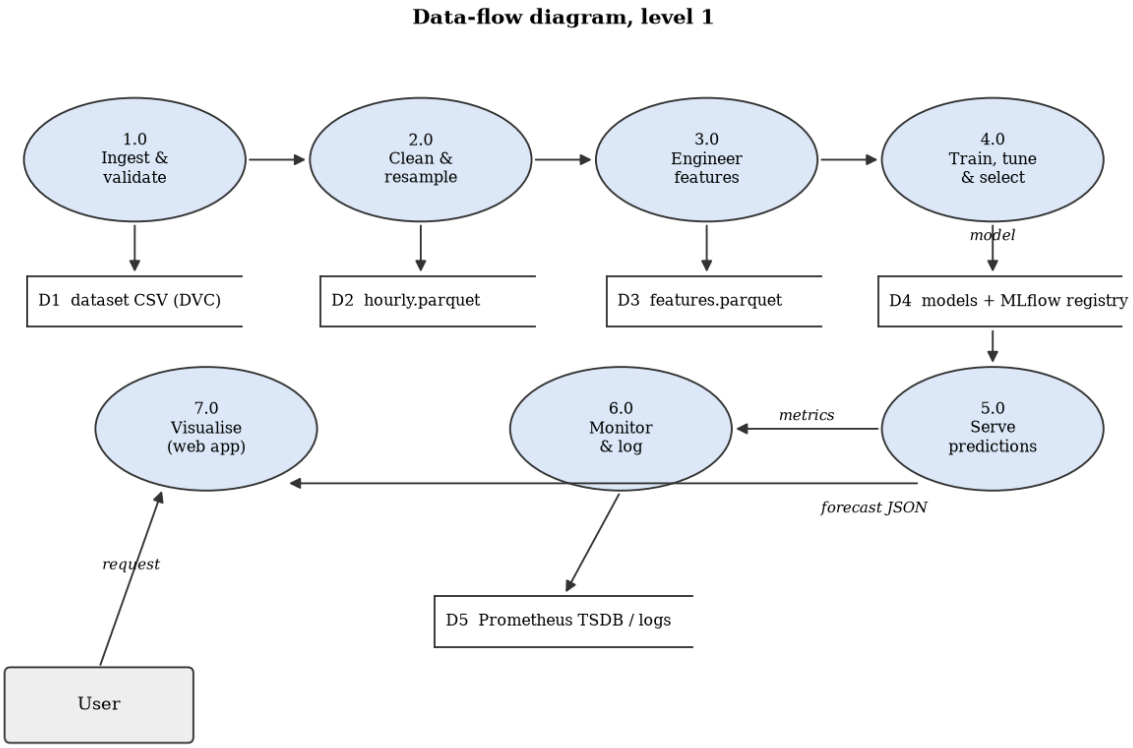


Figure 3.16*: Data-flow diagram, level 1*

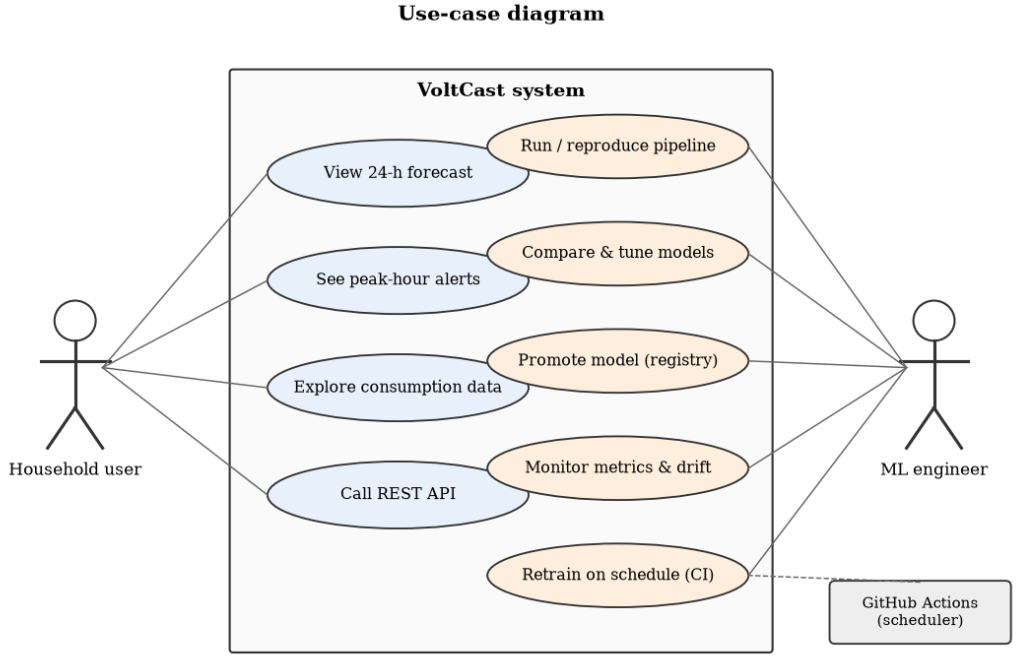


Figure 3.17*: Use-case diagram*

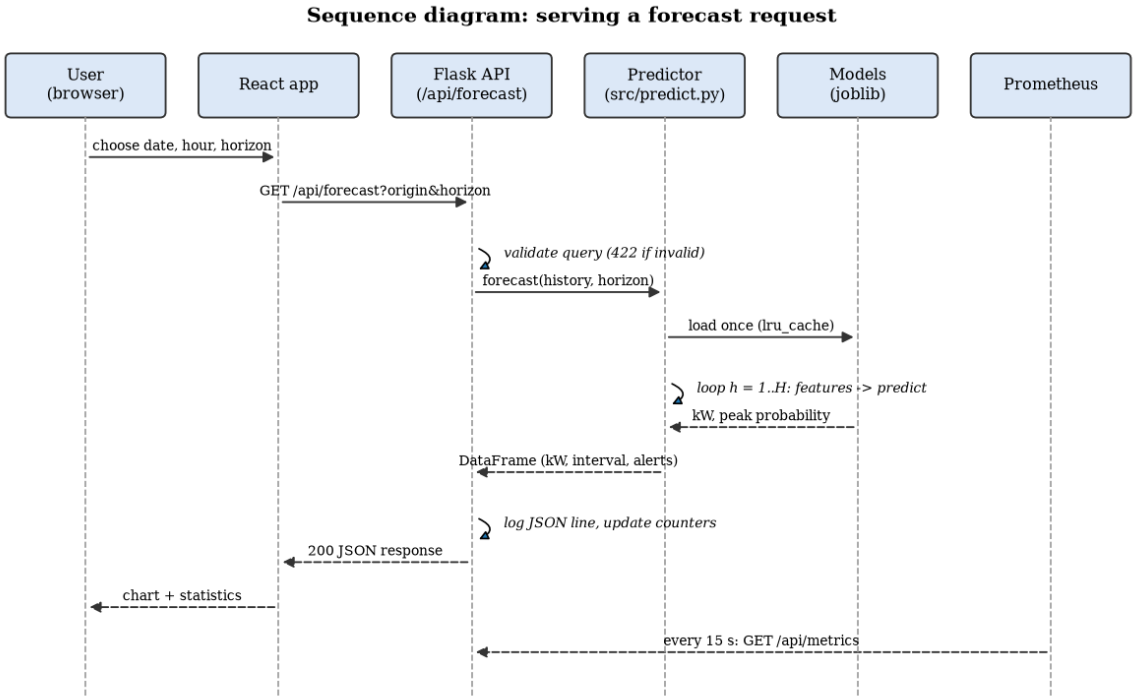


Figure 3.18*: Sequence diagram: serving a forecast request*

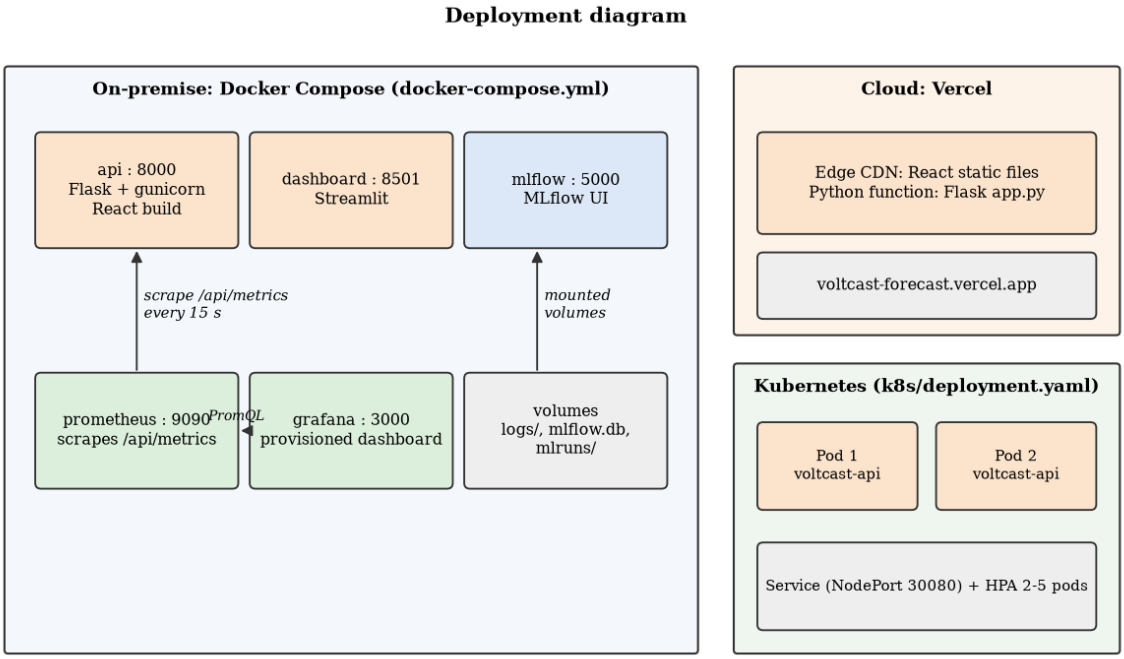


Figure 3.19*: Deployment diagram: Docker Compose, Vercel and Kubernetes*

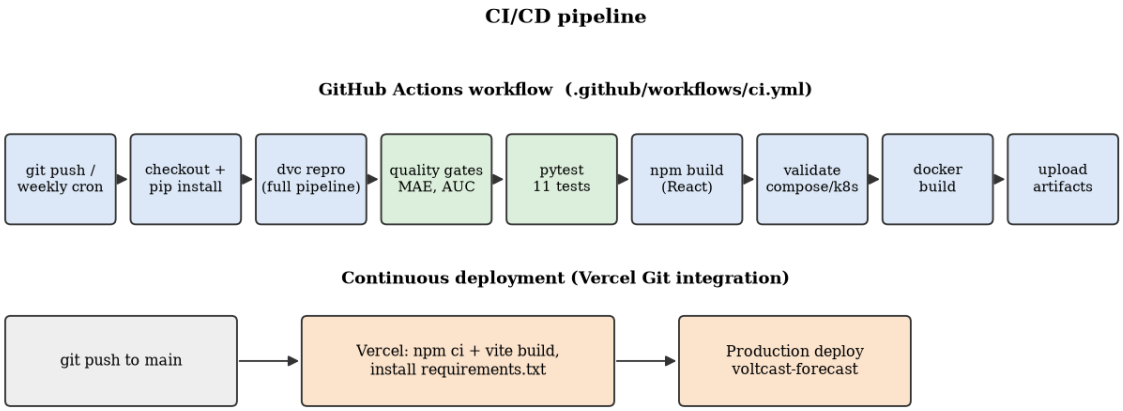


Figure 3.20*: CI/CD pipeline*

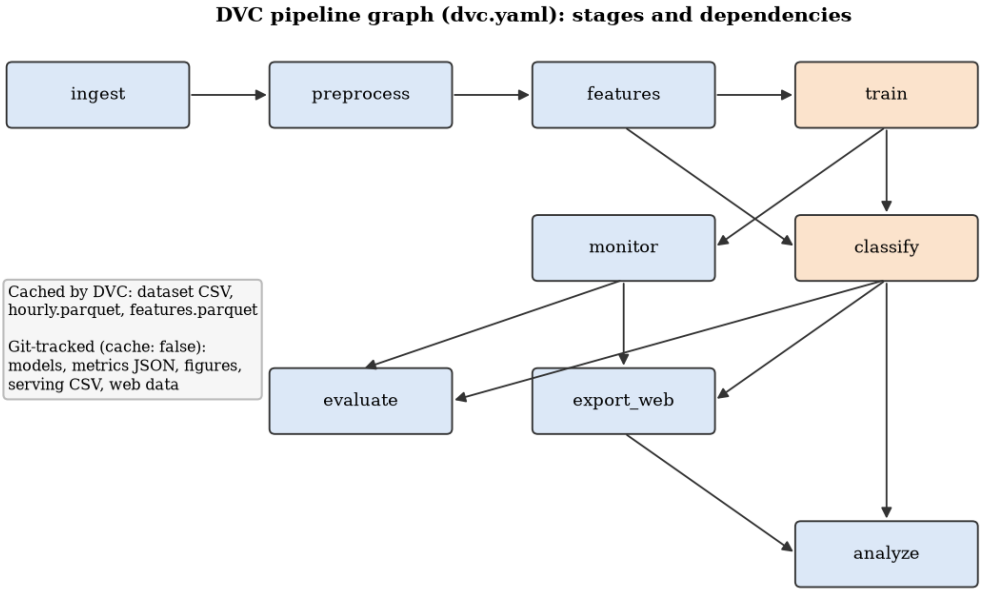


Figure 3.21*: DVC pipeline graph*

Each pipeline stage follows its own algorithm, shown as flowcharts in Figures 3.22–3.26.

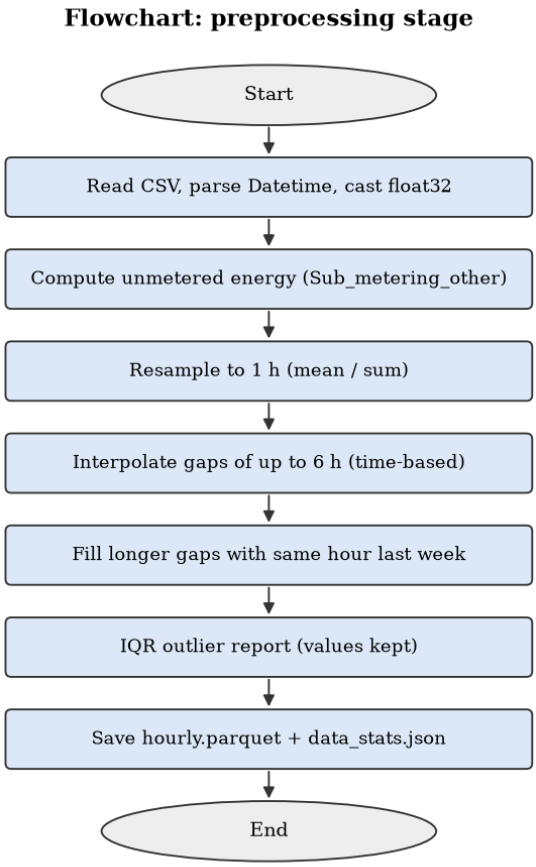


Figure 3.22*: Flowchart of the preprocessing stage*

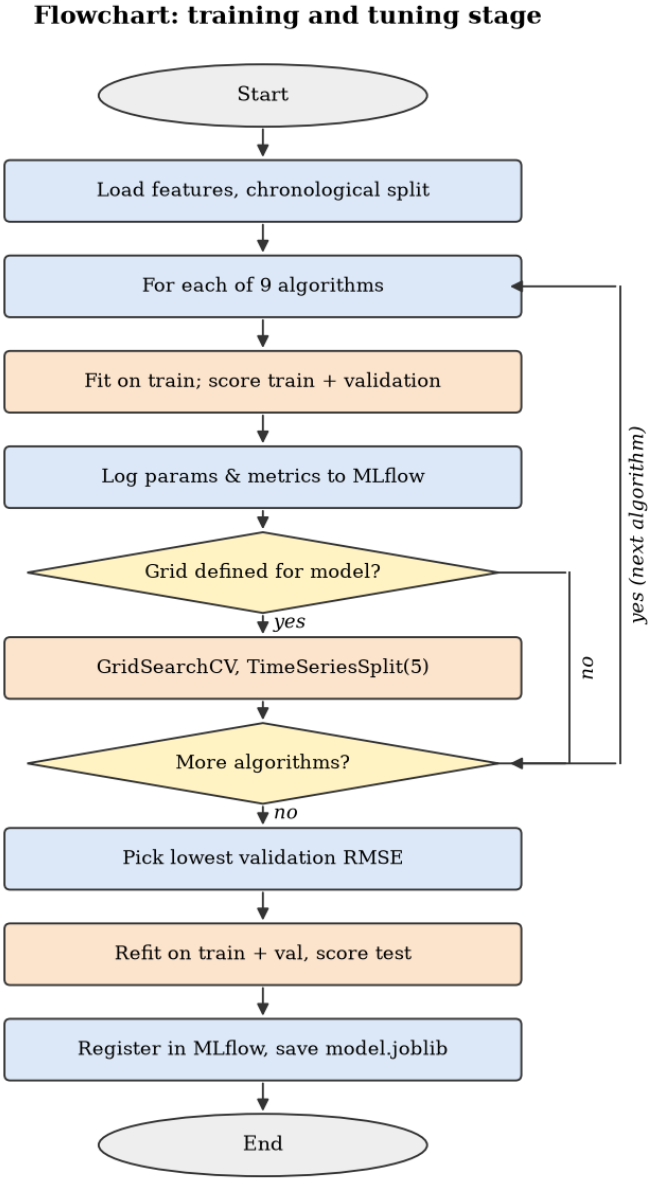


Figure 3.23*: Flowchart of the training and tuning stage*

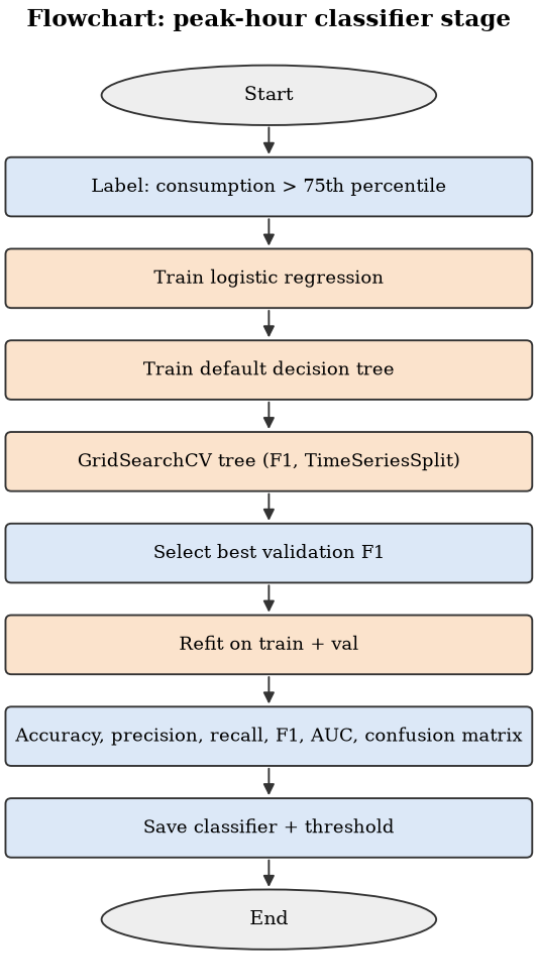


Figure 3.24*: Flowchart of the peak-hour classifier stage*

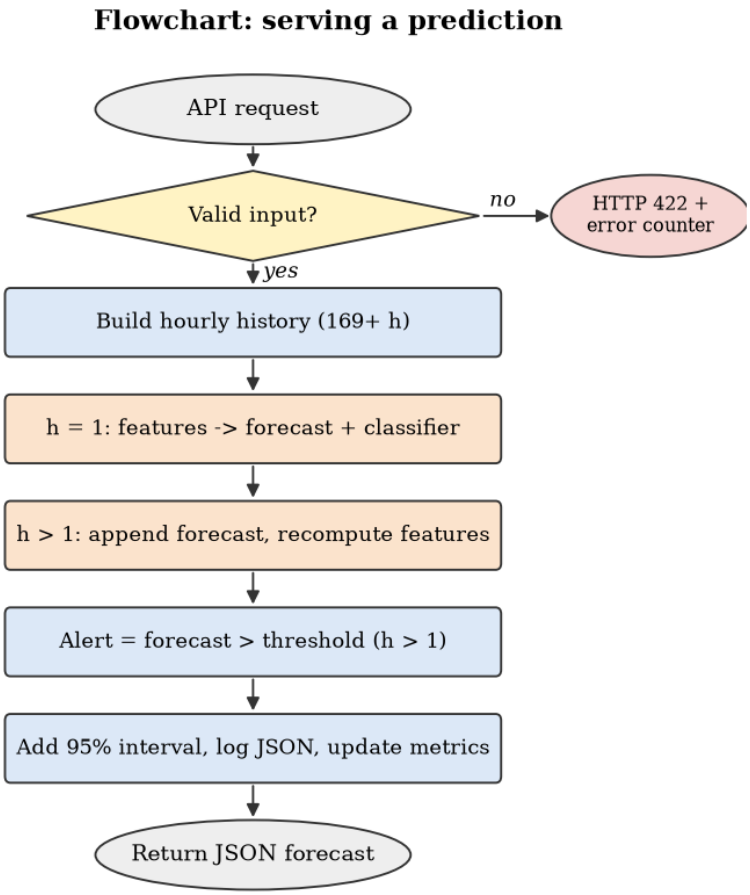


Figure 3.25*: Flowchart of serving a prediction*

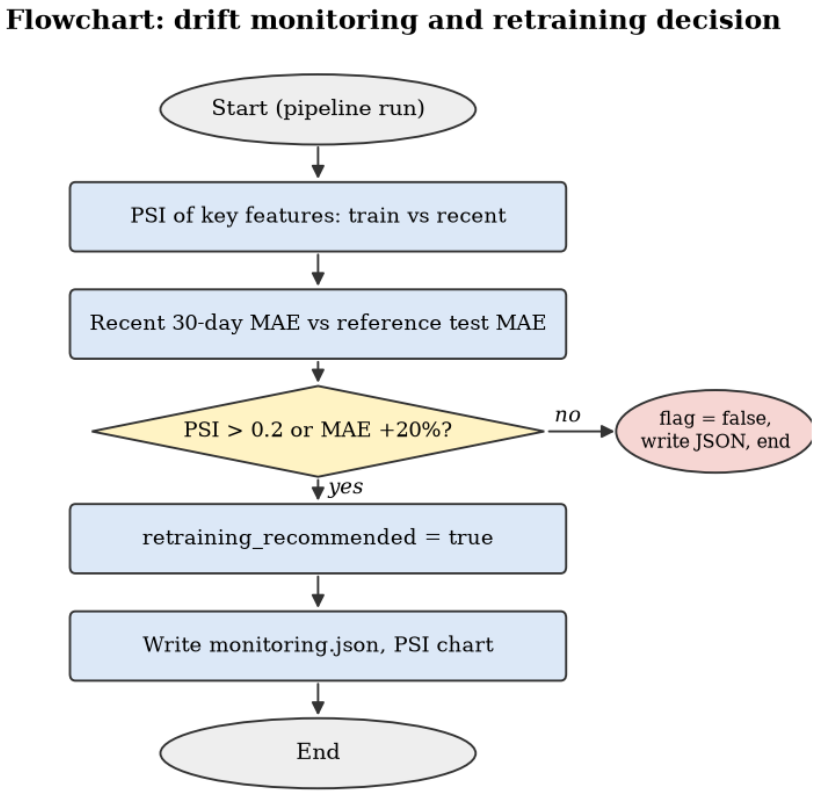


Figure 3.26*: Flowchart of drift monitoring and the retraining decision*

Data split. To respect time ordering, data is split chronologically: training up to 31 Dec 2009 (26,503 h), validation Jan–May 2010 (3,624 h) and test 2010-06-01 to 2010-11-26 (4,294 h). Models are selected on validation, refit on train+validation, and scored once on test. A random split is deliberately avoided: it would leak future information into training and overstate accuracy.

Regression models. Persistence ($\hat{y}_t = y_{t-1}$) and seasonal-naive ($\hat{y}_t = y_{t-24}$) baselines; linear regression and ridge regression (standardised inputs); decision tree; k-nearest neighbours ($k = 10$, standardised); support vector regression (RBF kernel, $C = 1$, $\varepsilon = 0.1$, standardised); random forest (200 trees, depth 16, minimum 5 rows per leaf) [8]; and, as a reference model outside the syllabus, histogram gradient boosting (500 iterations, $\eta = 0.05$, 31 leaves) [9], [18]. All are implemented with scikit-learn [20].

Model selection rule. All models are compared, but only the algorithms of the MLOps-1 syllabus (linear and ridge regression, decision tree, KNN, SVM and random forest) may be deployed. Among them the variant (default or tuned) with the lowest validation RMSE is chosen. Gradient boosting and the two naive baselines are reported as reference points, so the cost of this rule can be measured (Section 4.1.5).

Hyper-parameter tuning. GridSearchCV searches the grids in params.yaml using TimeSeriesSplit with five folds, so every fold trains on the past and validates on the following period, which is the time-series form of cross-validation [21]. Tuned and default variants are both compared on the validation window; regularisation is controlled through ridge α, tree depth and minimum leaf size.

Peak-hour classification. An hour is labelled a peak hour when consumption exceeds τ = 1.59 kW, the 75th percentile of training consumption. Logistic regression, a default decision tree and a GridSearchCV-tuned decision tree (F1 scoring, balanced class weights allowed) are compared on validation F1.

Metrics. Regression: MAE = $(1/n)\Sigma|y_t - \hat{y}_t|$, MSE = $(1/n)\Sigma(y_t - \hat{y}_t)^2$, RMSE = $\sqrt{MSE}$, R^2 and MAPE; RMSE selects the model because peak errors matter most for demand response. Classification: accuracy = (TP+TN)/n, precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = harmonic mean of precision and recall, plus the confusion matrix and ROC-AUC [21], [22].

Monitoring. PSI = $\Sigma(q_i - p_i)\cdot\ln(q_i/p_i)$ over 10 quantile bins of the training distribution; PSI > 0.2 signals significant drift. Retraining is recommended when any key feature drifts or recent 30-day MAE degrades more than 20%. At run time Prometheus scrapes request, latency, forecast and alert counters from the API every 15 s.

Table 3.5*: Software modules of the implementation*

Module	Responsibility
src/data_ingestion.py	Download the UCI archive, convert to CSV, verify schema
src/preprocess.py	Parse, fill gaps, resample hourly, IQR outlier report
src/features.py	Build 31 leakage-free features
src/train.py	9 regressors, GridSearchCV, MLflow tracking and model registry
src/classify.py	Peak-hour classifier and classification metrics
src/monitor.py	PSI drift and performance degradation, retraining decision
src/evaluate.py	EDA, diagnostics, validation curve, classification and bias figures
src/predict.py	Recursive multi-step forecasting with intervals and peak alerts
serving/main.py, app.py	Flask REST API, Pydantic validation, Prometheus metrics, prediction log
web/ (React), dashboard/app.py	Web application (Vercel) and Streamlit dashboard
monitoring/, docker-compose.yml, k8s/	Prometheus, Grafana, five-service stack, Kubernetes manifests
dvc.yaml, .github/workflows/ci.yml	Versioned pipeline and CI/CD automation
tests/test_pipeline.py	11 unit, leakage, classification, API and monitoring tests

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

Modern tools used. Analysis: pandas, NumPy, Matplotlib, Seaborn, Plotly. Design drawings: architecture, flow and Gantt diagrams generated in Matplotlib. Modelling: scikit-learn. Experiment management and versioning: MLflow (tracking and model registry), DVC, Git/GitHub. Deployment: Flask, gunicorn, Docker, Docker Compose, Kubernetes manifests, Vercel. Operations: GitHub Actions, Prometheus, Grafana. Testing and validation: pytest, PSI drift analysis. Report preparation: python-docx, generated directly from pipeline outputs, so every number in this chapter is reproducible with dvc repro.

4.1.1. Data preparation

1.252% of minute readings were missing ('?'). After cleaning and hourly resampling the series has 34,589 gap-free records with mean 1.09 kW (median 0.80 kW, max 6.56 kW). Feature building drops the first 168 h (needed for the weekly lag), leaving 34,421 modelling rows. Figure 4.1 shows that current intensity is almost perfectly correlated with active power, while voltage is weakly negatively correlated.

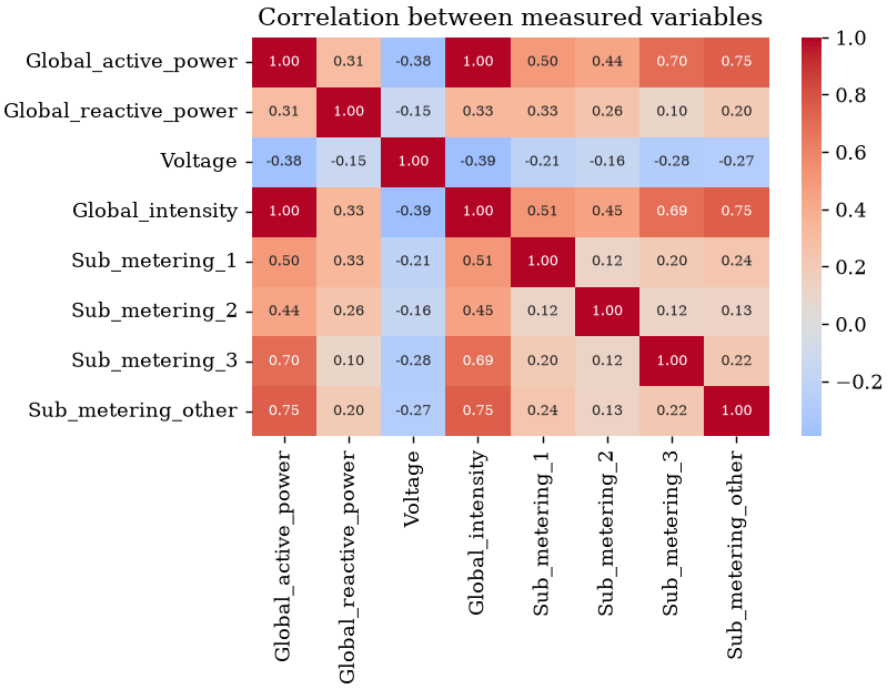


Figure 4.1: *Correlation matrix of the measured variables*

4.1.2. Model development and comparison

Table 4.1 compares all nine regression models with default hyper-parameters; every run is logged in MLflow (Figure 4.3 in this section shows the tracking UI).

Table 4.1: *Training and validation performance of all models (default hyper-parameters)*

Model	Train MAE	Val MAE	Val MSE	Val RMSE	Val R^2	Fit (s)
Persistence baseline	0.449	0.441	0.447	0.669	0.364	0.0
Seasonal naive (24 h)	0.663	0.636	0.832	0.912	-0.184	0.0
Linear regression	0.409	0.398	0.310	0.557	0.559	0.0
Ridge regression	0.409	0.398	0.310	0.557	0.559	0.0
Decision tree	0.000	0.506	0.602	0.776	0.142	0.8
K-nearest neighbours (KNN)	0.347	0.395	0.321	0.566	0.544	0.0
Support vector machine (SVR)	0.296	0.355	0.282	0.531	0.598	47.4
Random forest	**0.220**	**0.345**	**0.259**	**0.509**	**0.632**	**8.9**
Histogram gradient boosting	0.289	0.341	0.248	0.498	0.647	1.7

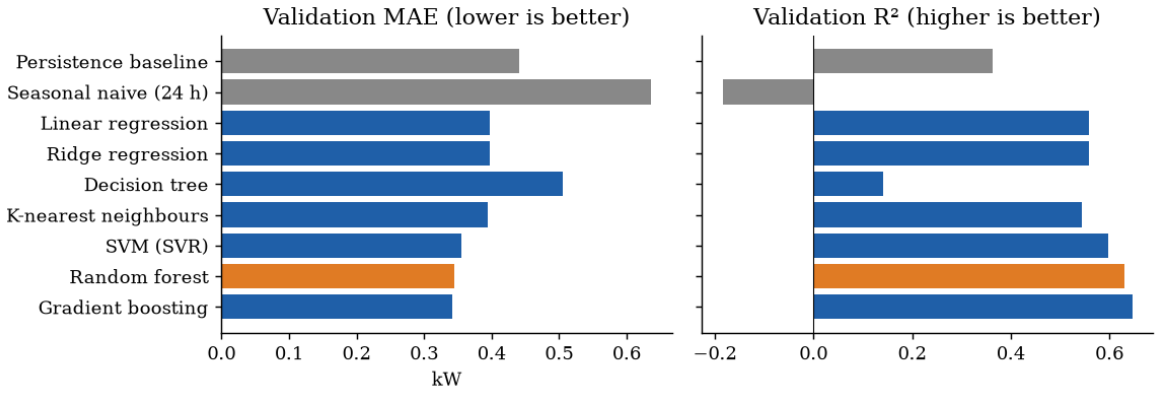


Figure 4.2: *Validation MAE and R^2 of all models (deployed model in orange, baselines in grey)*

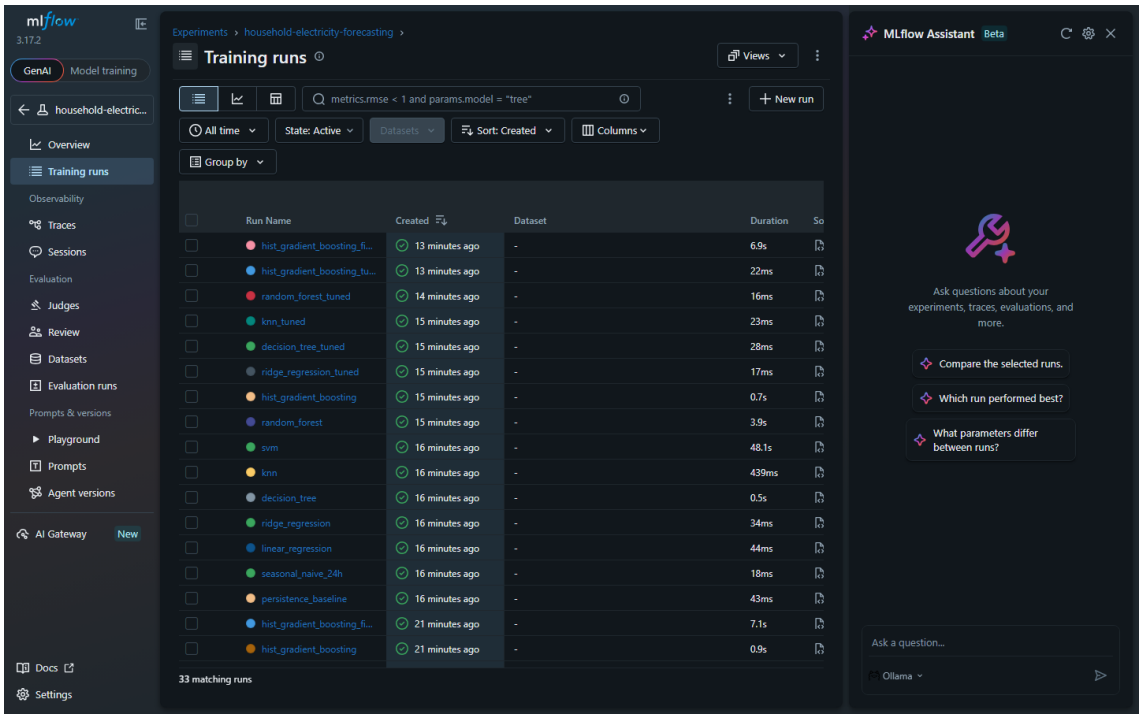


Figure 4.3: *MLflow tracking UI: runs of every model, tuned variant and final model*

Among the syllabus algorithms, random forest has the lowest validation RMSE (0.509 kW), followed by the SVM (0.531 kW); it is therefore the deployed model. The reference gradient-boosting model is slightly lower still (0.498 kW), a difference of 0.010 kW. Linear and ridge regression are identical at $\alpha = 1$ and form a strong linear reference (R^2 0.559). The seasonal-naive model has negative R^2: yesterday's same hour is a worse predictor than the mean, which confirms that household load is driven by irregular routines.

4.1.3. Over-fitting, under-fitting and regularisation

Figure 4.4 compares training and validation error. The default decision tree has a training MAE of 0.000 kW (it memorises every training hour) but the worst validation MAE of the machine-learning models (0.506 kW): a textbook case of over-fitting. Linear regression shows the opposite pattern, with training and validation errors almost equal, indicating mild under-fitting. The validation curve in Figure 4.5 shows both regimes for the tree: shallow trees under-fit, deep trees over-fit, and the best depth lies in between.

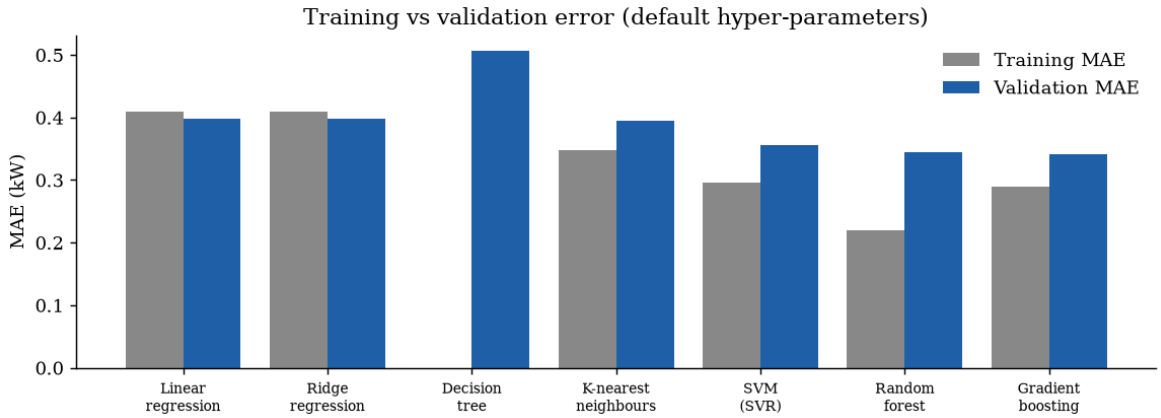


Figure 4.4: *Training vs validation MAE of each model (default hyper-parameters)*

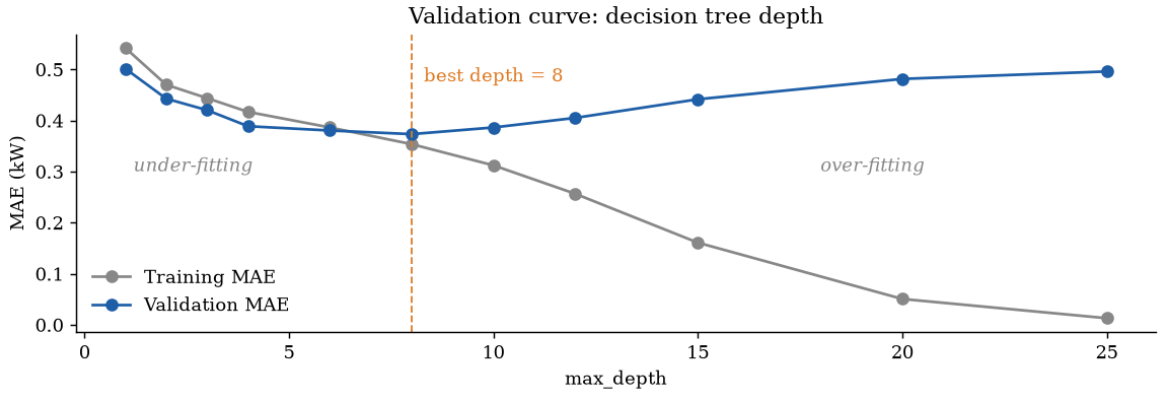


Figure 4.5: *Validation curve of the decision tree: error against max_depth*

Learning curves (Figure 4.6) show how training and cross-validation error change as the models see more history. For linear regression both curves meet quickly at a high error, the signature of under-fitting (more data does not help). The unrestricted decision tree keeps a near-zero training error and a large gap, the signature of over-fitting. Random forest and gradient boosting narrow the gap as data grows (gradient boosting CV MAE 0.562 → 0.435 kW), so more historical data would still help them.

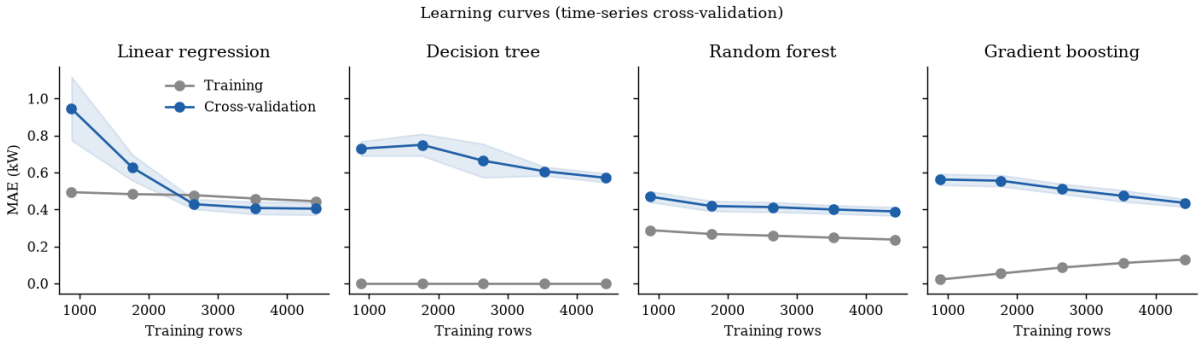


Figure 4.6: *Learning curves with time-series cross-validation*

4.1.4. Hyper-parameter tuning with GridSearchCV

Table 4.2 lists the GridSearchCV results (five time-series folds on the training period).

Table 4.2: *Hyper-parameter tuning results*

Model	Best parameters	Grid size	CV MAE	Val MAE default	Val MAE tuned
Ridge regression	alpha=100.0	5	0.403	0.398	0.398
Decision tree	max_depth=10, min_samples_leaf=100	24	0.397	0.506	0.380
K-nearest neighbours (KNN)	n_neighbors=40, weights=distance	8	0.406	0.395	0.388
Random forest	max_depth=16, min_samples_leaf=20	9	0.368	0.345	0.348
Histogram gradient boosting	learning_rate=0.03, max_leaf_nodes=15	9	0.363	0.341	0.344

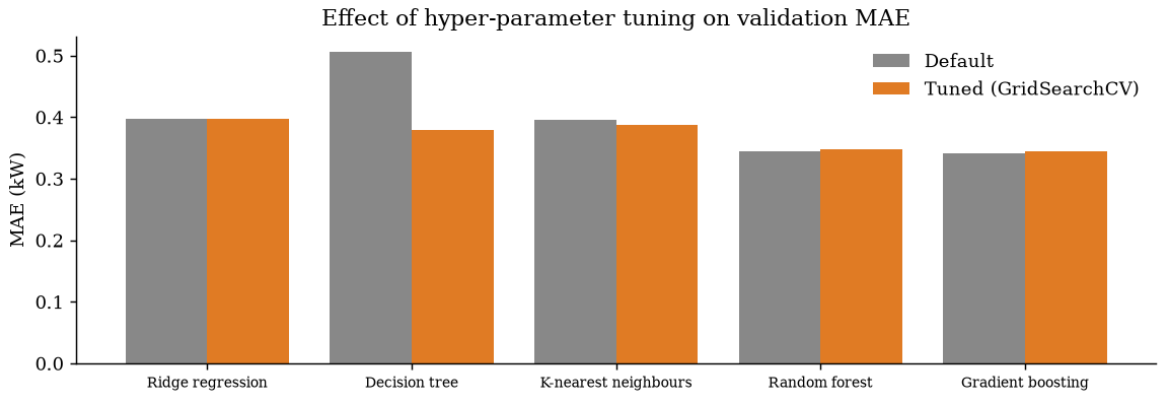


Figure 4.7: *Validation MAE before and after GridSearchCV tuning*

Figure 4.8 shows how the error responds to individual hyper-parameters. The decision-tree heat map covers the whole GridSearchCV grid: deep trees with tiny leaves are worst, and a large minimum leaf size makes depth almost irrelevant. KNN improves steadily from k = 1 (validation MAE 0.523) to k = 80 (0.389). Ridge is flat until α becomes very large, when strong regularisation starts to under-fit.

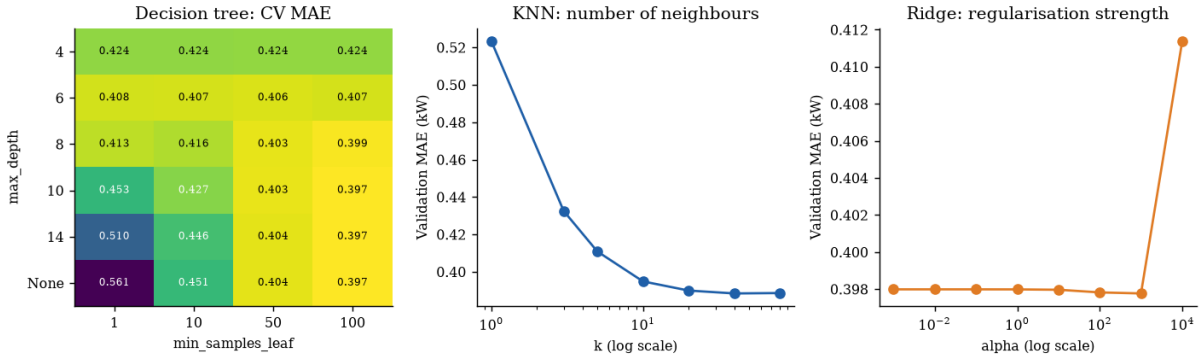


Figure 4.8: *Hyper-parameter response: decision-tree grid, KNN neighbours, ridge alpha*

Table 4.3 and Figure 4.9 give the error in each time-series fold for the tuned models. Every model is worse in fold 2, whose validation window is a winter period with high, variable consumption, and the ranking of models is consistent across folds, which supports the model choice.

Table 4.3*: Cross-validation MAE (kW) per time-series fold, tuned models*

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
Ridge regression	0.431	0.445	0.370	0.402	0.367	0.403	0.035
Decision tree	0.437	0.452	0.353	0.389	0.353	0.397	0.046
K-nearest neighbours (KNN)	0.473	0.443	0.371	0.394	0.348	0.406	0.051
Random forest	0.405	0.414	0.342	0.356	0.322	0.368	0.040
Histogram gradient boosting	0.391	0.428	0.332	0.347	0.317	0.363	0.046

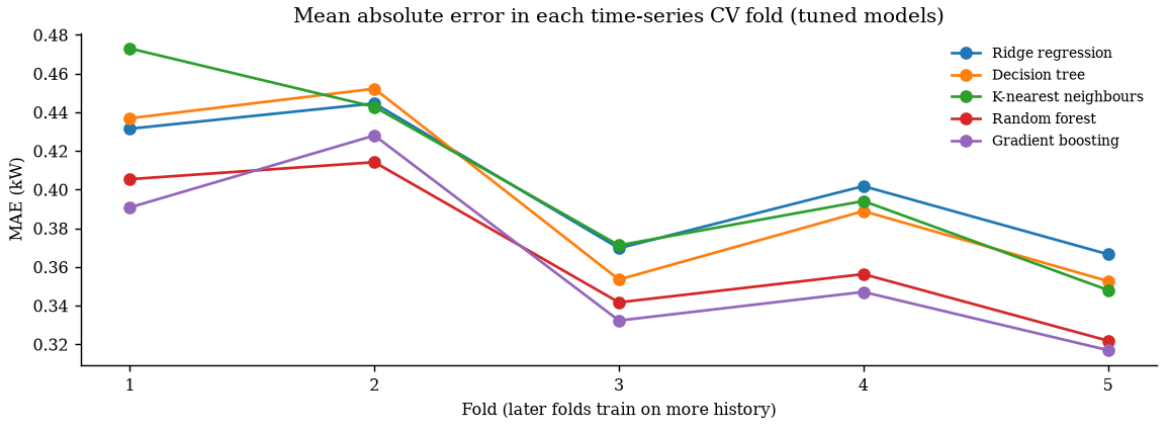


Figure 4.9*: MAE in each time-series cross-validation fold*

Tuning helps most where the default was poorly regularised: the decision tree improves from 0.506 to 0.380 kW once depth and minimum leaf size are limited. KNN gains by averaging over more neighbours with distance weighting; ridge prefers strong regularisation (α = 100.0) but gains little. For random forest and gradient boosting the defaults were already well chosen, and the small differences are within the noise between cross-validation and the validation window. The default random forest (200 trees, depth 16, minimum 5 rows per leaf) therefore remains the selected model.

4.1.5. Test-set results of the forecaster

The selected model was refit on train+validation and evaluated once on the untouched test period (Table 4.4).

Table 4.4: *Held-out test performance (Jun–Nov 2010)*

Model	MAE (kW)	MSE	RMSE (kW)	R^2	MAPE (%)
Persistence baseline	0.377	0.329	0.574	0.335	46.9
Random forest	**0.295**	**0.188**	**0.434**	**0.620**	**40.8**
Improvement	21.9%	42.9%	24.4%	+0.285	–

To confirm that selecting on the validation window did not favour one model by chance, every model (default and tuned) was also refit on train+validation and scored on the test period. Tables 4.5 and 4.6 give the complete results; Figure 4.10 compares the main ones.

Table 4.5: *Test-set error metrics of every model (kW, kW²)*

Model	MAE	MSE	RMSE	R^2	MAPE %	Median AE	Max error
Persistence baseline	0.377	0.329	0.574	0.335	46.9	0.206	3.83
Seasonal naive (24 h)	0.485	0.532	0.730	-0.076	64.7	0.299	5.18
Linear regression	0.344	0.233	0.482	0.530	48.6	0.245	3.85
Ridge regression	0.344	0.233	0.482	0.530	48.6	0.245	3.85
Decision tree	0.423	0.422	0.650	0.147	54.3	0.233	3.78
K-nearest neighbours (KNN)	0.333	0.240	0.490	0.515	43.4	0.208	4.29
Support vector machine (SVR)	0.296	0.197	0.444	0.602	37.0	0.176	3.84
Random forest	0.295	0.188	0.434	0.620	40.8	0.187	3.47
Histogram gradient boosting	0.293	0.187	0.432	0.623	39.5	0.185	3.67
Ridge regression (tuned)	0.343	0.232	0.482	0.530	48.5	0.245	3.85
Decision tree (tuned)	0.330	0.227	0.477	0.541	45.5	0.210	3.41
K-nearest neighbours (KNN) (tuned)	0.326	0.228	0.477	0.539	42.7	0.205	4.05
Random forest (tuned)	0.298	0.191	0.437	0.613	41.3	0.185	3.57
Histogram gradient boosting (tuned)	0.295	0.186	0.431	0.625	40.0	0.191	3.53

Table 4.6: *Test-set accuracy-style metrics, training time and prediction time of every model*

Model	Accuracy (1 − WAPE)	Within ±0.25 kW	Within ±0.5 kW	Explained variance	Fit (s)	ms / prediction
Persistence baseline	58.9%	53.4%	70.0%	0.335	0.0	0.0000
Seasonal naive (24 h)	47.3%	46.2%	63.4%	-0.076	0.0	0.0000
Linear regression	62.6%	50.7%	76.8%	0.530	0.0	0.0006
Ridge regression	62.6%	50.7%	76.8%	0.530	0.0	0.0004
Decision tree	53.9%	51.8%	67.9%	0.149	1.1	0.0005
K-nearest neighbours (KNN)	63.8%	54.8%	76.0%	0.521	0.0	0.0438
Support vector machine (SVR)	67.8%	59.9%	79.6%	0.606	79.4	1.9554
Random forest	67.9%	58.6%	80.8%	0.621	10.2	0.0186
Histogram gradient boosting	68.1%	59.1%	80.9%	0.623	1.5	0.0047
Ridge regression (tuned)	62.6%	50.8%	76.7%	0.530	0.0	0.0008
Decision tree (tuned)	64.1%	54.3%	76.7%	0.542	0.5	0.0003
K-nearest neighbours	64.5%	55.1%	77.5%	0.547	0.0	0.0471

Model	Accuracy (1 − WAPE)	Within ±0.25 kW	Within ±0.5 kW	Explained variance	Fit (s)	ms / prediction
(KNN) (tuned)						
Random forest (tuned)	67.6%	58.8%	80.2%	0.614	8.8	0.0179
Histogram gradient boosting (tuned)	67.9%	58.3%	81.3%	0.625	1.9	0.0078

Accuracy-style metrics make the regression results easier to read. Accuracy as 1 − WAPE (weighted absolute percentage error) is the share of total consumption predicted correctly: 67.9% for the deployed model against 58.9% for persistence. 58.6% of hourly forecasts are within ±0.25 kW of the actual value and 80.8% within ±0.5 kW. The ranking on the test set matches validation: random forest is the best syllabus algorithm, ahead of the SVM, and the reference gradient-boosting model is better by only 0.0013 kW MAE (0.1 percentage points of accuracy). Restricting deployment to syllabus algorithms therefore costs almost nothing, and the selection was not a lucky choice.

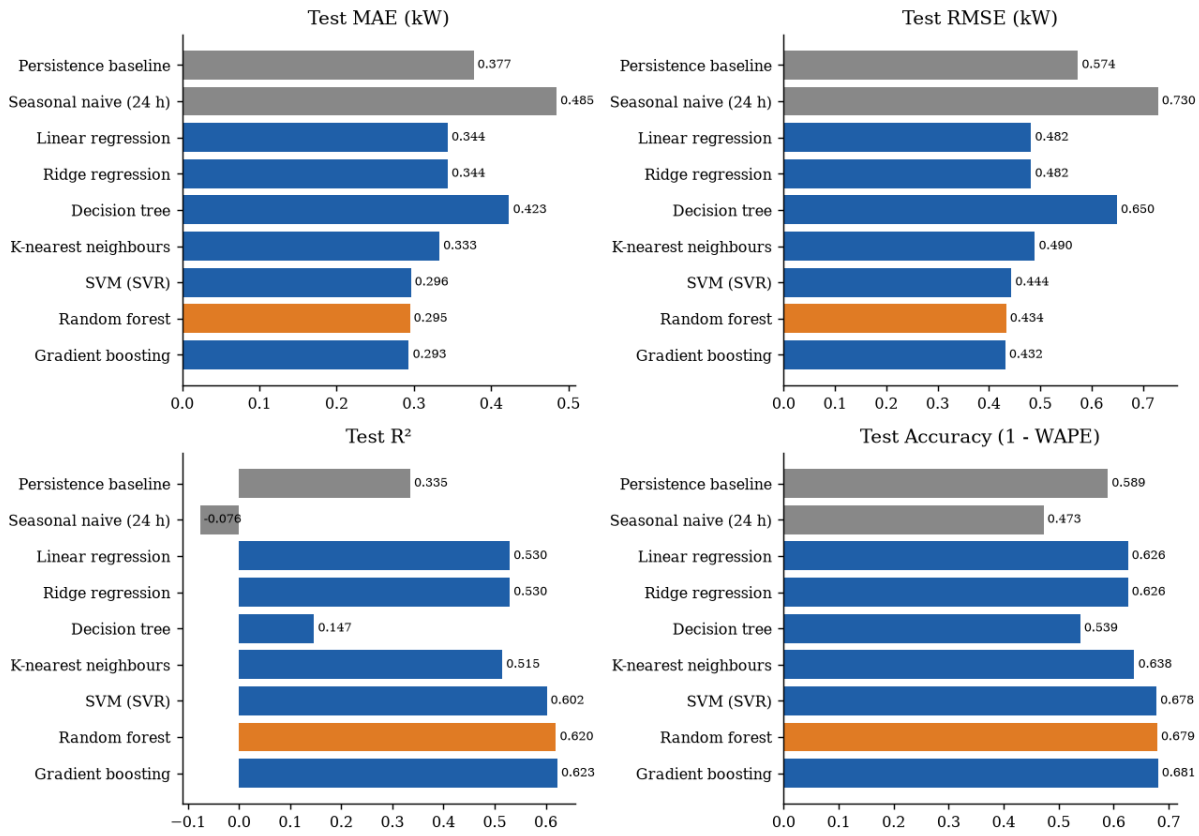


Figure 4.10*: Test MAE, RMSE, R² and accuracy (1 − WAPE) of every model*

Figures 4.11–4.13 show the behaviour of each model on the test set: a week of forecasts, the predicted-vs-actual scatter, and the residual distribution.

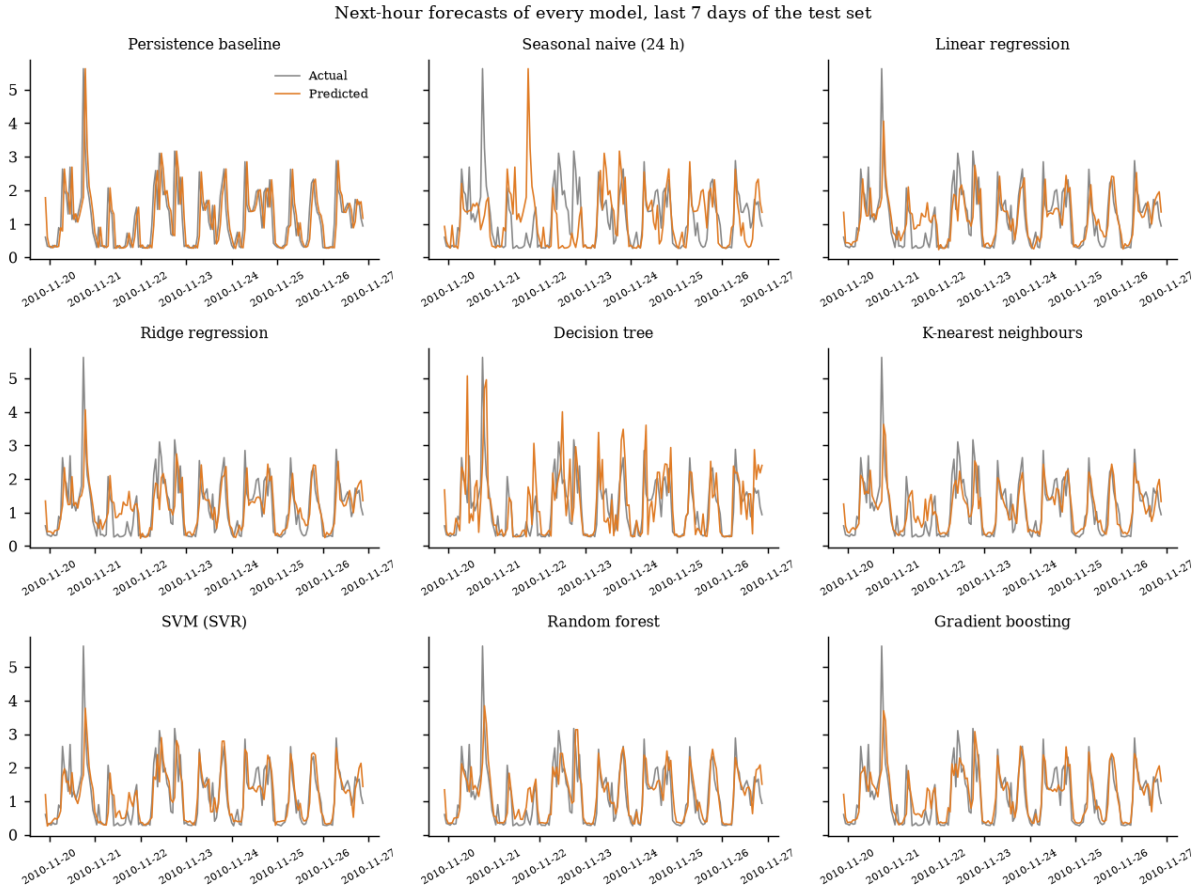


Figure 4.11*: Forecasts of every model over the last seven days of the test set*

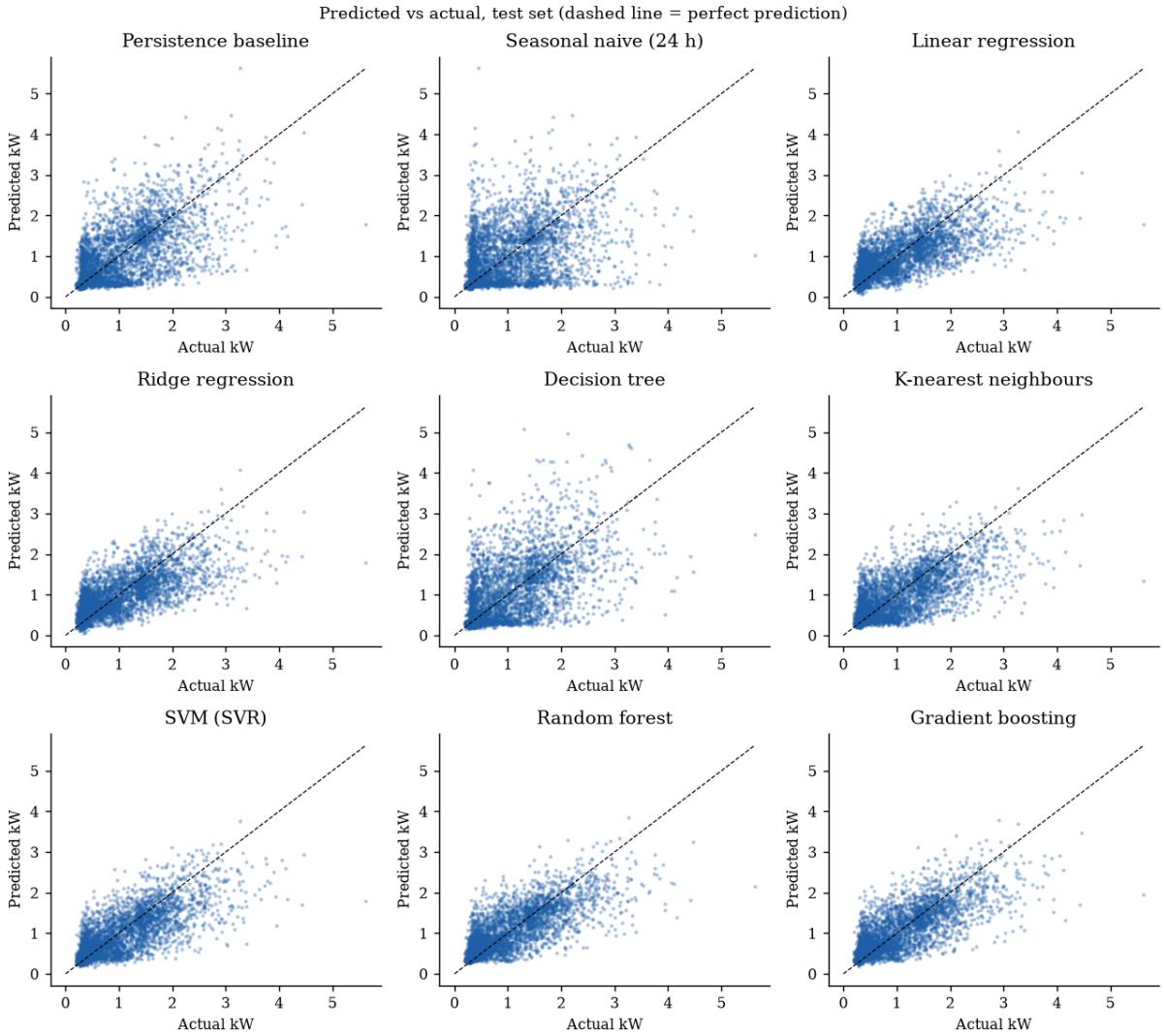


Figure 4.12*: Predicted vs actual for every model (test set)*

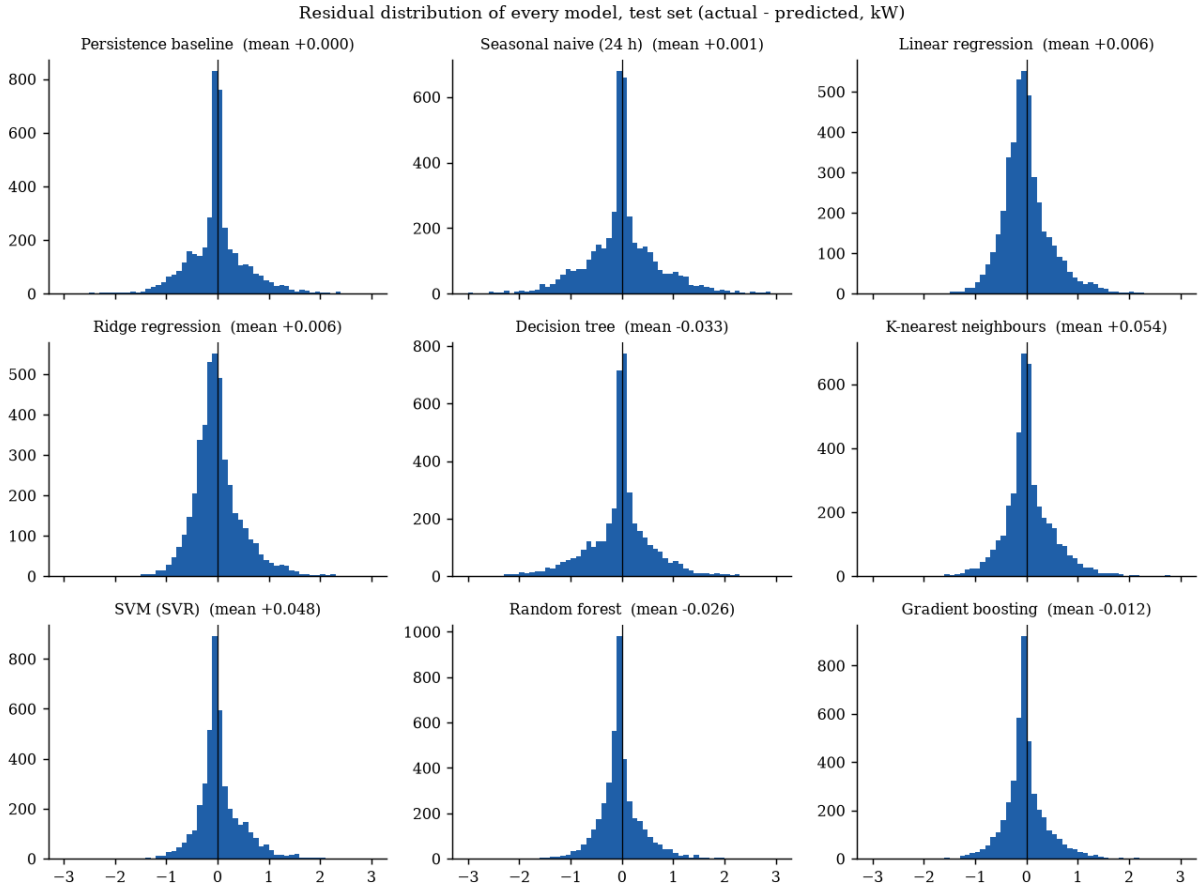


Figure 4.13*: Residual distribution of every model (test set)*

The forecasts follow the daily cycle and the timing of evening peaks closely (Figure 4.14). Residuals are centred near zero but right-skewed (Figure 4.15): the largest errors are sudden short spikes, such as an oven or water heater switching on, which are inherently unpredictable from past meter data alone.

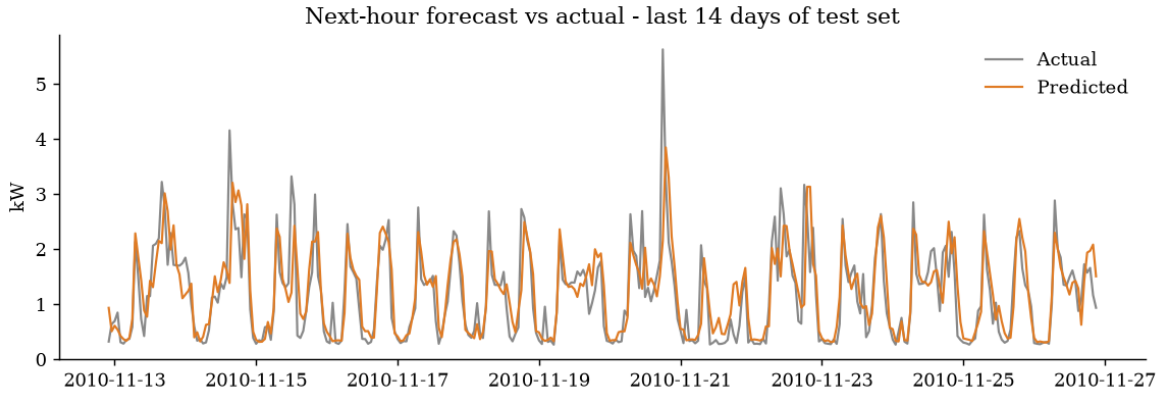


Figure 4.14*: Next-hour forecast vs actual consumption, last 14 days of the test set*

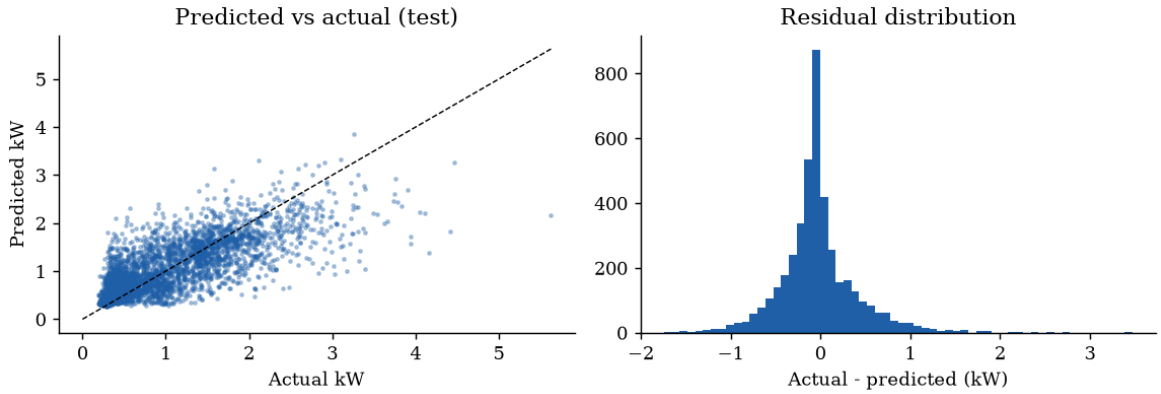


Figure 4.15*: Predicted vs actual scatter and residual distribution*

Explainability. Permutation importance (Figure 4.16) shows the inputs the model relies on: Global_intensity_lag1 (0.268), lag_1 (0.082), Sub_metering_3_lag1 (0.031), hour (0.018). The previous hour's current and power carry the household's present state, the water-heater/AC sub-meter signals heavy thermal loads, and the hour encodings capture daily routines, which is physically consistent.

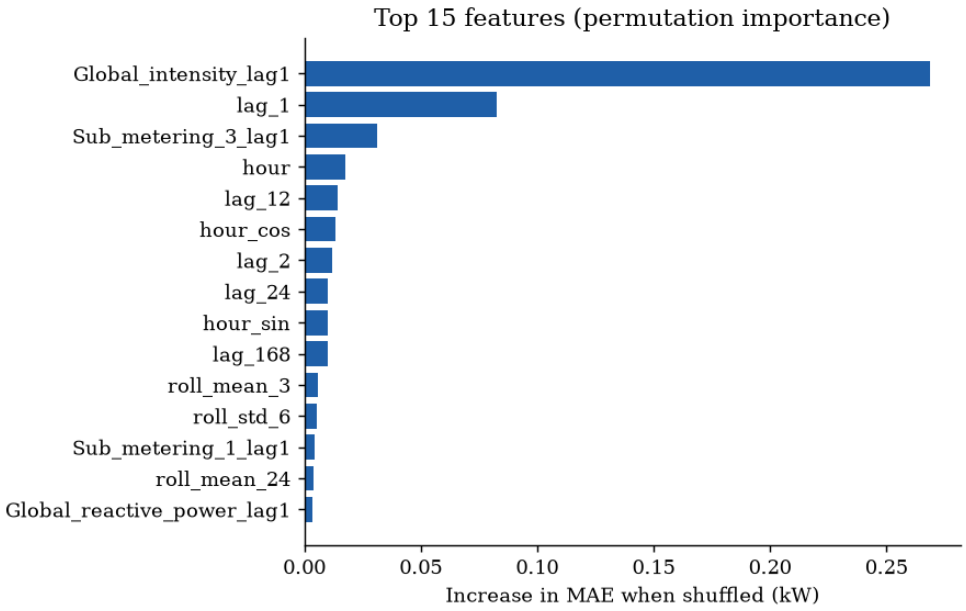


Figure 4.16*: Top 15 features by permutation importance*

Error by forecast horizon. The deployed service forecasts up to 48 hours ahead by feeding each prediction back as input (recursive forecasting). Figure 4.17 and Table 4.7 show the error for 60 random starting points in the test period. Error grows from 0.285 kW one hour ahead to 0.499 kW 24 hours ahead as uncertainty accumulates, but stays below the same-hour-yesterday baseline at every horizon (0.570 kW). This is why the API widens its prediction interval with √h.

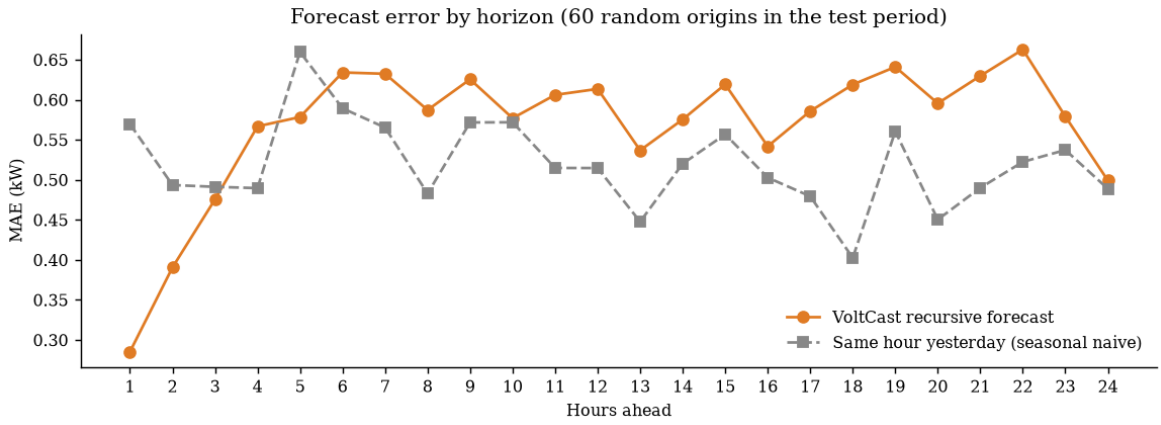


Figure 4.17*: Forecast MAE by hours ahead, compared with the seasonal-naive baseline*

Table 4.7*: Forecast error by horizon*

Hours ahead	1	2	3	6	12	18	24
VoltCast MAE (kW)	0.285	0.391	0.475	0.634	0.614	0.619	0.499
Seasonal naive MAE (kW)	0.570	0.493	0.491	0.589	0.515	0.403	0.489

4.1.6. Peak-hour classification

Peak hours make up 25% of training hours and 17.0% of the (summer-heavy) test period. Table 4.8 compares the classifiers on the validation window.

Table 4.8*: Peak-hour classifiers on the validation set*

Classifier	Train F1	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic regression	0.662	0.807	0.764	0.492	0.599	0.887
Decision tree default	1.000	0.770	0.610	0.589	0.599	0.717
Decision tree tuned	0.735	0.790	0.603	0.821	0.696	0.879

The default decision tree again over-fits (training F1 1.00). GridSearchCV selects max_depth = 12, min_samples_leaf = 50 and class_weight = balanced, raising validation F1 from 0.599 to 0.696, so the tuned tree is selected. On the test period it reaches accuracy 0.803, precision 0.454, recall 0.779, F1 0.574 and ROC-AUC 0.873 (Figure 4.18).

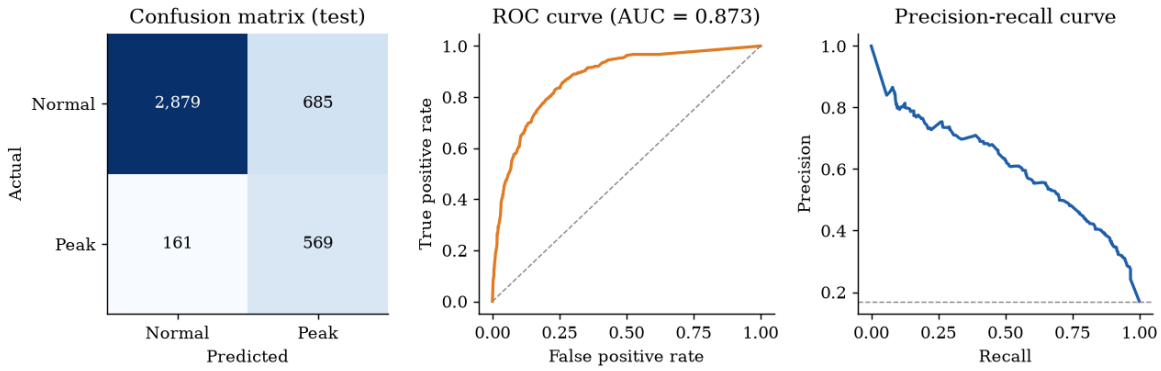


Figure 4.18: *Peak-hour classifier on the test set: confusion matrix, ROC and precision-recall curves*

Figure 4.19 compares the confusion matrices of all three classifiers on the test set, and Figure 4.20 their ROC and precision-recall curves. Table 4.9 is the full classification report of each classifier, giving precision, recall, F1 and support for both classes plus accuracy and the macro and weighted averages.

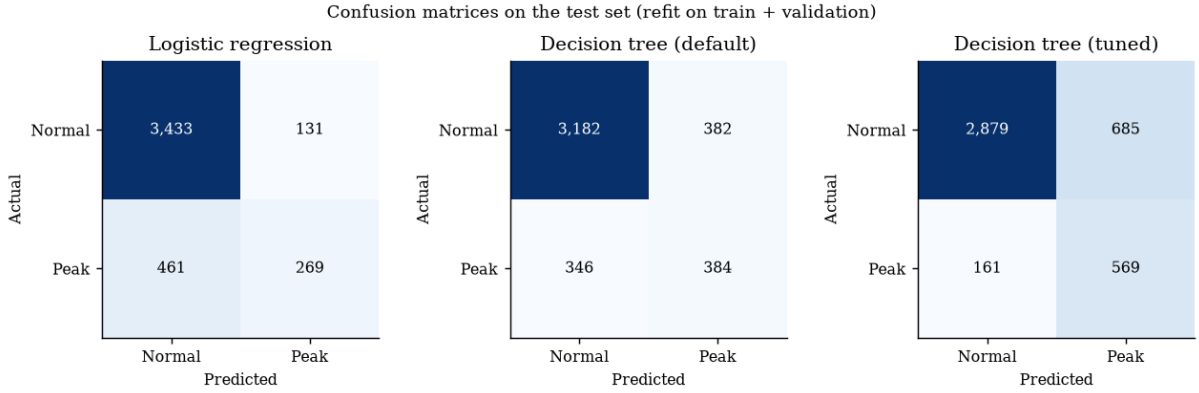


Figure 4.19: *Confusion matrices of the three classifiers (test set)*

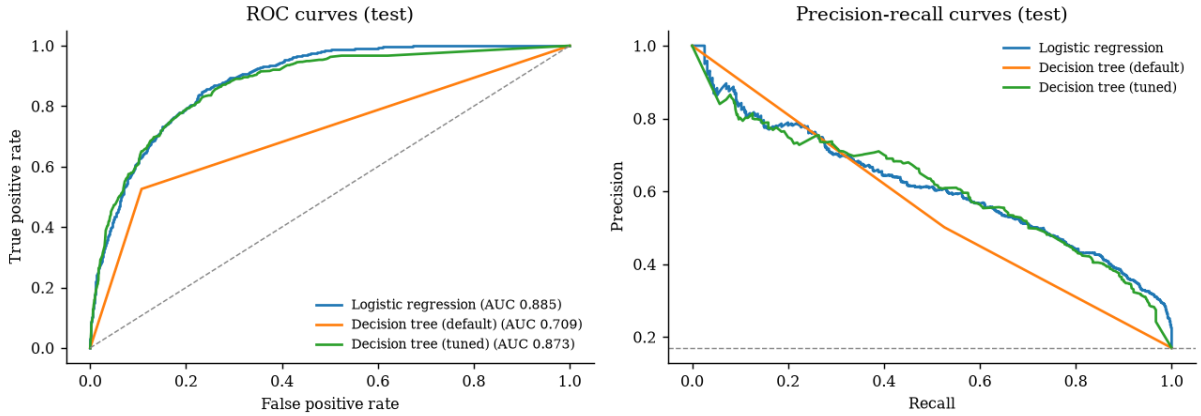


Figure 4.20: *ROC and precision-recall curves of the three classifiers (test set)*

Table 4.9: *Classification reports on the test set*

Classifier	Class	Precision	Recall	F1-score	Support
Logistic regression	Normal hour	0.882	0.963	0.921	3564
	Peak hour	0.672	0.368	0.476	730
	macro avg	0.777	0.666	0.698	4294
	weighted avg	0.846	0.862	0.845	4294
	accuracy			0.862	4294
Decision tree (default)	Normal hour	0.902	0.893	0.897	3564
	Peak hour	0.501	0.526	0.513	730
	macro avg	0.702	0.709	0.705	4294
	weighted avg	0.834	0.830	0.832	4294
	accuracy			0.830	4294
Decision tree (tuned)	Normal hour	0.947	0.808	0.872	3564
	Peak hour	0.454	0.779	0.574	730
	macro avg	0.700	0.794	0.723	4294
	weighted avg	0.863	0.803	0.821	4294
	accuracy			0.803	4294

The tuned tree outputs a probability; the alert uses 0.5 as the cut-off. Figure 4.21 and Table 4.10 show the trade-off for other cut-offs: a lower threshold catches more peaks (recall) but raises more false alarms (lower precision), and a higher threshold does the opposite. The F1-score is highest near the middle of the range, which supports the default.

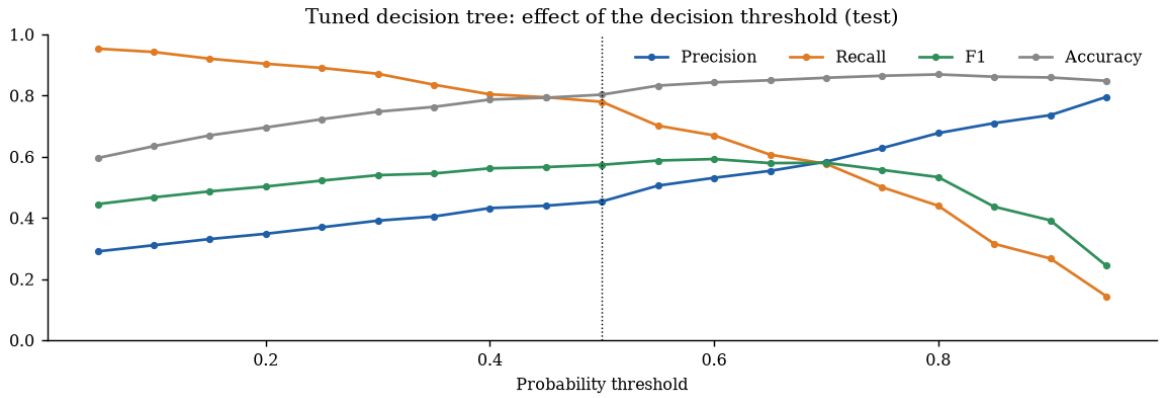


Figure 4.21: *Precision, recall, F1 and accuracy against the decision threshold*

Table 4.10: *Effect of the probability threshold (tuned decision tree, test set)*

Threshold	Precision	Recall	F1	Accuracy
0.05	0.290	0.953	0.445	0.596
0.15	0.331	0.921	0.487	0.670
0.25	0.369	0.890	0.522	0.723
0.35	0.405	0.836	0.545	0.763
0.45	0.440	0.795	0.566	0.793
0.55	0.505	0.701	0.587	0.833
0.65	0.554	0.607	0.579	0.850
0.75	0.628	0.500	0.557	0.865
0.85	0.710	0.315	0.436	0.862

Threshold	Precision	Recall	F1	Accuracy
0.95	0.795	0.144	0.244	0.848

Of 730 real peak hours, 569 were caught and 161 missed; 685 normal hours raised a false alarm. High recall with moderate precision is the safer trade-off for an alert, since a missed peak costs more than an unnecessary reminder. A simpler rule, flagging hours where the regression forecast exceeds τ, is more accurate (0.878) but catches only 61% of peaks. The deployed system uses the classifier for the next hour, and the forecast-threshold rule for later hours, where the classifier's inputs are no longer current. On 40 random test origins this rule was 76% accurate over hours 2–24, against 58% for the classifier.

4.1.7. Deployment

The forecaster and classifier are served by a Flask REST API (Table 4.11). Every request body is validated with Pydantic, which checks types, non-negative readings, positive voltage, a horizon of 1–48 h, at least 169 hours of history and no duplicate timestamps. Invalid input returns HTTP 422 with an explanation.

Table 4.11*: REST API endpoints*

Method	Endpoint	Purpose
GET	/api/health	Liveness and loaded model
GET	/api/model-info	Model, hyper-parameters, training window, test metrics, peak threshold
POST	/api/predict	Forecast + peak alerts from user-supplied hourly readings (JSON)
GET	/api/forecast	Forecast from a point in the test period, returned with actual values
GET	/api/sample-request	Ready-made request body for /api/predict
GET	/api/metrics	Prometheus metrics (OpenMetrics text format)

On-premise (Docker). A multi-stage Dockerfile builds the React web app with Node and then the Python runtime with gunicorn. Docker Compose starts five microservices: the API with the web app, the Streamlit dashboard, the MLflow UI, Prometheus and Grafana. The full stack was built and run locally and every service responded. **Kubernetes.** k8s/deployment.yaml defines a two-replica Deployment with readiness and liveness probes on /api/health, a NodePort Service and a CPU-based HorizontalPodAutoscaler (2–5 pods). The manifests are validated in CI; they were not applied to a live cluster in this project. **Cloud.** The same Flask app runs as a serverless function on Vercel, with the React app served from its CDN, at https://voltcast-forecast.vercel.app. A 24-hour forecast request completes in well under one second.

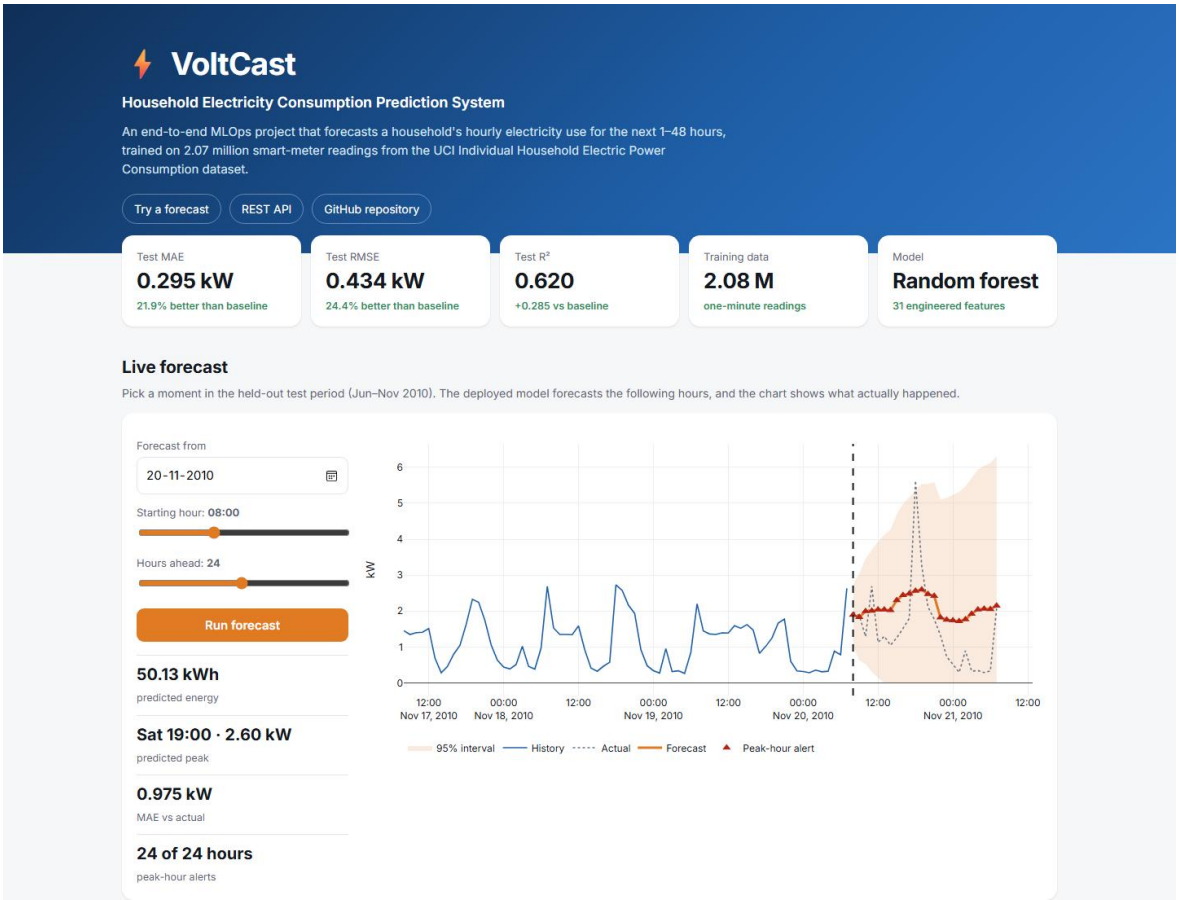


Figure 4.22*: VoltCast web application (React) with a live 24-hour forecast and peak-hour alerts*

4.1.8. CI/CD and versioning

The GitHub Actions workflow runs on every push, on pull requests and weekly (continuous training). It installs the dependencies, reproduces the pipeline with dvc repro, prints the DVC-tracked metrics, applies two quality gates (forecaster MAE below the persistence baseline; classifier ROC-AUC above 0.75), runs the 11 tests, builds the React app, validates the Docker Compose, Prometheus/Grafana and Kubernetes files, builds the Docker image and uploads the models and reports as artifacts. Vercel's Git integration provides continuous deployment: every push to the main branch is built and published automatically.

Versioning. DVC records each stage's inputs, parameters and outputs in dvc.lock, so a given Git commit identifies the exact dataset, parameters, models and metrics; dvc repro re-runs only the stages whose inputs changed. The dataset is kept in the DVC cache rather than in Git. In addition, every trained forecaster is registered as a new version of voltcast-forecaster in the MLflow model registry.

Automated testing. Table 4.12 lists the pytest suite.

Table 4.12: *Automated test suite results*

Test	What it validates	Result
test_resample_fills_gaps	Missing hours are filled; output is gap-free	Pass
test_features_have_no_leakage	lag_1 and rolling means use only past values	Pass
test_metrics_perfect_prediction	Regression metric implementation	Pass
test_psi_detects_shift	PSI < 0.1 for same distribution, > 0.2 for shifted	Pass
test_classification_scores	Accuracy, precision, recall and confusion matrix	Pass
test_health_endpoint	Service liveness and model loading	Pass
test_predict_endpoint_returns_horizon	Horizon, interval ordering, peak alerts, strictly valid JSON	Pass
test_predict_rejects_short_history	Too little history returns HTTP 422	Pass
test_predict_rejects_invalid_reading	Negative voltage returns HTTP 422	Pass
test_prometheus_metrics_exposed	Prometheus counters and histograms are published	Pass
test_model_beats_baseline	Quality gate: model MAE < baseline MAE	Pass

4.1.9. Monitoring and logging

Prometheus scrapes /api/metrics every 15 seconds (Figure 4.23). The API exports request counts and latency histograms per endpoint, the number of forecast hours served, the distribution of forecast values, peak alerts, rejected requests and the loaded model's test MAE. A provisioned Grafana dashboard visualises them (Figure 4.24); invalid requests sent on purpose during the test run appear under 'Rejected requests'. Every prediction is also written as one JSON line to logs/predictions.jsonl (time, model, horizon, forecast values, number of peak hours), without the submitted meter readings.

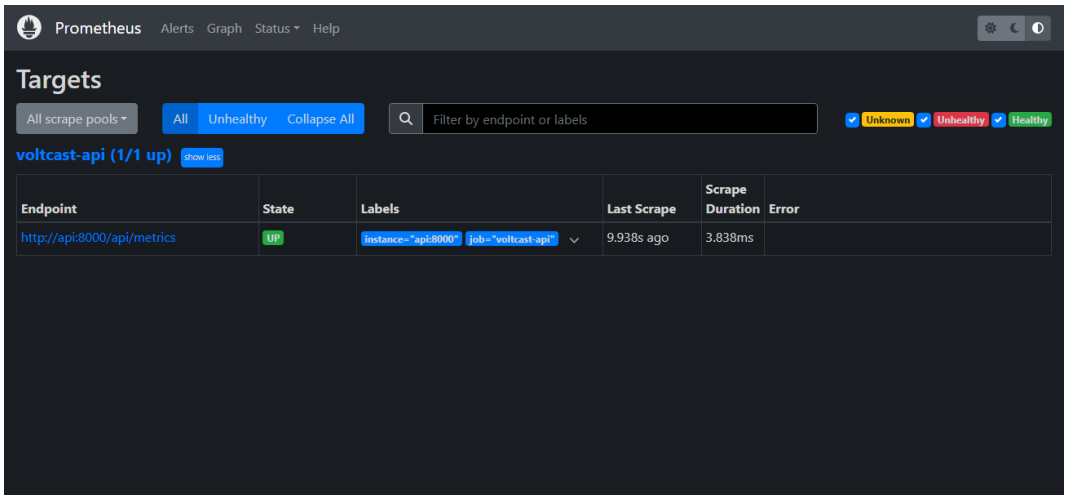


Figure 4.23: *Prometheus target page: the VoltCast API is being scraped (state UP)*

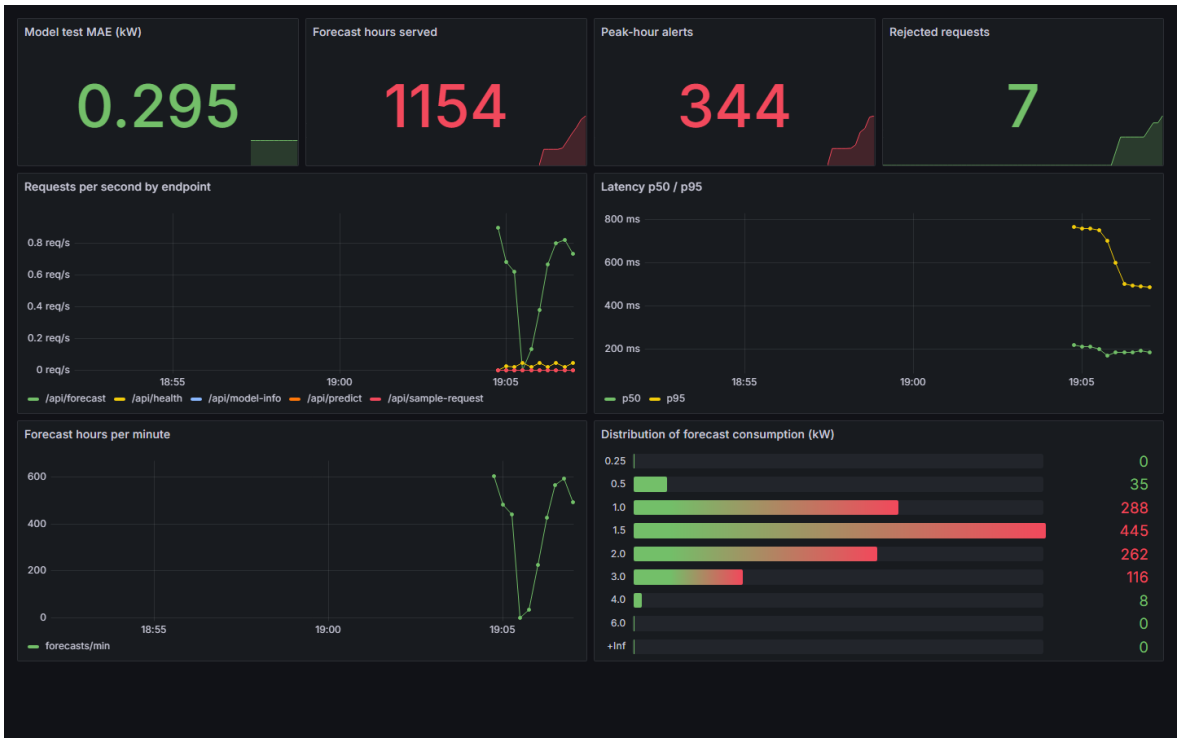


Figure 4.24: *Grafana dashboard "VoltCast – Model Monitoring" fed by Prometheus*

Data drift. Figure 4.25 shows the PSI between training and test distributions.

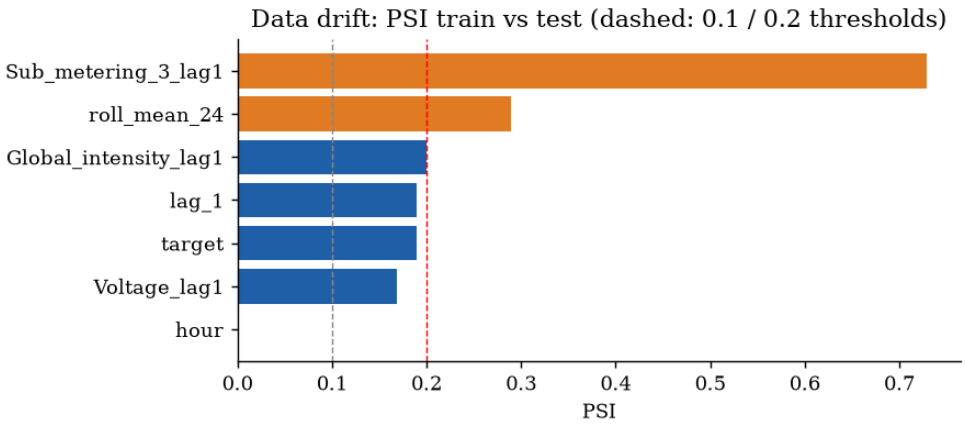


Figure 4.25: *Population Stability Index of monitored features (train vs test)*

Significant drift was detected in roll_mean_24, Sub_metering_3_lag1 (PSI 0.73 for the water-heater/AC sub-meter). This is genuine seasonal drift: the training data covers three winters, while the test period is June–November, when heating load is low. The 30-day rolling MAE (0.351 kW) was 19.1% above the reference test MAE as consumption climbed again in November. The monitor therefore set retraining_recommended = true, which is the behaviour a production MLOps loop requires.

Figure 4.26 follows drift month by month through the test period. The water-heater/AC sub-meter is far from its training distribution in every summer month and moves back towards it in November, as heating returns. Test MAE follows the same seasonal pattern (0.177–0.379 kW; Table 4.13).

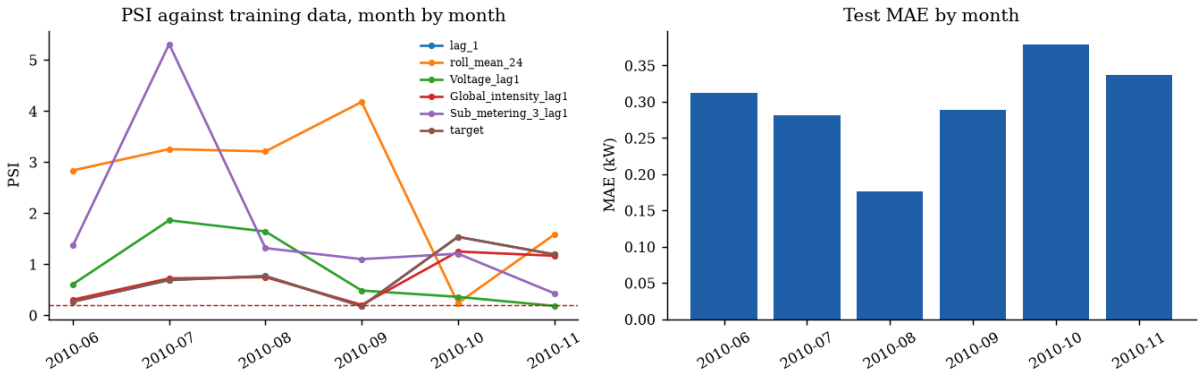


Figure 4.26*: PSI against the training data and test MAE, month by month*

Table 4.13*: Monthly PSI of monitored features and monthly test MAE*

Month	lag 1	roll mean 24	Voltage	Global intensity	Sub metering 3	target	MAE (kW)
2010-06	0.268	2.839	0.609	0.306	1.366	0.267	0.313
2010-07	0.690	3.256	1.862	0.724	5.313	0.695	0.281
2010-08	0.770	3.211	1.643	0.749	1.317	0.767	0.177
2010-09	0.179	4.180	0.485	0.206	1.101	0.179	0.289
2010-10	1.538	0.238	0.362	1.250	1.208	1.537	0.379
2010-11	1.194	1.581	0.185	1.166	0.433	1.194	0.337

4.1.10. Ethical considerations

Fairness and bias. The system makes no decisions about people, so fairness is assessed as equal quality of service across conditions [24]. Figure 4.27 breaks the test error down by group. Error is lowest at night (0.15 kW) and highest in the evening (0.39 kW) and in high-use hours (0.42 kW), and higher at weekends (0.33 vs 0.28 kW) and in autumn (0.34 vs 0.26 kW). The forecasts are therefore least reliable exactly when demand is highest, and this is stated to users. There is also sampling bias: one French household from 2006–2010 cannot represent Indian homes with air-conditioning loads, so the model must be re-validated before any wider use.

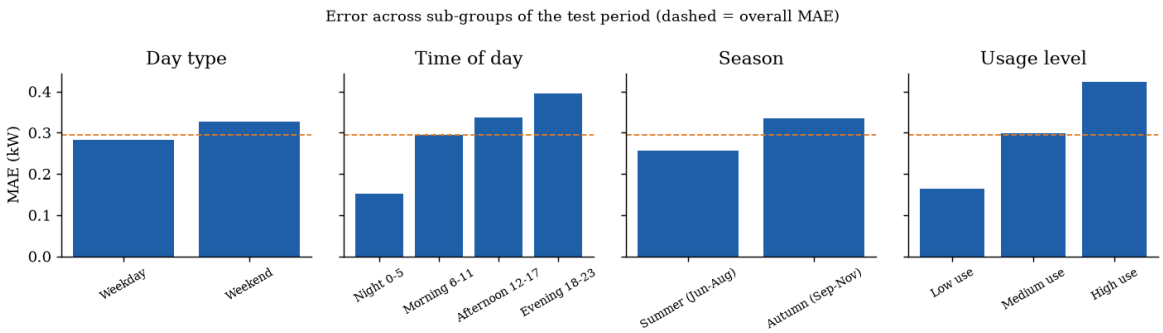


Figure 4.27*: Test error across sub-groups (dashed line: overall MAE)*

Transparency. Feature importance is published, every number is reproducible from the repository, forecasts carry a 95% interval, and a model card (MODEL_CARD.md) documents intended use, data, performance, limitations and monitoring. **Privacy.** Smart-meter data can reveal when residents are home or asleep. The project uses only a public anonymised dataset; the API does not store submitted readings; real deployments should aggregate data, obtain consent and comply with the Digital Personal Data Protection Act, 2023 [25]. **Security.** Input validation, a 2 MB request limit, loading only models produced by the pipeline, and monitoring of rejected requests protect the service against malformed or abusive input.



Figure 4.28*: Responsible-AI section of the web application*

4.1.11. Execution evidence (terminal output)

The figures in this section are the terminal output of the commands that build, test, deploy and monitor the system, captured while they ran on the development machine (Windows

11, PowerShell). Each image shows the exact command line and its unedited output; long outputs are split into numbered parts. The raw text of every capture is kept in reports/terminal/ and the script that produced them is report/capture_terminal.py.

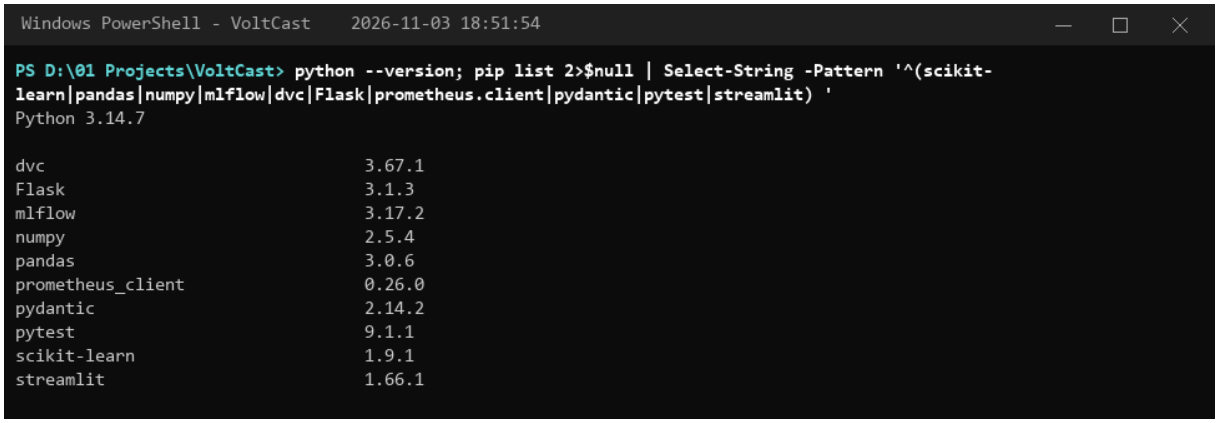


Figure 4.29*: Python and library versions used, run 2026-11-03 18:51:54*

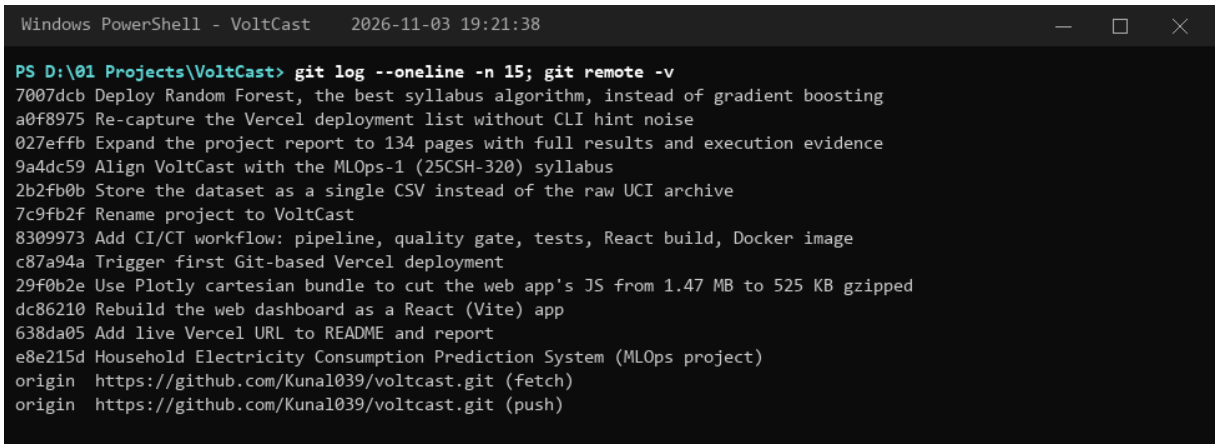


Figure 4.30*: Git history of the project repository, run 2026-11-03 19:21:38*

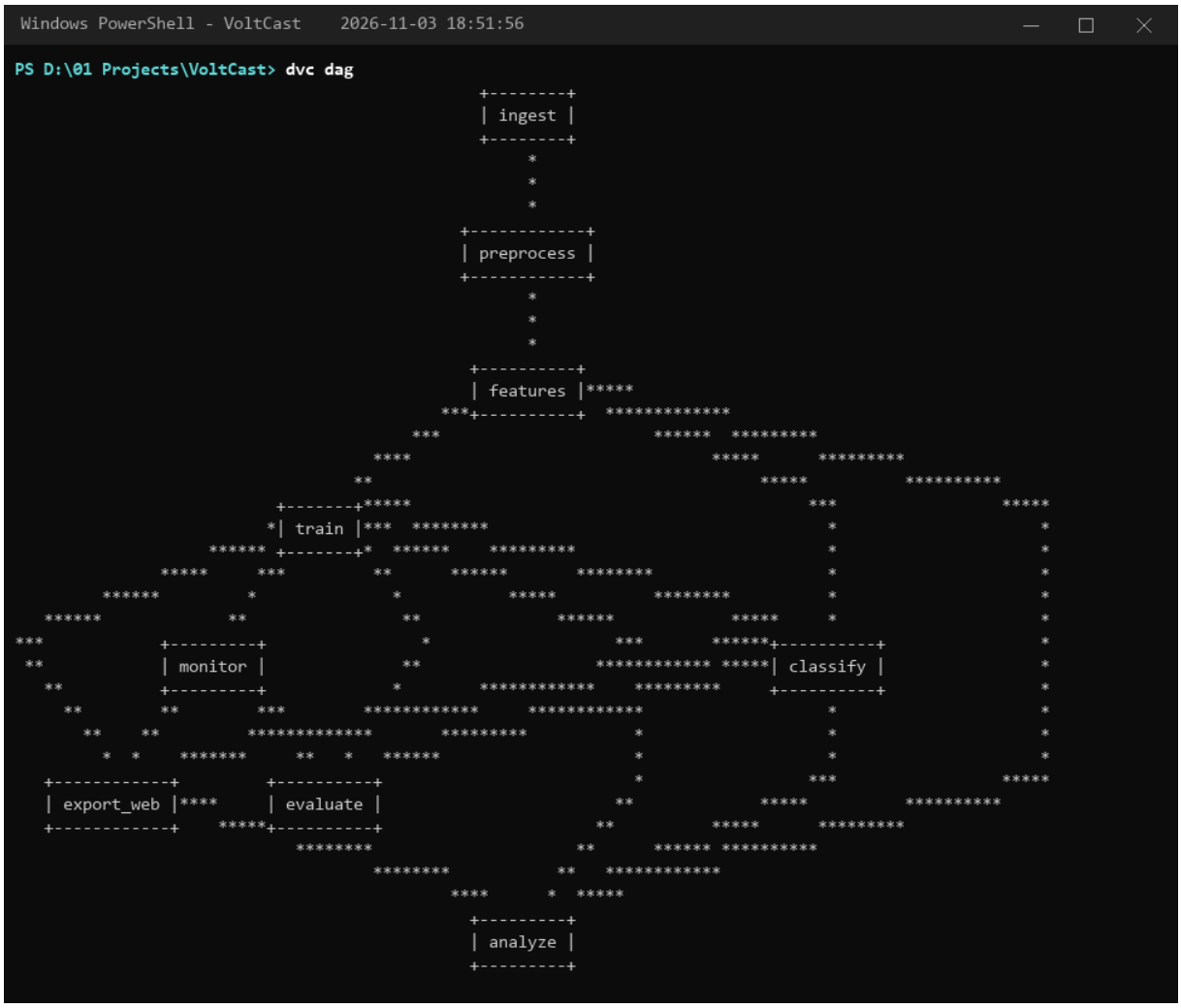


Figure 4.31*: dvc dag: the pipeline stages and their dependencies, run 2026-11-03 18:51:56*

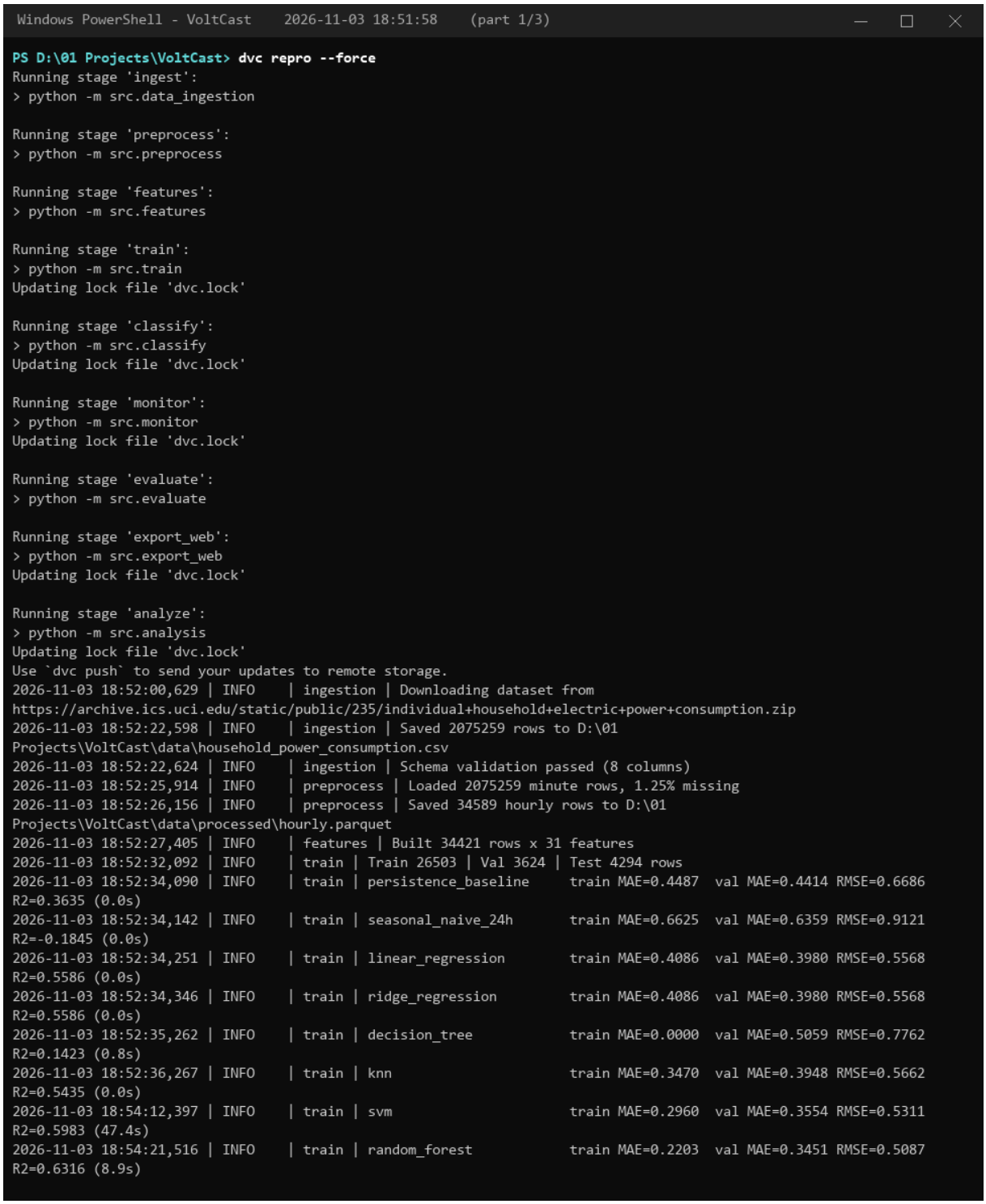


Figure 4.32*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 1 of 3), run 2026-11-03 18:51:58*

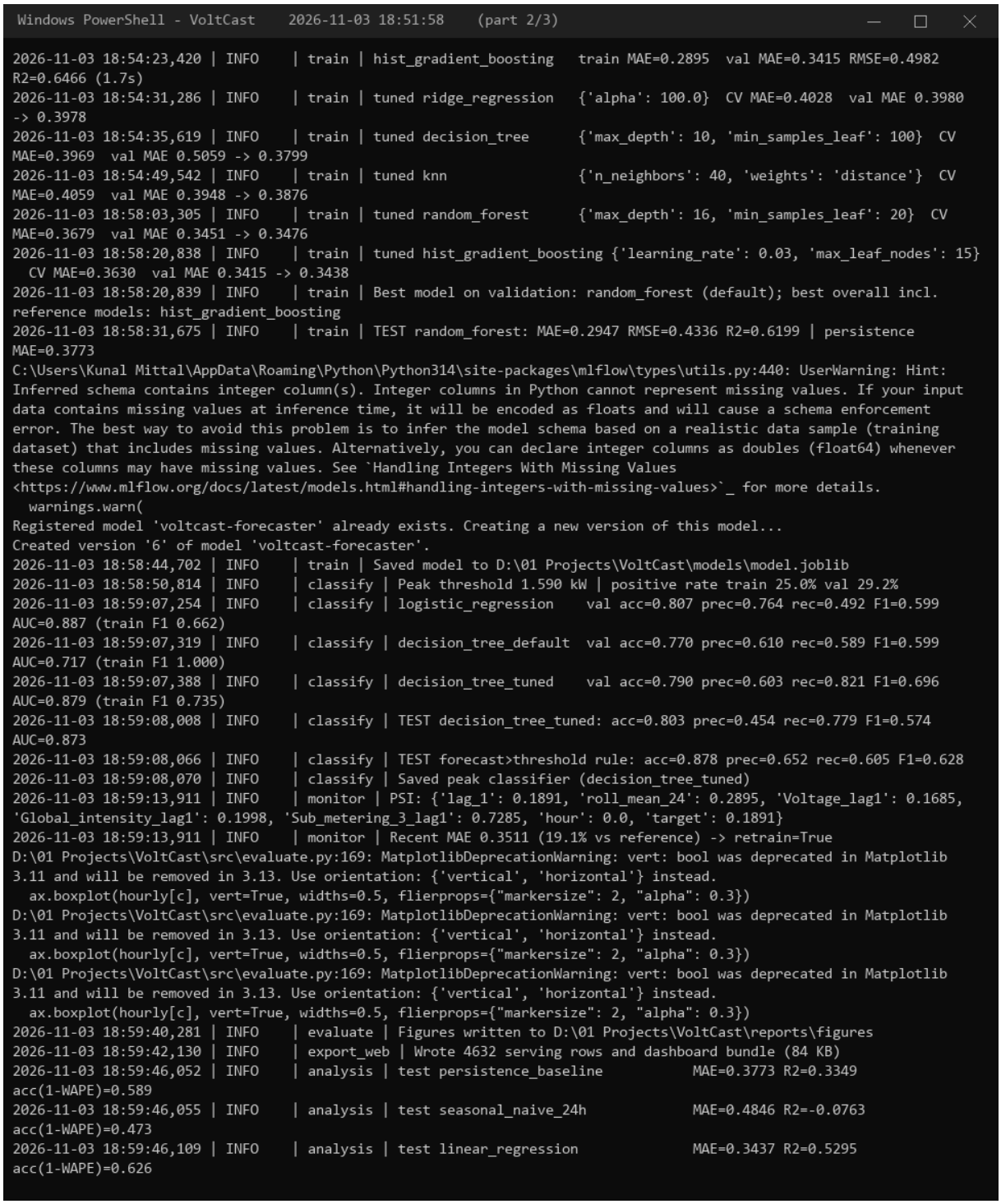


Figure 4.33*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 2 of 3), run 2026-11-03 18:51:58*

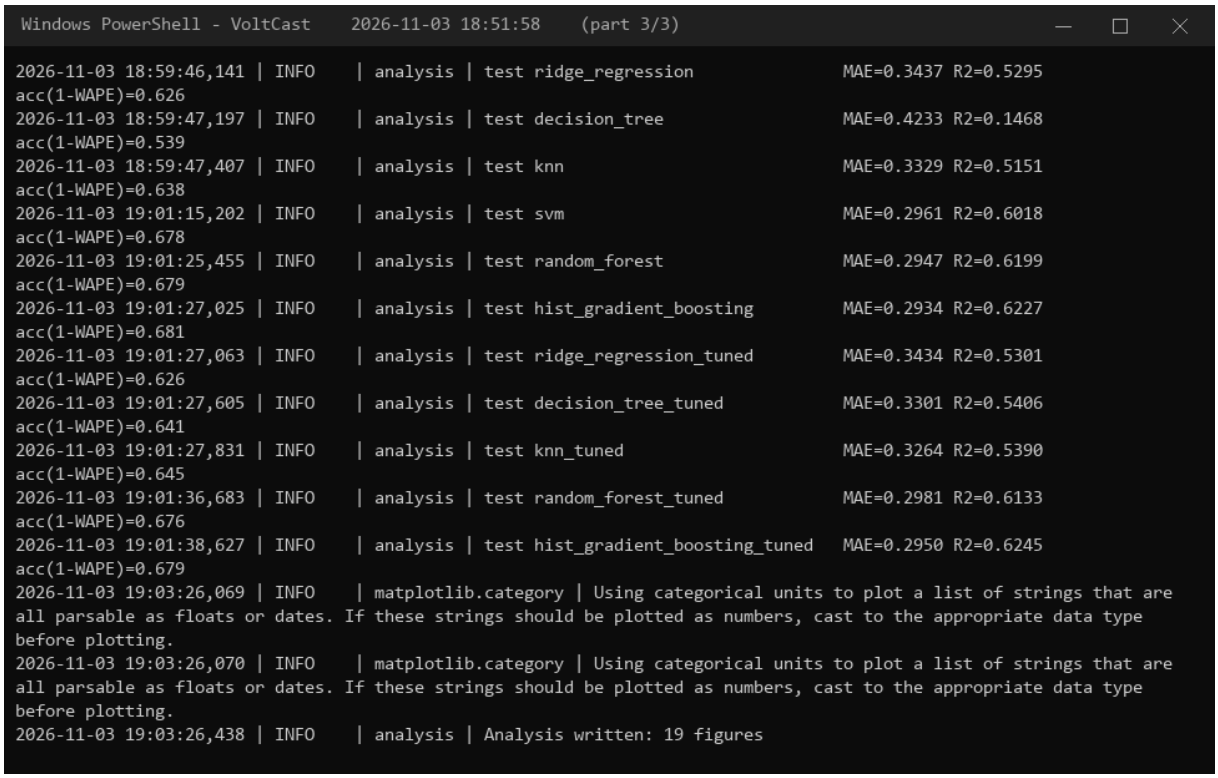


Figure 4.34*: dvc repro --force: full pipeline run (ingest, preprocess, features, training with GridSearchCV, classifier, monitoring, evaluation, export, analysis) (part 3 of 3), run 2026-11-03 18:51:58*

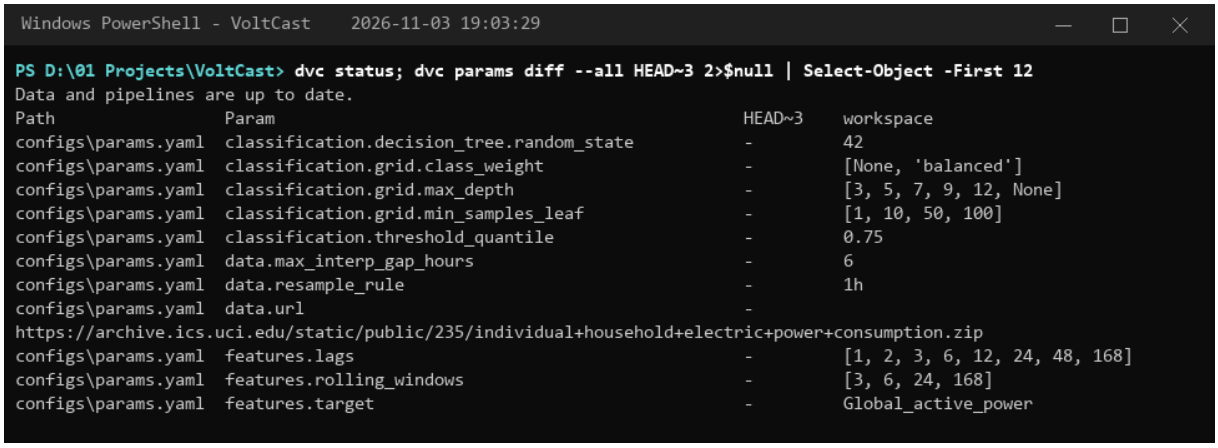


Figure 4.35*: dvc status after the run, run 2026-11-03 19:03:29*

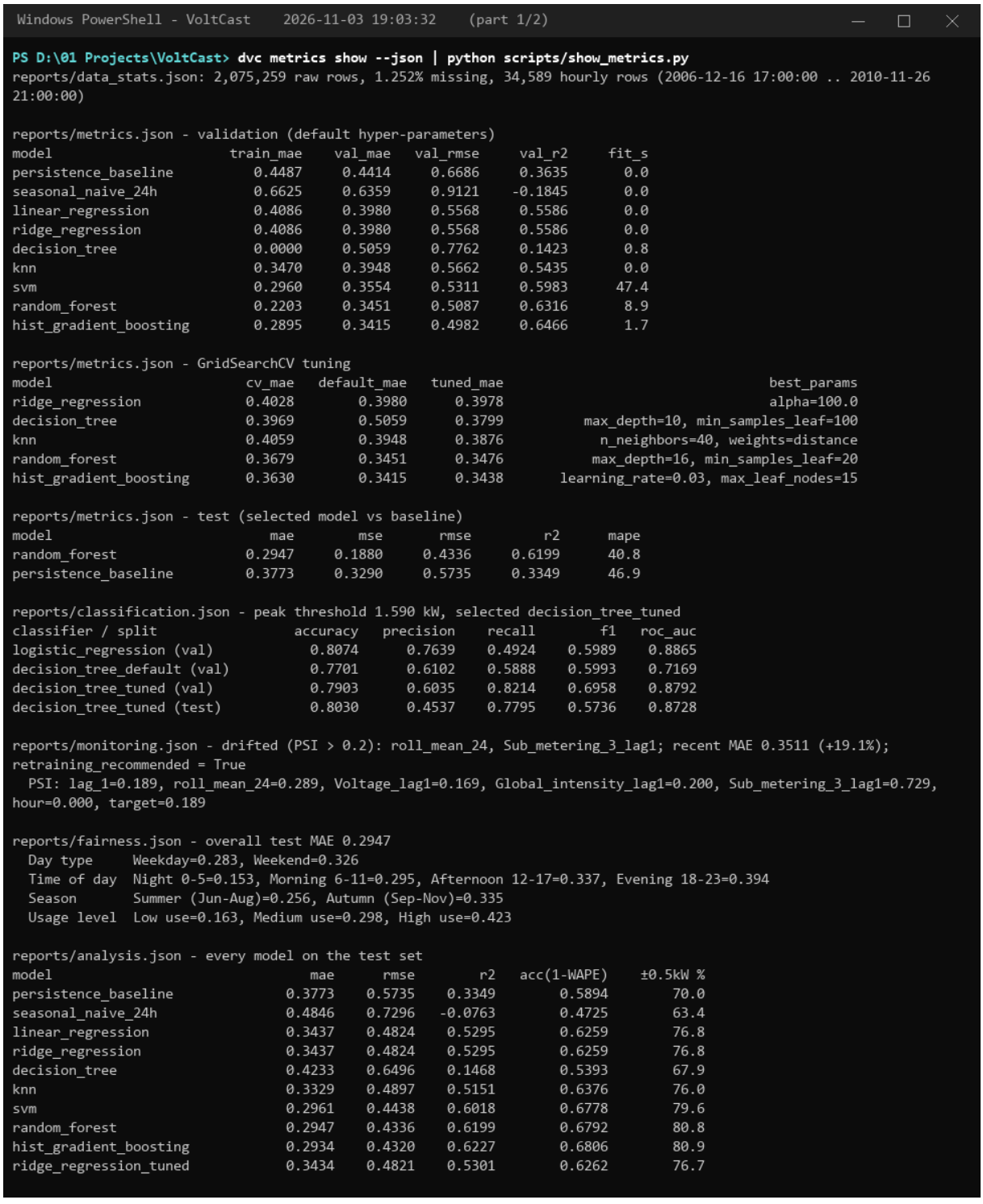


Figure 4.36*: dvc metrics show: every tracked metric (part 1 of 2), run 2026-11-03 19:03:32*

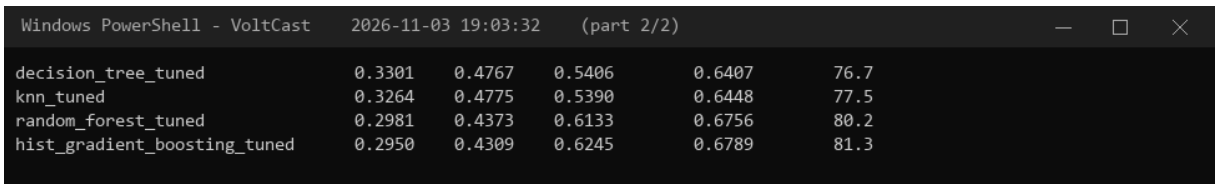


Figure 4.37*: dvc metrics show: every tracked metric (part 2 of 2), run 2026-11-03 19:03:32*

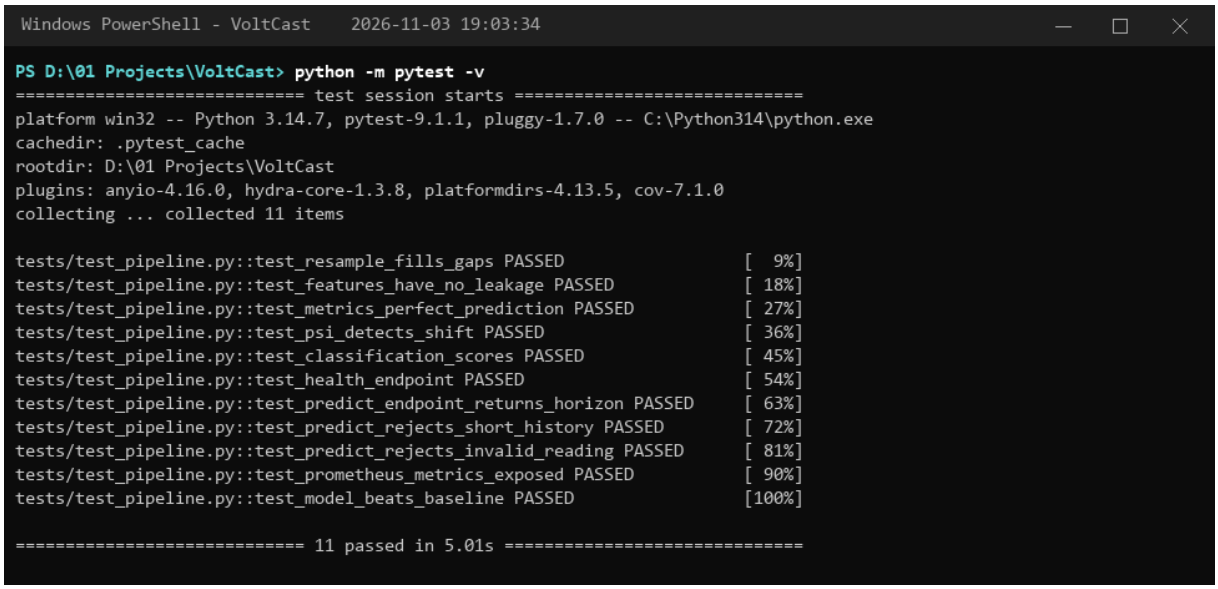


Figure 4.38*: pytest -v: the automated test suite, run 2026-11-03 19:03:34*



Figure 4.39*: MLflow: tracked runs of the forecasting experiment (part 1 of 3), run 2026-11-03 19:03:41*



Figure 4.40*: MLflow: tracked runs of the forecasting experiment (part 2 of 3), run 2026-11-03 19:03:41*

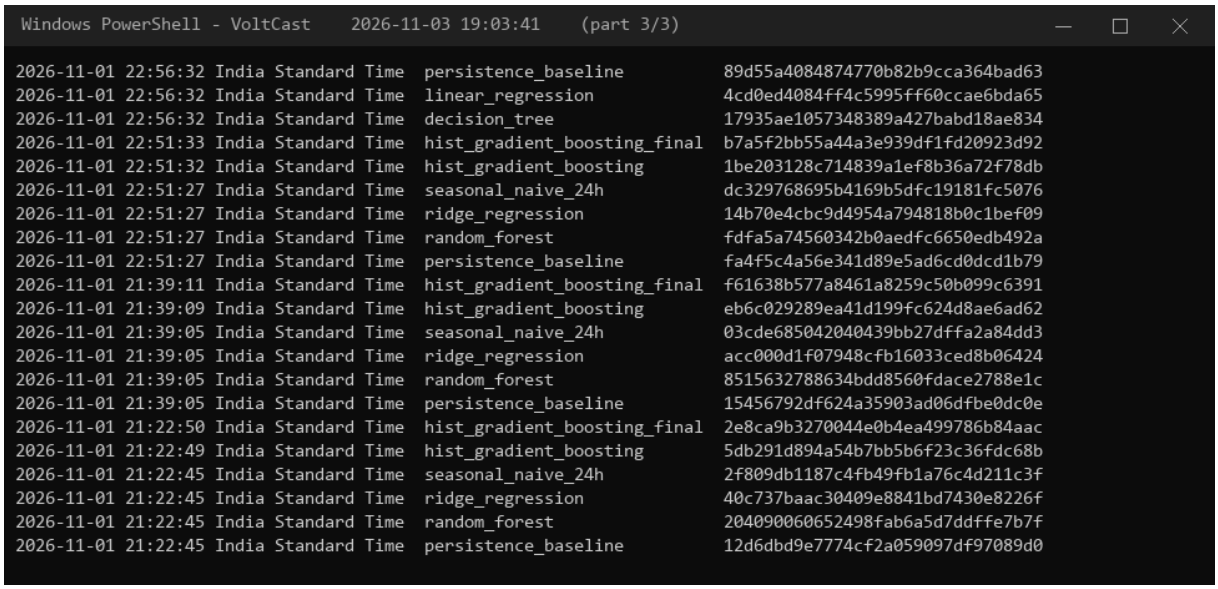


Figure 4.41*: MLflow: tracked runs of the forecasting experiment (part 3 of 3), run 2026-11-03 19:03:41*

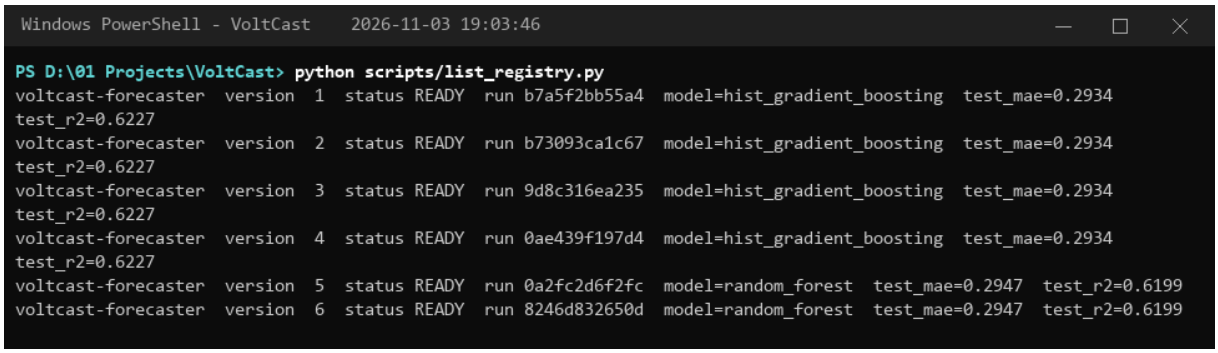


Figure 4.42*: MLflow model registry: versions of voltcast-forecaster, run 2026-11-03 19:03:46*

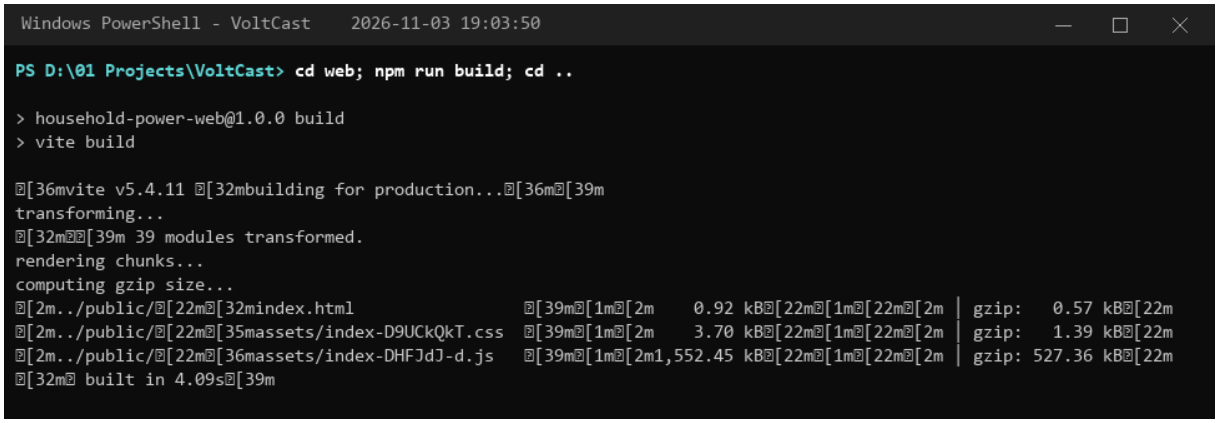


Figure 4.43*: Building the React web application, run 2026-11-03 19:03:50*

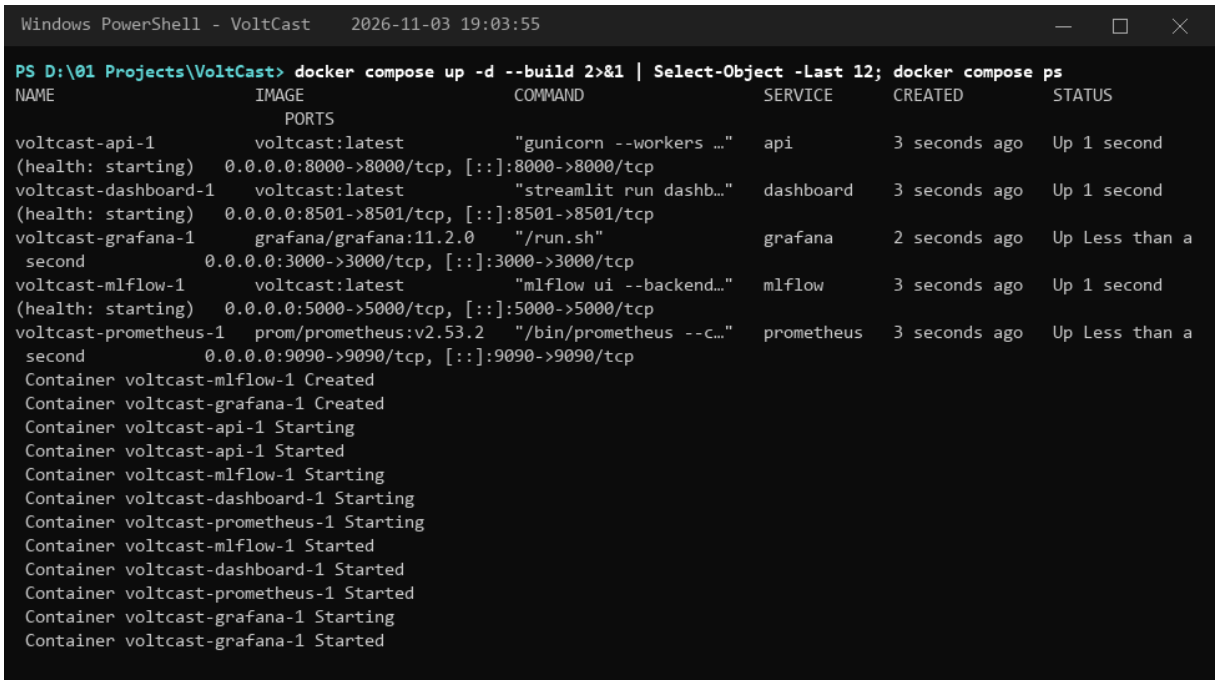


Figure 4.44*: docker compose up: building the image and starting the five services, run 2026-11-03 19:03:55*

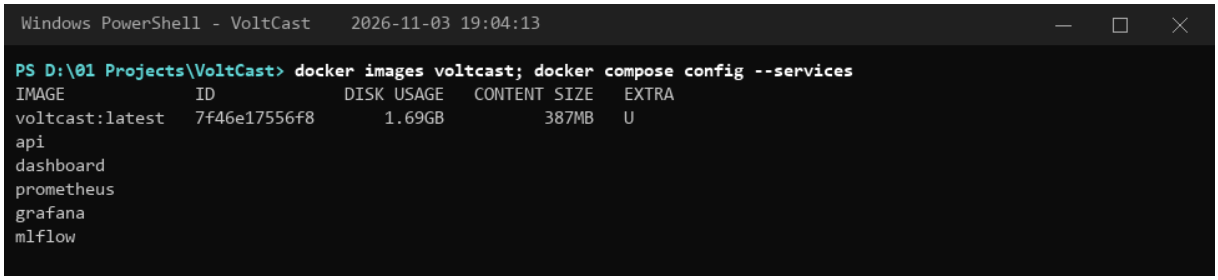


Figure 4.45*: Docker image and Compose services, run 2026-11-03 19:04:13*

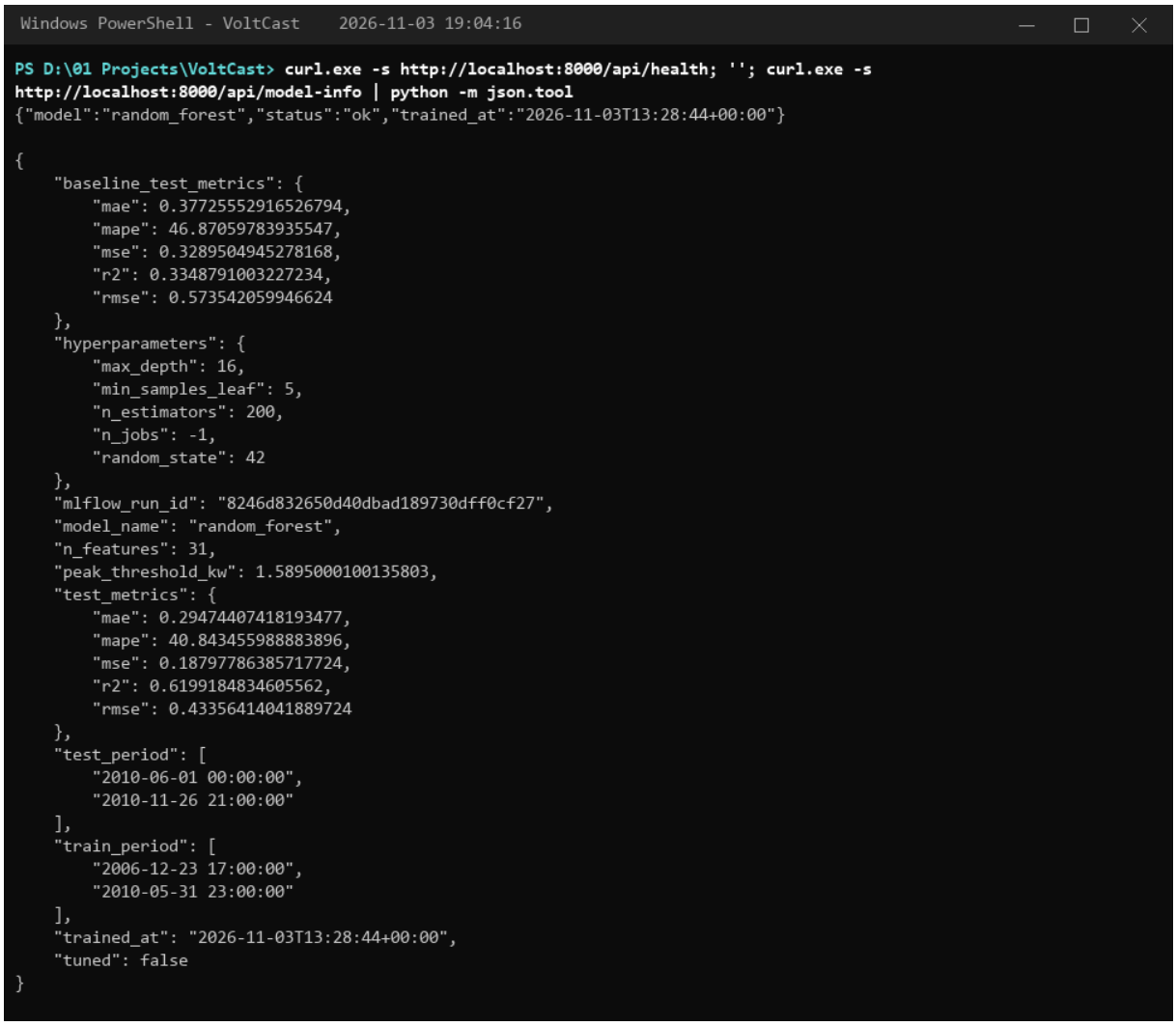


Figure 4.46*: Flask API: health check and model information, run 2026-11-03 19:04:16*

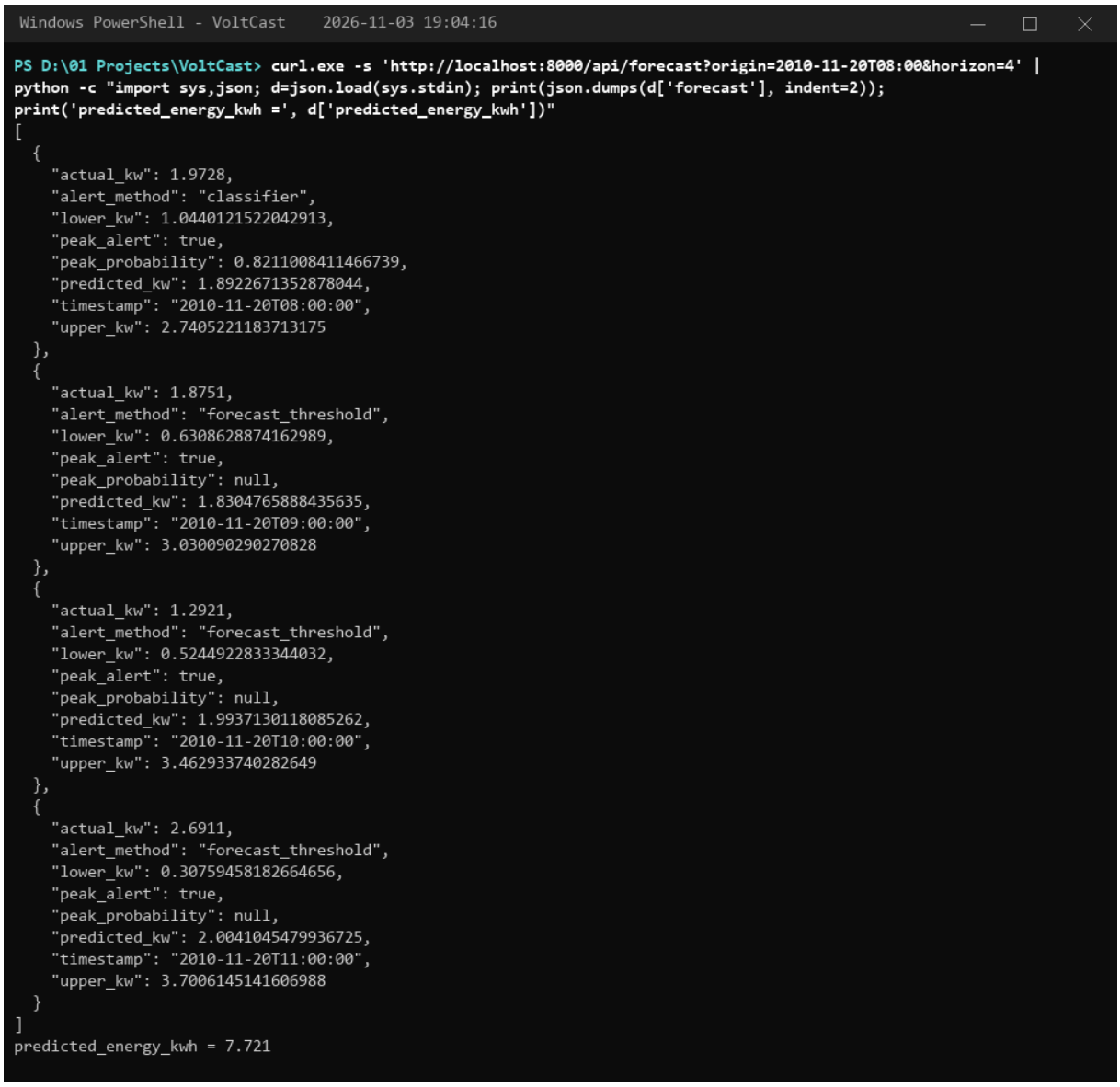


Figure 4.47*: Flask API: 4-hour forecast with peak alerts, run 2026-11-03 19:04:16*

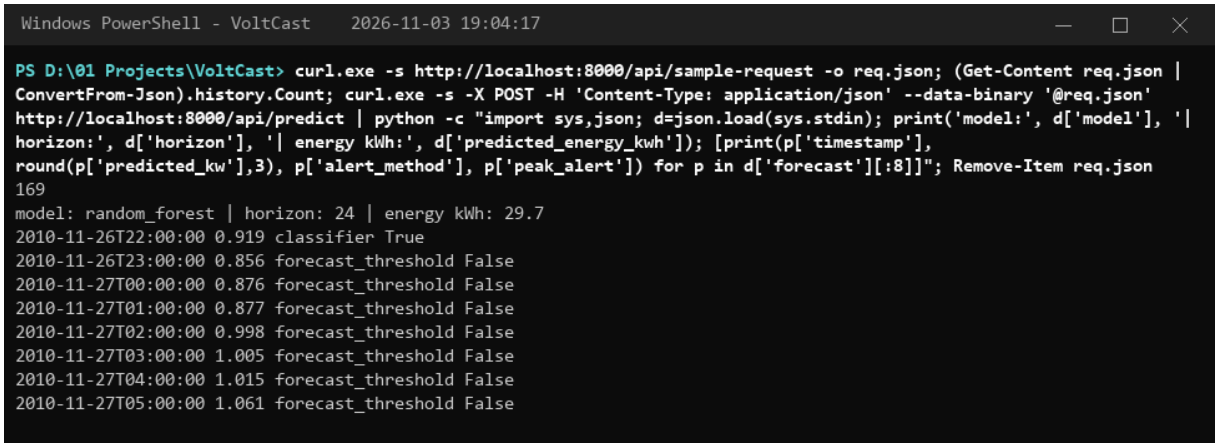


Figure 4.48*: Flask API: POST /api/predict with one week of readings, run 2026-11-03 19:04:17*

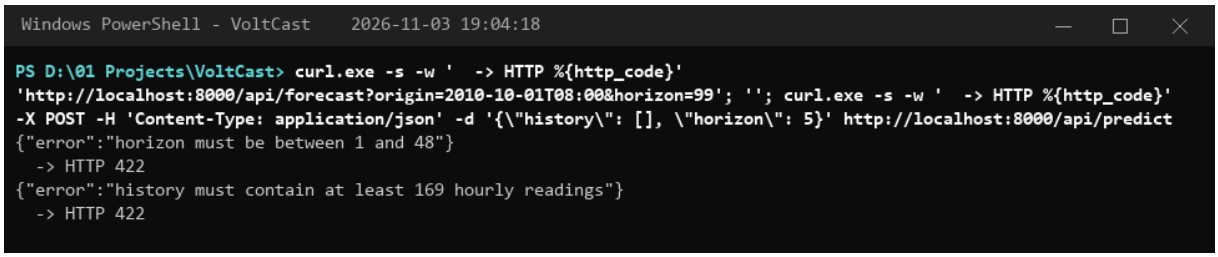


Figure 4.49*: Flask API: invalid requests are rejected with HTTP 422, run 2026-11-03 19:04:18*

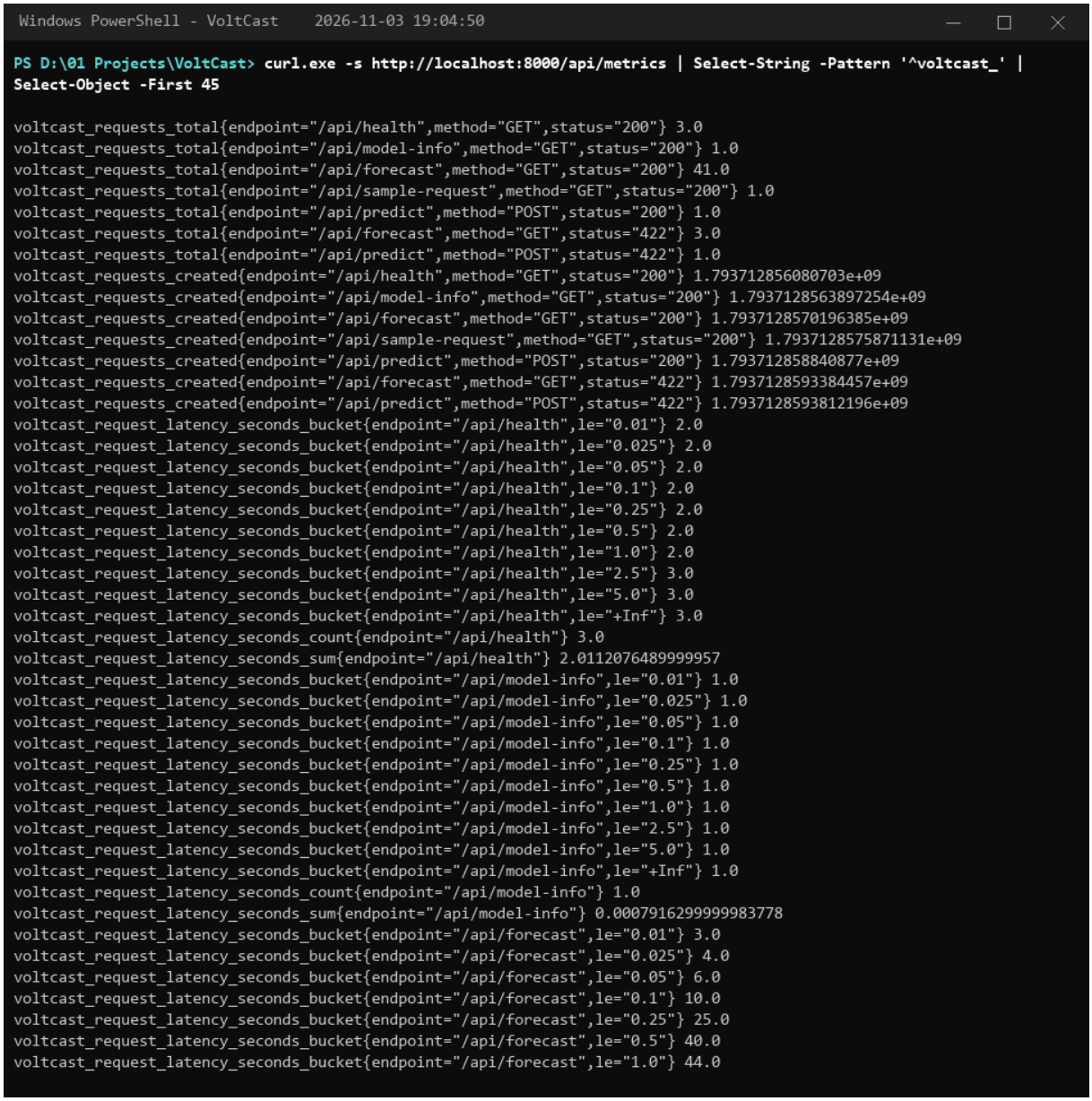


Figure 4.50*: Prometheus metrics exposed by the API at /api/metrics, run 2026-11-03 19:04:50*

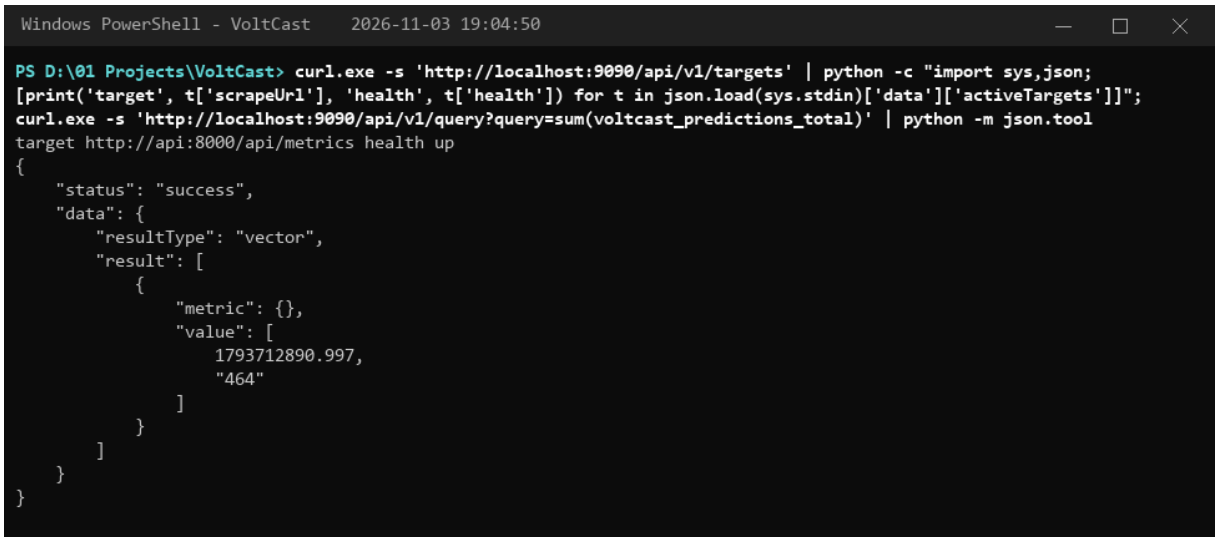


Figure 4.51*: Prometheus: scrape target health and a PromQL query, run 2026-11-03 19:04:50*

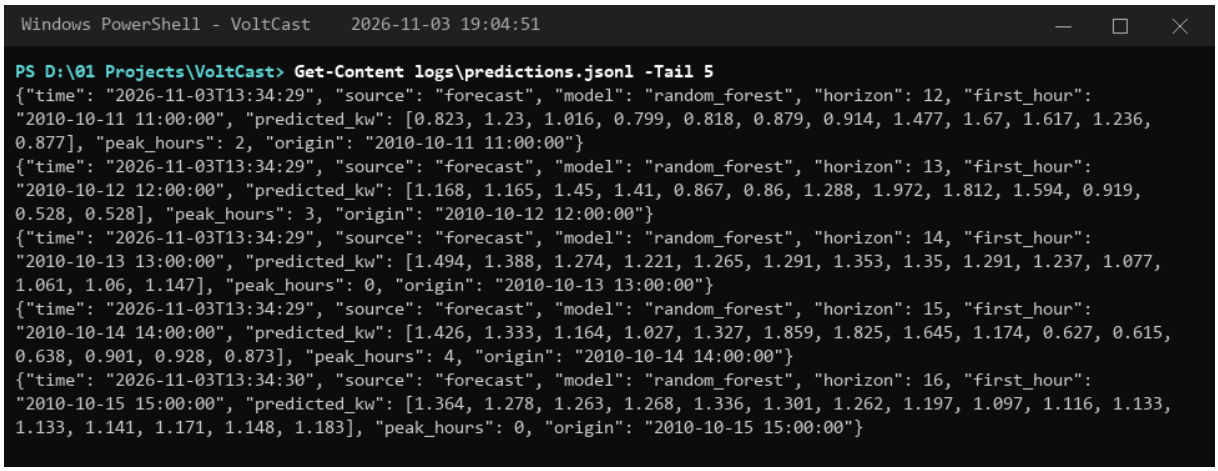


Figure 4.52*: Prediction log (logs/predictions.jsonl), run 2026-11-03 19:04:51*

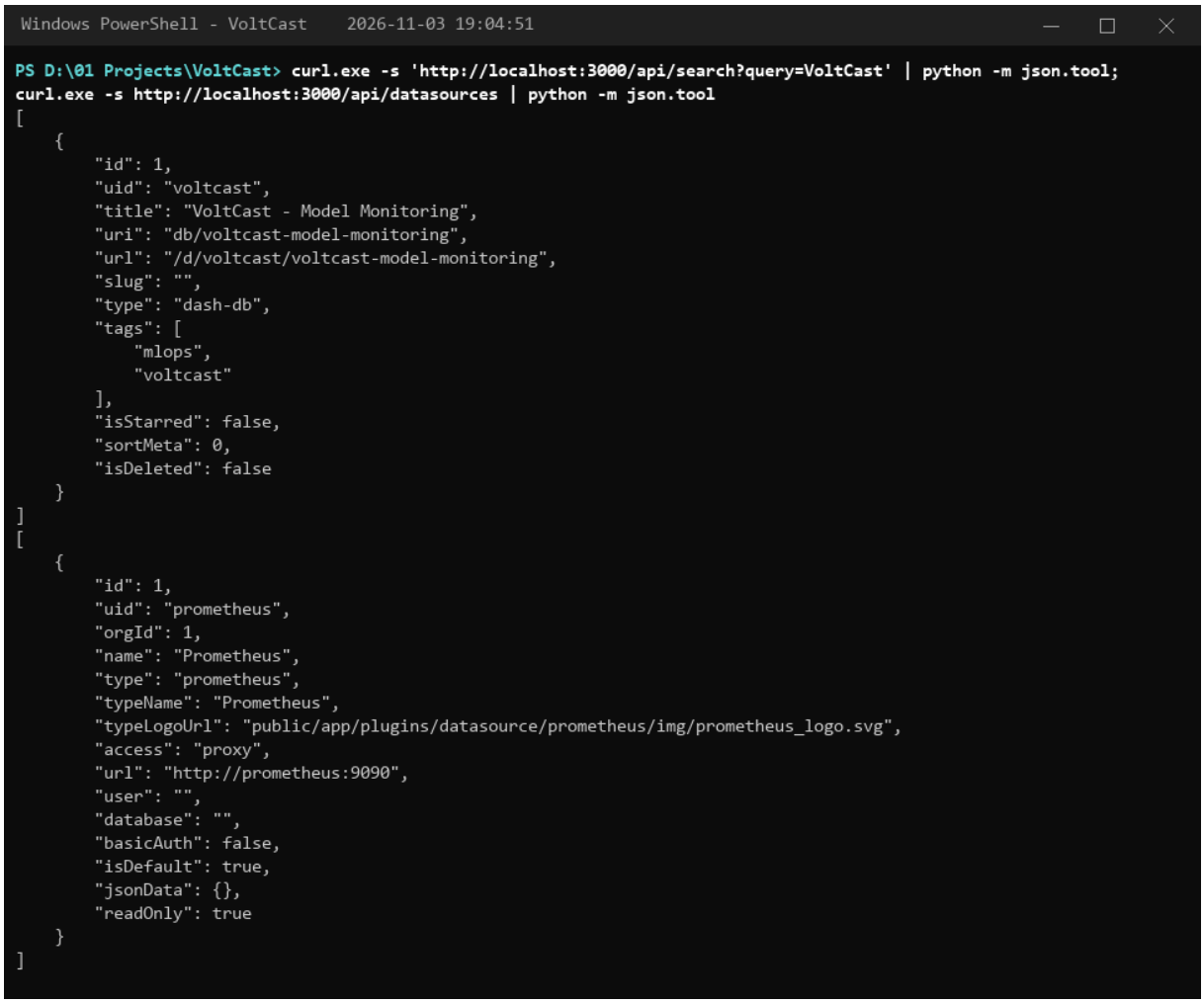


Figure 4.53*: Grafana: provisioned dashboard and data source, run 2026-11-03 19:04:51*

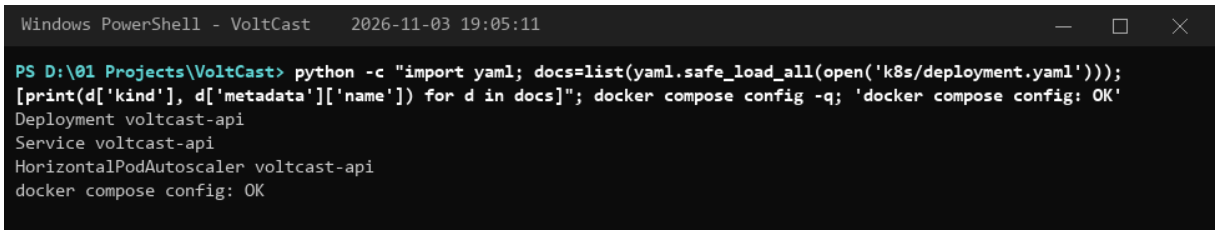


Figure 4.54*: Kubernetes manifest and Docker Compose configuration validation, run 2026-11-03 19:05:11*

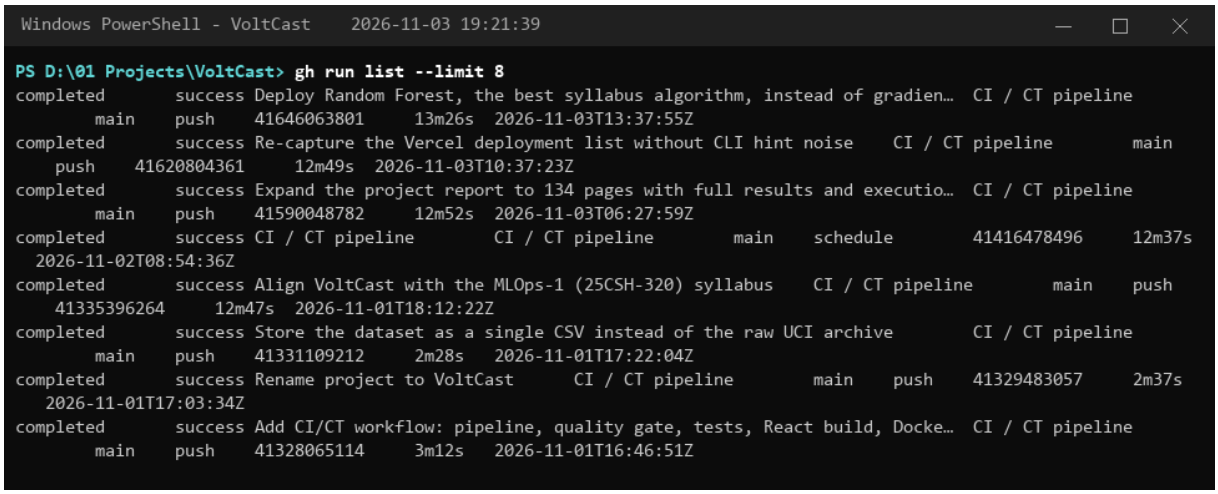


Figure 4.55*: GitHub Actions: recent CI/CD runs, run 2026-11-03 19:21:39*

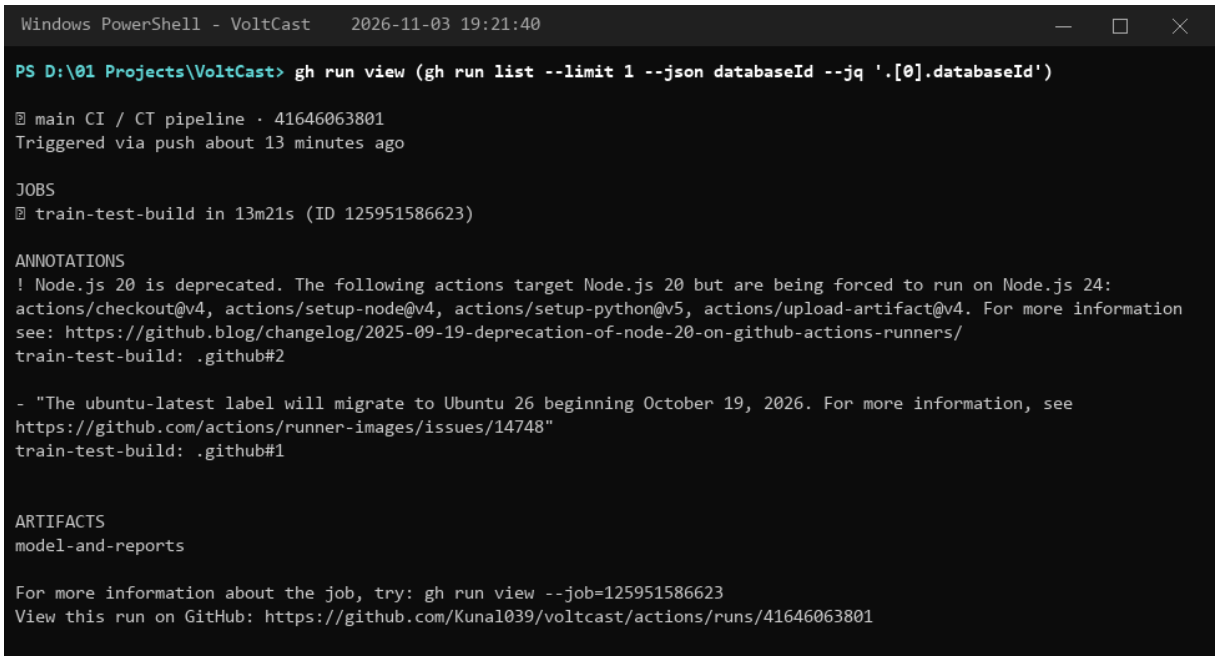


Figure 4.56*: GitHub Actions: steps of the latest run, run 2026-11-03 19:21:40*

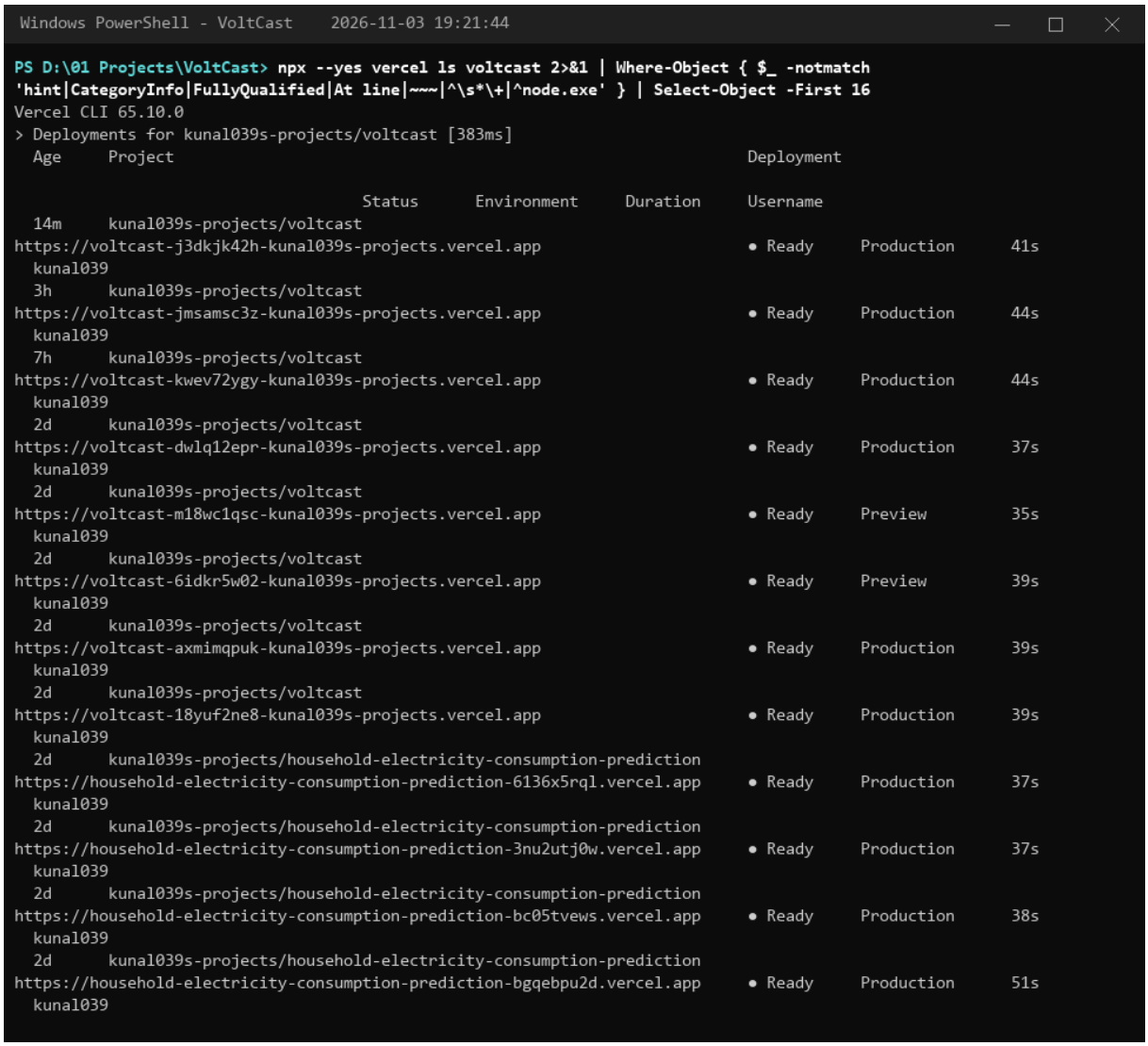


Figure 4.57*: Vercel: production deployments created from GitHub pushes, run 2026-11-03 19:21:44*

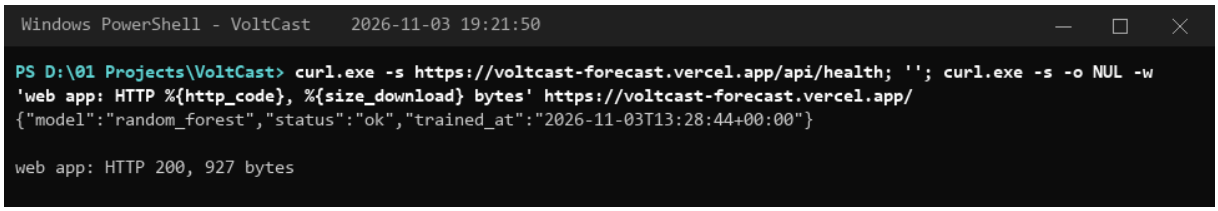


Figure 4.58*: The live deployment answering requests, run 2026-11-03 19:21:50*

4.1.11. Achievement of objectives

Table 4.14*: Achievement of objectives*

Objective (Section 2.6)	Target	Achieved
Clean hourly series	Gap-free, documented	34,589 hourly rows, 0 gaps, outlier report
Feature engineering	≥ 25 leakage-free	31 features, leakage test passes
Course algorithms compared and tuned	LR, DT, KNN, SVM, RF	9 regressors, 5 tuned with GridSearchCV

Objective (Section 2.6)	Target	Achieved
Improvement over baseline	≥ 15% MAE	21.9% MAE, 24.4% RMSE
Peak-hour classifier	Accuracy, P, R, F1, ROC	Accuracy 0.803, ROC-AUC 0.873
Flask API, Docker, cloud	< 1 s response	Flask + Docker + Vercel, sub-second
CI/CD and versioning	Pipeline + gates + tests + build	GitHub Actions, DVC, MLflow registry
Monitoring and logging	Prometheus, Grafana, drift	Dashboard live, 2 drifted features flagged
Ethics	Fairness, privacy, security	Bias analysis, model card, validation

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

VoltCast is a complete, working Household Electricity Consumption Prediction System built on MLOps principles. From 2.07 million raw smart-meter readings, a DVC-versioned pipeline produces a clean hourly series and 31 leakage-free features. Nine regression models, covering every algorithm in the course, are compared and tuned with GridSearchCV under MLflow tracking. The deployed random forest, the best syllabus algorithm and within a few thousandths of a kilowatt of the gradient-boosting reference, forecasts with a test MAE of 0.295 kW and R^2 of 0.620, 21.9% better than the persistence baseline. A tuned decision tree flags peak hours with 78% recall and ROC-AUC 0.873. The models are served by a Flask API in Docker and on the Vercel cloud, protected by CI/CD quality gates and tests, observed through Prometheus and Grafana, and reviewed for fairness, privacy and security.

Expected vs obtained results. The ≥ 15% improvement over the baseline was met. However, the absolute R^2 (~0.62) is lower than the >0.9 values sometimes reported in the literature, for three reasons. (i) We forecast a single household at hourly resolution, where behaviour-driven spikes are inherently unpredictable. (ii) We use a strict chronological split and refuse features computed from the hour being predicted, whereas some studies shuffle data or use same-hour intensity, which inflates scores. (iii) The test period has a different seasonal distribution from training, which the monitor correctly flagged. MAPE (~41%) is inflated by very low night-time loads, where small absolute errors become large percentages; MAE, MSE and R^2 are the more meaningful measures here. Hyper-parameter tuning helped the poorly regularised models (decision tree, KNN) but not the ensembles, whose defaults were already near-optimal. The peak classifier's precision on the test period is lower than on validation because summer has fewer peak hours, which is another effect of seasonal drift.

5.2. Future work

- **Weather and occupancy inputs:** add outdoor temperature (heating/cooling degree hours) and holiday calendars from an API; expected to reduce winter peak error substantially.

- **Deep and probabilistic models:** evaluate LSTM/Temporal Fusion Transformer and quantile gradient boosting for calibrated prediction intervals [12], [15].
- **Multi-household, Indian data:** train a global model across many homes using data from IS 16444 smart meters, with federated learning to protect privacy.
- **Automated retraining:** connect the monitor's retraining flag to a workflow trigger and promote a new model only if it beats the current one (champion/challenger) in the MLflow registry.
- **Production Kubernetes:** apply the manifests to a managed cluster (AWS EKS, GCP GKE or Azure AKS) with Prometheus service discovery and Grafana alerting rules.
- **Actionable recommendations:** combine forecasts with time-of-day tariffs to suggest the cheapest time to run flexible appliances.

REFERENCES

[1] International Energy Agency, “Buildings – Energy System,” IEA, Paris. [Online]. Available: https://www.iea.org/energy-system/buildings

[2] Central Electricity Authority, “General Review (All India Electricity Statistics),” Ministry of Power, Government of India, New Delhi.

[3] Ministry of Power, Government of India, “Revamped Distribution Sector Scheme (RDSS),” 2021. [Online]. Available: https://recindia.nic.in/revamped-distribution-sector-scheme

[4] D. Sculley et al., “Hidden technical debt in machine learning systems,” in Advances in Neural Information Processing Systems 28, 2015.

[5] D. Kreuzberger, N. Kühl and S. Hirschl, “Machine Learning Operations (MLOps): Overview, definition, and architecture,” IEEE Access, vol. 11, pp. 31866–31879, 2023.

[6] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.

[7] G. W. Hart, “Nonintrusive appliance load monitoring,” Proceedings of the IEEE, vol. 80, no. 12, pp. 1870–1891, 1992.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] G. Hebrail and A. Berard, “Individual Household Electric Power Consumption,” UCI Machine Learning Repository, 2012. doi: 10.24432/C58K54.

[11] D. L. Marino, K. Amarasinghe and M. Manic, “Building energy load forecasting using deep neural networks,” in Proc. IECON 2016 – 42nd Annual Conference of the IEEE Industrial Electronics Society, 2016, pp. 7046–7051.

[12] T.-Y. Kim and S.-B. Cho, “Predicting residential energy consumption using CNN-LSTM neural networks,” Energy, vol. 182, pp. 72–81, 2019.

[13] K. Amasyali and N. M. El-Gohary, “A review of data-driven building energy consumption prediction studies,” Renewable and Sustainable Energy Reviews, vol. 81, pp. 1192–1205, 2018.

[14] M. Zaharia et al., “Accelerating the machine learning lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[15] T. Hong and S. Fan, “Probabilistic electric load forecasting: A tutorial review,” International Journal of Forecasting, vol. 32, no. 3, pp. 914–938, 2016.

[16] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne: OTexts, 2021.

[17] C. Fan, F. Xiao and S. Wang, “Development of prediction models for next-day building energy consumption and peak power demand using data mining techniques,” Applied Energy, vol. 127, pp. 1–10, 2014.

[18] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30, 2017.

[19] S. Rabanser, S. Günnemann and Z. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in Advances in Neural Information Processing Systems 32, 2019.

[20] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[21] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.

[22] A. C. Müller and S. Guido, Introduction to Machine Learning with Python: A Guide for Data Scientists. Sebastopol, CA: O'Reilly Media, 2016.

[23] E. Ameisen, Building Machine Learning Powered Applications: Going from Idea to Product. Sebastopol, CA: O'Reilly Media, 2020.

[24] A. Ng, Machine Learning Yearning, 2018. [Online]. Available: https://www.deeplearning.ai/machine-learning-yearning/

[25] Government of India, “The Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023, Ministry of Law and Justice, New Delhi, 2023.

APPENDIX

1. Plagiarism Report

2. Design Checklist

Table A.1*: Design checklist*

Item	Status
Problem identified and justified with documented sources	✓
Literature survey with comparison of existing solutions	✓
Measurable objectives defined and mapped to course outcomes	✓
Design constraints (standards, cost, ethics, privacy, security, safety) considered	✓
At least two alternative designs compared and one selected with reasons	✓
Flowchart / block diagram of implementation	✓
Data preprocessing: missing values, outliers, normalisation, chronological split	✓
Linear regression, decision tree, KNN, SVM and random forest implemented and compared	✓
GridSearchCV tuning, cross-validation, over-/under-fitting analysis	✓
Classification metrics: accuracy, precision, recall, F1, confusion matrix, ROC	✓
Version control with Git; data and pipeline versioning with DVC; MLflow registry	✓
Flask REST API, Docker containerisation, cloud deployment, Kubernetes manifests	✓
CI/CD with GitHub Actions and automatic deployment	✓
Monitoring with Prometheus and Grafana; prediction logging; drift detection	✓
Fairness, transparency, privacy and security analysed (model card)	✓
User manual with step-by-step instructions	✓

3. Syllabus experiments performed in the project

The practical experiments of MLOps-1 (25CSH-320) were carried out on this project's data instead of the sample datasets suggested in the syllabus. Table A.2 maps each experiment to the code and the evidence in this report.

Table A.2*: Syllabus experiments mapped to the project*

Exp.	Aim (syllabus)	How it was done in VoltCast	Code	Evidence
1	Preprocess data: cleaning, missing values, normalisation, split	1.25% missing readings filled (interpolation + weekly fill), IQR outlier report, StandardScaler for linear/KNN/SVM, chronological train/validation/test split	src/preprocess.py, src/train.py	3.1, 4.1.1
2	Linear regression	Linear regression: test MSE	src/train.py	4.1.2, 4.1.5

Exp.	Aim (syllabus)	How it was done in VoltCast	Code	Evidence
	evaluated with MSE	0.233, MAE 0.344, R^2 0.530		
3	Decision tree classifier: accuracy, confusion matrix, precision-recall	Peak-hour decision tree: accuracy 0.803, precision 0.454, recall 0.779; confusion matrix, ROC and PR curves	src/classify.py	4.1.6
4	GridSearchCV tuning; tuned vs default	Grids for 5 regressors and the classifier, TimeSeriesSplit(5); tree MAE 0.506 → 0.380	src/train.py, src/classify.py	4.1.4
5	Version-controlled repository; ML in a DevOps pipeline	Git + GitHub (Kunal039/voltcast), DVC pipeline, GitHub Actions	dvc.yaml, .github/	4.1.8, 4.1.11
6	Containerise the model with Docker	Multi-stage Dockerfile (Node build + Python runtime), five-service Docker Compose stack built and run	Dockerfile, docker-compose.yml	4.1.7, 4.1.11
7	Flask REST API; Prometheus metrics	Flask blueprint with 6 endpoints, Pydantic validation, Prometheus counters and histograms	serving/main.py, app.py	4.1.7, 4.1.9
8	CI/CD with GitHub Actions	dvc repro, quality gates, pytest, React and Docker builds; Vercel deploys every push	.github/workflows/ci.yml	4.1.8, 4.1.11
9	Model versioning with DVC	dvc.yaml with 9 stages, dvc.lock, dvc metrics; MLflow model registry	dvc.yaml, dvc.lock	4.1.8, 4.1.11
10	Monitoring and logging with Prometheus and Grafana	Prometheus scrape config, provisioned Grafana dashboard, JSON prediction log, PSI drift	monitoring/, src/monitor.py	4.1.9, 4.1.11
11–15	Mini project phases 1–5	Problem and data (Ch. 1–3), models (4.1.2–4.1.6), deployment (4.1.7), automation (4.1.8), monitoring and ethics (4.1.9–4.1.10)	whole repository	Ch. 1–5

4. Source code

This section lists the complete source code of the project as submitted (repository https://github.com/Kunal039/voltcast). Generated files (trained models, metrics, figures), third-party packages and the report-generation scripts are not listed.

configs/params.yaml: central configuration

```
  1  # Central configuration for the VoltCast (Household Electricity Consumption Prediction) pipeline.
  2  data:
  3    url:
"https://archive.ics.uci.edu/static/public/235/individual+household+electric+power+consumption.zip"
  4    raw_csv: data/household_power_consumption.csv
  5    processed: data/processed/hourly.parquet
  6    features: data/processed/features.parquet
  7    serving: data/serving/recent_hourly.csv
  8    web_data: web/public/data/dashboard.json
```

```yaml
  resample_rule: "1h"
  max_interp_gap_hours: 6

split:
  # Chronological split: no shuffling, so no future data leaks into training.
  train_end: "2009-12-31 23:00:00"
  val_end: "2010-05-31 23:00:00"

features:
  target: Global_active_power
  lags: [1, 2, 3, 6, 12, 24, 48, 168]
  rolling_windows: [3, 6, 24, 168]

# Default hyper-parameters of each candidate (keys match the model names used in MLflow).
models:
  linear_regression: {}
  ridge_regression:
    alpha: 1.0
  decision_tree:
    random_state: 42
  knn:
    n_neighbors: 10
  svm:
    C: 1.0
    epsilon: 0.1
  random_forest:
    n_estimators: 200
    max_depth: 16
    min_samples_leaf: 5
    n_jobs: -1
    random_state: 42
  hist_gradient_boosting:
    max_iter: 500
    learning_rate: 0.05
    max_leaf_nodes: 31
    l2_regularization: 1.0
    random_state: 42

# Only algorithms from the MLOps-1 syllabus may be deployed. Gradient boosting and the
# naive baselines are still trained and reported as reference points.
selection:
  deployable: [linear_regression, ridge_regression, decision_tree, knn, svm, random_forest]

# GridSearchCV with time-series cross-validation (folds never train on the future).
tuning:
  cv_splits: 5
  scoring: neg_mean_absolute_error
  grids:
    ridge_regression:
      alpha: [0.01, 0.1, 1.0, 10.0, 100.0]
    decision_tree:
      max_depth: [4, 6, 8, 10, 14, null]
      min_samples_leaf: [1, 10, 50, 100]
    knn:
      n_neighbors: [5, 10, 20, 40]
      weights: [uniform, distance]
    random_forest:
      max_depth: [10, 16, null]
      min_samples_leaf: [1, 5, 20]
    hist_gradient_boosting:
      learning_rate: [0.03, 0.05, 0.1]
      max_leaf_nodes: [15, 31, 63]

# Peak-hour alert: classify whether the next hour is a high-consumption hour.
classification:
  threshold_quantile: 0.75   # of training-period consumption
  decision_tree:
    random_state: 42
  grid:
    max_depth: [3, 5, 7, 9, 12, null]
    min_samples_leaf: [1, 10, 50, 100]
    class_weight: [null, balanced]

mlflow:
  tracking_uri: "sqlite:///mlflow.db"
  experiment: household-electricity-forecasting
  registered_model: voltcast-forecaster

monitoring:
```

```
  88    psi_threshold: 0.2
  89    mae_degradation_pct: 20
```

dvc.yaml: DVC pipeline definition

```
   1  # Reproducible pipeline (`dvc repro`), mirrors run_pipeline.py.
   2  # The dataset and processed data are versioned in the DVC cache; small artifacts that the
   3  # hosted app needs (model, metrics, serving data) are also committed to Git, so they use cache:
false.
   4  stages:
   5    ingest:
   6      cmd: python -m src.data_ingestion
   7      deps: [src/data_ingestion.py]
   8      params: [{configs/params.yaml: [data.url]}]
   9      outs: [data/household_power_consumption.csv]
  10    preprocess:
  11      cmd: python -m src.preprocess
  12      deps: [src/preprocess.py, data/household_power_consumption.csv]
  13      params: [{configs/params.yaml: [data.resample_rule, data.max_interp_gap_hours]}]
  14      outs: [data/processed/hourly.parquet]
  15      metrics: [reports/data_stats.json: {cache: false}]
  16    features:
  17      cmd: python -m src.features
  18      deps: [src/features.py, data/processed/hourly.parquet]
  19      params: [{configs/params.yaml: [features]}]
  20      outs: [data/processed/features.parquet]
  21    train:
  22      cmd: python -m src.train
  23      deps: [src/train.py, src/features.py, data/processed/features.parquet]
  24      params: [{configs/params.yaml: [split, models, tuning]}]
  25      outs:
  26        - models/model.joblib: {cache: false}
  27        - models/metadata.json: {cache: false}
  28        - reports/test_predictions.parquet: {cache: false}
  29      metrics: [reports/metrics.json: {cache: false}]
  30    classify:
  31      cmd: python -m src.classify
  32      deps: [src/classify.py, data/processed/features.parquet, reports/test_predictions.parquet]
  33      params: [{configs/params.yaml: [split, classification]}]
  34      outs: [models/peak_classifier.joblib: {cache: false}, models/peak_classifier.json: {cache:
false}]
  35      metrics: [reports/classification.json: {cache: false}]
  36    monitor:
  37      cmd: python -m src.monitor
  38      deps: [src/monitor.py, models/model.joblib, reports/test_predictions.parquet]
  39      params: [{configs/params.yaml: [monitoring]}]
  40      metrics: [reports/monitoring.json: {cache: false}]
  41    evaluate:
  42      cmd: python -m src.evaluate
  43      deps: [src/evaluate.py, models/model.joblib, reports/monitoring.json,
reports/classification.json]
  44      outs: [reports/figures: {cache: false}]
  45      metrics: [reports/fairness.json: {cache: false}]
  46    export_web:
  47      cmd: python -m src.export_web
  48      deps: [src/export_web.py, models/model.joblib, reports/monitoring.json,
reports/classification.json]
  49      outs:
  50        - data/serving/recent_hourly.csv: {cache: false}
  51        - web/public/data/dashboard.json: {cache: false}
  52    analyze:
  53      cmd: python -m src.analysis
  54      deps: [src/analysis.py, src/train.py, data/processed/features.parquet, models/model.joblib,
  55             models/peak_classifier.json, reports/metrics.json, data/serving/recent_hourly.csv]
  56      params: [{configs/params.yaml: [split, models, tuning, classification]}]
  57      outs: [reports/analysis: {cache: false}]
  58      metrics: [reports/analysis.json: {cache: false}]
```

run_pipeline.py: pipeline runner

```
  1  """Run the full MLOps pipeline end to end: ingest -> preprocess -> features -> train -> classify ->
monitor -> evaluate -> export -> analysis."""
  2  import time
  3
  4  from src import analysis, classify, data_ingestion, evaluate, export_web, features, monitor,
preprocess, train
  5  from src.utils import get_logger
  6
  7  log = get_logger("pipeline")
  8
  9  STAGES = [
 10      ("ingestion", data_ingestion.main),
 11      ("preprocess", preprocess.main),
 12      ("features", features.main),
 13      ("train", train.main),
 14      ("classify", classify.main),
 15      ("monitor", monitor.main),
 16      ("evaluate", evaluate.main),
 17      ("export_web", export_web.main),
 18      ("analysis", analysis.main),
 19  ]
 20
 21  if __name__ == "__main__":
 22      for name, fn in STAGES:
 23          t0 = time.perf_counter()
 24          log.info("=== Stage: %s ===", name)
 25          fn()
 26          log.info("Stage %s finished in %.1fs", name, time.perf_counter() - t0)
```

src/utils.py: shared helpers

```
  1  """Shared helpers: config loading, paths and logging."""
  2  import logging
  3  from pathlib import Path
  4
  5  import yaml
  6
  7  ROOT = Path(__file__).resolve().parents[1]
  8
  9
 10  def load_params(path: Path | str = ROOT / "configs" / "params.yaml") -> dict:
 11      with open(path, "r", encoding="utf-8") as f:
 12          return yaml.safe_load(f)
 13
 14
 15  def resolve(rel: str) -> Path:
 16      p = ROOT / rel
 17      p.parent.mkdir(parents=True, exist_ok=True)
 18      return p
 19
 20
 21  def get_logger(name: str) -> logging.Logger:
 22      logging.basicConfig(
 23          level=logging.INFO,
 24          format="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
 25      )
 26      return logging.getLogger(name)
```

src/data_ingestion.py: download, CSV conversion and schema check

```
  1  """Stage 1 - Data ingestion: download the UCI dataset once, store it as a clean CSV, validate its
schema.
  2
  3  The UCI archive holds a semicolon-separated .txt with '?' for missing readings and
  4  separate Date/Time columns. It is converted to a standard CSV with one Datetime
  5  column and blank cells for missing values; the downloaded archive is not kept.
  6  """
  7  import io
```

```
import tempfile
import zipfile
from pathlib import Path

import pandas as pd
import requests

from src.utils import get_logger, load_params, resolve

log = get_logger("ingestion")

EXPECTED_COLUMNS = [
    "Datetime", "Global_active_power", "Global_reactive_power", "Voltage",
    "Global_intensity", "Sub_metering_1", "Sub_metering_2", "Sub_metering_3",
]


def download(params: dict) -> None:
    csv_path = resolve(params["data"]["raw_csv"])
    if csv_path.exists():
        log.info("Dataset already present at %s - skipping download", csv_path)
        return
    log.info("Downloading dataset from %s", params["data"]["url"])
    with tempfile.TemporaryDirectory() as tmp:
        zip_path = Path(tmp) / "dataset.zip"
        with requests.get(params["data"]["url"], stream=True, timeout=120) as r:
            r.raise_for_status()
            with open(zip_path, "wb") as f:
                for chunk in r.iter_content(chunk_size=1 << 20):
                    f.write(chunk)
        with zipfile.ZipFile(zip_path) as z:
            name = next(n for n in z.namelist() if n.endswith(".txt"))
            raw = pd.read_csv(io.TextIOWrapper(z.open(name), encoding="utf-8"),
                              sep=";", na_values=["?"], low_memory=False)
    raw.insert(0, "Datetime", pd.to_datetime(raw["Date"] + " " + raw["Time"], format="%d/%m/%Y
%H:%M:%S"))
    raw.drop(columns=["Date", "Time"]).to_csv(csv_path, index=False, date_format="%Y-%m-%d
%H:%M:%S")
    log.info("Saved %d rows to %s", len(raw), csv_path)


def validate_schema(params: dict) -> None:
    with open(resolve(params["data"]["raw_csv"]), "r", encoding="utf-8") as f:
        header = f.readline().strip().split(",")
    if header != EXPECTED_COLUMNS:
        raise ValueError(f"Unexpected schema: {header}")
    log.info("Schema validation passed (%d columns)", len(header))


def main() -> None:
    params = load_params()
    download(params)
    validate_schema(params)


if __name__ == "__main__":
    main()
```

src/preprocess.py: cleaning, resampling and outlier report

```
"""Stage 2 - Preprocessing: clean minute-level readings and resample to hourly."""
import json

import numpy as np
import pandas as pd

from src.utils import ROOT, get_logger, load_params, resolve

log = get_logger("preprocess")

NUMERIC = [
    "Global_active_power", "Global_reactive_power", "Voltage", "Global_intensity",
    "Sub_metering_1", "Sub_metering_2", "Sub_metering_3",
]

```

```
 16
 17  def load_raw(path) -> pd.DataFrame:
 18      df = pd.read_csv(path, parse_dates=["Datetime"], index_col="Datetime").sort_index()
 19      df.index.name = "datetime"
 20      df[NUMERIC] = df[NUMERIC].astype("float32")
 21      return df
 22
 23
 24  def clean_and_resample(df: pd.DataFrame, rule: str, max_gap: int) -> pd.DataFrame:
 25      # Energy not captured by the three sub-meters (Wh per minute).
 26      df["Sub_metering_other"] = (
 27          df["Global_active_power"] * 1000 / 60
 28          - df["Sub_metering_1"] - df["Sub_metering_2"] - df["Sub_metering_3"]
 29      ).clip(lower=0)
 30      agg = {c: "mean" for c in ["Global_active_power", "Global_reactive_power", "Voltage",
"Global_intensity"]}
 31      agg.update({c: "sum" for c in ["Sub_metering_1", "Sub_metering_2", "Sub_metering_3",
"Sub_metering_other"]})
 32      # min_count=1 keeps all-missing hours as NaN instead of 0 for summed columns.
 33      hourly = df.resample(rule).agg(agg)
 34      counts = df["Global_active_power"].resample(rule).count()
 35      for c in ["Sub_metering_1", "Sub_metering_2", "Sub_metering_3", "Sub_metering_other"]:
 36          hourly.loc[counts == 0, c] = np.nan
 37      # Fill short outages by time interpolation; longer gaps use same-hour-last-week values.
 38      hourly = hourly.interpolate(method="time", limit=max_gap, limit_area="inside")
 39      hourly = hourly.fillna(hourly.shift(168)).ffill().bfill()
 40      return hourly
 41
 42
 43  def outlier_report(hourly: pd.DataFrame) -> dict:
 44      """IQR rule (outside Q1 - 1.5 IQR .. Q3 + 1.5 IQR) per variable.
 45
 46      Outliers are reported, not removed: high readings are real peak usage (heaters,
 47      ovens), exactly the hours the forecaster must learn. Distance-based models
 48      (KNN, SVM) instead get standardised inputs so large-valued columns do not dominate.
 49      """
 50      out = {}
 51      for c in ["Global_active_power", "Global_reactive_power", "Voltage", "Global_intensity"]:
 52          q1, q3 = hourly[c].quantile([0.25, 0.75])
 53          lo, hi = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
 54          n = int(((hourly[c] < lo) | (hourly[c] > hi)).sum())
 55          out[c] = {"lower": round(float(lo), 3), "upper": round(float(hi), 3), "count": n,
 56                    "pct": round(n / len(hourly) * 100, 2)}
 57      return out
 58
 59
 60  def main() -> None:
 61      params = load_params()
 62      raw = load_raw(resolve(params["data"]["raw_csv"]))
 63      missing_pct = float(raw["Global_active_power"].isna().mean() * 100)
 64      log.info("Loaded %d minute rows, %.2f%% missing", len(raw), missing_pct)
 65      hourly = clean_and_resample(raw, params["data"]["resample_rule"],
params["data"]["max_interp_gap_hours"])
 66      out = resolve(params["data"]["processed"])
 67      hourly.to_parquet(out)
 68      stats = {
 69          "raw_rows": int(len(raw)),
 70          "raw_missing_pct": round(missing_pct, 3),
 71          "hourly_rows": int(len(hourly)),
 72          "start": str(hourly.index.min()),
 73          "end": str(hourly.index.max()),
 74          "outliers": outlier_report(hourly),
 75      }
 76      (ROOT / "reports").mkdir(exist_ok=True)
 77      (ROOT / "reports" / "data_stats.json").write_text(json.dumps(stats, indent=2))
 78      log.info("Saved %d hourly rows to %s", len(hourly), out)
 79
 80
 81  if __name__ == "__main__":
 82      main()
```

src/features.py: feature engineering

```
  1  """Stage 3 - Feature engineering for next-hour consumption forecasting.
  2
```

```
Every feature at time t uses only information available up to hour t-1,
so the model forecasts consumption for hour t without leakage.
"""
import numpy as np
import pandas as pd

from src.utils import get_logger, load_params, resolve

log = get_logger("features")

# Public holidays in France (dataset location: Sceaux, near Paris), fixed-date ones.
FIXED_HOLIDAYS = {(1, 1), (5, 1), (5, 8), (7, 14), (8, 15), (11, 1), (11, 11), (12, 25)}


def calendar_features(idx: pd.DatetimeIndex) -> pd.DataFrame:
    f = pd.DataFrame(index=idx)
    f["hour"] = idx.hour
    f["dayofweek"] = idx.dayofweek
    f["month"] = idx.month
    f["is_weekend"] = (idx.dayofweek >= 5).astype(int)
    f["is_holiday"] = [int((d.month, d.day) in FIXED_HOLIDAYS) for d in idx]
    # Cyclical encodings so 23:00 and 00:00 are close in feature space.
    f["hour_sin"] = np.sin(2 * np.pi * idx.hour / 24)
    f["hour_cos"] = np.cos(2 * np.pi * idx.hour / 24)
    f["month_sin"] = np.sin(2 * np.pi * (idx.month - 1) / 12)
    f["month_cos"] = np.cos(2 * np.pi * (idx.month - 1) / 12)
    return f


def build_features(hourly: pd.DataFrame, target: str, lags: list[int], windows: list[int]) ->
pd.DataFrame:
    y = hourly[target]
    f = calendar_features(hourly.index)
    for lag in lags:
        f[f"lag_{lag}"] = y.shift(lag)
    past = y.shift(1)
    for w in windows:
        f[f"roll_mean_{w}"] = past.rolling(w).mean()
        f[f"roll_std_{w}"] = past.rolling(w).std()
    for c in ["Global_reactive_power", "Voltage", "Global_intensity",
              "Sub_metering_1", "Sub_metering_2", "Sub_metering_3"]:
        f[f"{c}_lag1"] = hourly[c].shift(1)
    f["target"] = y
    return f.dropna()


def feature_columns(params: dict) -> list[str]:
    """Deterministic feature list used by training, the API and the dashboard."""
    fp = params["features"]
    cols = ["hour", "dayofweek", "month", "is_weekend", "is_holiday",
            "hour_sin", "hour_cos", "month_sin", "month_cos"]
    cols += [f"lag_{l}" for l in fp["lags"]]
    for w in fp["rolling_windows"]:
        cols += [f"roll_mean_{w}", f"roll_std_{w}"]
    cols += [f"{c}_lag1" for c in ["Global_reactive_power", "Voltage", "Global_intensity",
                                   "Sub_metering_1", "Sub_metering_2", "Sub_metering_3"]]
    return cols


def main() -> None:
    params = load_params()
    hourly = pd.read_parquet(resolve(params["data"]["processed"]))
    feats = build_features(hourly, params["features"]["target"],
                           params["features"]["lags"], params["features"]["rolling_windows"])
    assert list(feats.columns[:-1]) == feature_columns(params)
    feats.to_parquet(resolve(params["data"]["features"]))
    log.info("Built %d rows x %d features", len(feats), feats.shape[1] - 1)


if __name__ == "__main__":
    main()
```

src/train.py: model training, tuning and registry

```
"""Stage 4 - Training: compare candidate models with MLflow tracking, tune them, register the best.
```

```
  2
  3  1. Every syllabus algorithm (linear regression, decision tree, KNN, SVM, random forest)
  4     plus ridge, gradient boosting and two naive baselines is trained with default settings.
  5     Train and validation errors are both recorded to expose over- and under-fitting.
  6  2. Models with a grid in params.yaml are tuned with GridSearchCV using time-series
  7     cross-validation on the training period (each fold only trains on the past).
  8  3. The best variant on the chronological validation window is refit on train+validation
  9     and scored once on the untouched test window.
 10  """
 11  import json
 12  import time
 13  from datetime import datetime, timezone
 14
 15  import joblib
 16  import mlflow
 17  import mlflow.sklearn
 18  import numpy as np
 19  import pandas as pd
 20  from mlflow.models import infer_signature
 21  from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestRegressor
 22  from sklearn.linear_model import LinearRegression, Ridge
 23  from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
 24  from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
 25  from sklearn.neighbors import KNeighborsRegressor
 26  from sklearn.pipeline import Pipeline, make_pipeline
 27  from sklearn.preprocessing import StandardScaler
 28  from sklearn.svm import SVR
 29  from sklearn.tree import DecisionTreeRegressor
 30
 31  from src.features import feature_columns
 32  from src.utils import ROOT, get_logger, load_params, resolve
 33
 34  log = get_logger("train")
 35  # Tree node storage of the models this pipeline itself trains (forests, single trees, boosting).
 36  TRUSTED_TYPES = ["sklearn.ensemble._hist_gradient_boosting.predictor.TreePredictor",
"sklearn.tree._tree.Tree"]
 37
 38
 39  def split(df: pd.DataFrame, params: dict):
 40      train_end = pd.Timestamp(params["split"]["train_end"])
 41      val_end = pd.Timestamp(params["split"]["val_end"])
 42      return df[df.index <= train_end], df[(df.index > train_end) & (df.index <= val_end)],
df[df.index > val_end]
 43
 44
 45  def metrics(y_true, y_pred) -> dict:
 46      y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
 47      return {
 48          "mae": float(mean_absolute_error(y_true, y_pred)),
 49          "mse": float(mean_squared_error(y_true, y_pred)),
 50          "rmse": float(np.sqrt(mean_squared_error(y_true, y_pred))),
 51          "r2": float(r2_score(y_true, y_pred)),
 52          "mape": float(np.mean(np.abs((y_true - y_pred) / np.clip(y_true, 0.1, None))) * 100),
 53      }
 54
 55
 56  class NaiveForecaster:
 57      """Baseline that copies one lag column (persistence or seasonal naive)."""
 58
 59      def __init__(self, column: str, columns: list[str]):
 60          self.idx = columns.index(column)
 61
 62      def fit(self, X, y):
 63          return self
 64
 65      def predict(self, X):
 66          return np.asarray(X)[:, self.idx]
 67
 68
 69  def build(name: str, params: dict, cols: list[str], overrides: dict | None = None):
 70      """Create a fresh, unfitted model. Distance- and margin-based models get feature scaling."""
 71      hp = {**params["models"].get(name, {}), **(overrides or {})}
 72      makers = {
 73          "persistence_baseline": lambda: NaiveForecaster("lag_1", cols),
 74          "seasonal_naive_24h": lambda: NaiveForecaster("lag_24", cols),
 75          "linear_regression": lambda: make_pipeline(StandardScaler(), LinearRegression(**hp)),
 76          "ridge_regression": lambda: make_pipeline(StandardScaler(), Ridge(**hp)),
 77          "decision_tree": lambda: DecisionTreeRegressor(**hp),
 78          "knn": lambda: make_pipeline(StandardScaler(), KNeighborsRegressor(**hp)),
```

```
        "svm": lambda: make_pipeline(StandardScaler(), SVR(**hp)),
        "random_forest": lambda: RandomForestRegressor(**hp),
        "hist_gradient_boosting": lambda: HistGradientBoostingRegressor(**hp),
    }
    return makers[name]()


CANDIDATES = ["persistence_baseline", "seasonal_naive_24h", "linear_regression", "ridge_regression",
              "decision_tree", "knn", "svm", "random_forest", "hist_gradient_boosting"]


def tune(name: str, params: dict, cols: list[str], X, y) -> GridSearchCV:
    model = build(name, params, cols)
    grid = params["tuning"]["grids"][name]
    # Inside a pipeline, parameters are addressed as <step>__<param>.
    prefix = f"{model.steps[-1][0]}__" if isinstance(model, Pipeline) else ""
    search = GridSearchCV(
        model, {prefix + k: v for k, v in grid.items()},
        cv=TimeSeriesSplit(n_splits=params["tuning"]["cv_splits"]),
        scoring=params["tuning"]["scoring"],
        n_jobs=1 if name == "random_forest" else -1,  # the forest already uses every core
        return_train_score=True,
    )
    search.fit(X, y)
    search.best_params_unprefixed = {k.removeprefix(prefix): v for k, v in
search.best_params_.items()}
    return search


def main() -> None:
    params = load_params()
    cols = feature_columns(params)
    df = pd.read_parquet(resolve(params["data"]["features"]))
    train, val, test = split(df, params)
    X_tr, y_tr, X_val, y_val = train[cols].values, train["target"].values, val[cols].values,
val["target"].values
    log.info("Train %d | Val %d | Test %d rows", len(train), len(val), len(test))

    mlflow.set_tracking_uri(params["mlflow"]["tracking_uri"])
    mlflow.set_experiment(params["mlflow"]["experiment"])

    # 1. Default models: train vs validation error.
    results = {}
    for name in CANDIDATES:
        model = build(name, params, cols)
        with mlflow.start_run(run_name=name):
            t0 = time.perf_counter()
            model.fit(X_tr, y_tr)
            fit_s = time.perf_counter() - t0
            m = metrics(y_val, model.predict(X_val))
            train_m = metrics(y_tr, model.predict(X_tr))
            mlflow.log_params({"model": name, "variant": "default", "n_features": len(cols),
**params["models"].get(name, {})})
            mlflow.log_metrics({**{f"val_{k}": v for k, v in m.items()}, "train_mae":
train_m["mae"],
                                "train_r2": train_m["r2"], "fit_seconds": fit_s})
            results[name] = {**m, "train_mae": train_m["mae"], "train_r2": train_m["r2"],
"fit_seconds": fit_s}
            log.info("%-24s train MAE=%.4f  val MAE=%.4f RMSE=%.4f R2=%.4f (%.1fs)",
                     name, train_m["mae"], m["mae"], m["rmse"], m["r2"], fit_s)

    # 2. Hyper-parameter tuning with time-series cross-validation.
    tuning = {}
    for name in params["tuning"]["grids"]:
        t0 = time.perf_counter()
        search = tune(name, params, cols, X_tr, y_tr)
        tuned_val = metrics(y_val, search.best_estimator_.predict(X_val))
        tuning[name] = {
            "best_params": search.best_params_unprefixed,
            "cv_mae": float(-search.best_score_),
            "candidates_tried": len(search.cv_results_["params"]),
            "default_val": {k: results[name][k] for k in ["mae", "rmse", "r2"]},
            "tuned_val": {k: tuned_val[k] for k in ["mae", "rmse", "r2"]},
            "search_seconds": time.perf_counter() - t0,
        }
        with mlflow.start_run(run_name=f"{name}_tuned"):
            mlflow.log_params({"model": name, "variant": "tuned", **search.best_params_unprefixed})
            mlflow.log_metrics({"cv_mae": tuning[name]["cv_mae"], **{f"val_{k}": v for k, v in
tuned_val.items()}})
```

```
 152          log.info("tuned %-18s %s  CV MAE=%.4f  val MAE %.4f -> %.4f", name,
search.best_params_unprefixed,
 153                   tuning[name]["cv_mae"], results[name]["mae"], tuned_val["mae"])
 154
 155      # 3. Pick the best variant (default or tuned) on validation RMSE among the syllabus algorithms;
 156      #    the best model overall is recorded for comparison only.
 157      variants = [(n, None, results[n]["rmse"]) for n in CANDIDATES if n not in
("persistence_baseline", "seasonal_naive_24h")]
 158      variants += [(n, t["best_params"], t["tuned_val"]["rmse"]) for n, t in tuning.items()]
 159      overall = min(variants, key=lambda v: v[2])
 160      allowed = params["selection"]["deployable"]
 161      best, best_params, _ = min((v for v in variants if v[0] in allowed), key=lambda v: v[2])
 162      log.info("Best model on validation: %s (%s); best overall incl. reference models: %s", best,
 163               "tuned " + str(best_params) if best_params else "default", overall[0])
 164
 165      full = pd.concat([train, val])
 166      final = build(best, params, cols, best_params)
 167      final.fit(full[cols].values, full["target"].values)
 168      test_pred = final.predict(test[cols].values)
 169      test_m = metrics(test["target"], test_pred)
 170      baseline_m = metrics(test["target"], test["lag_1"])
 171      log.info("TEST %s: MAE=%.4f RMSE=%.4f R2=%.4f | persistence MAE=%.4f",
 172               best, test_m["mae"], test_m["rmse"], test_m["r2"], baseline_m["mae"])
 173
 174      with mlflow.start_run(run_name=f"{best}_final") as run:
 175          mlflow.log_params({"model": best, "trained_on": "train+val", **(best_params or {})})
 176          mlflow.log_metrics({f"test_{k}": v for k, v in test_m.items()})
 177          mlflow.sklearn.log_model(
 178              final, name="model", signature=infer_signature(test[cols].iloc[:5], test_pred[:5]),
 179              registered_model_name=params["mlflow"]["registered_model"],
 180              # We produced this model ourselves in this run, so its tree type is trusted.
 181              skops_trusted_types=TRUSTED_TYPES,
 182          )
 183          run_id = run.info.run_id
 184
 185      models_dir = ROOT / "models"
 186      models_dir.mkdir(exist_ok=True)
 187      joblib.dump(final, models_dir / "model.joblib", compress=3)  # a 200-tree forest is ~65 MB
uncompressed
 188      metadata = {
 189          "model_name": best,
 190          "hyperparameters": {**params["models"].get(best, {}), **(best_params or {})},
 191          "tuned": best_params is not None,
 192          "selection": {"deployable": allowed, "best_overall": overall[0], "best_overall_val_rmse":
overall[2]},
 193          "trained_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
 194          "mlflow_run_id": run_id,
 195          "features": cols,
 196          "train_period": [str(full.index.min()), str(full.index.max())],
 197          "test_period": [str(test.index.min()), str(test.index.max())],
 198          "test_metrics": test_m,
 199          "baseline_test_metrics": baseline_m,
 200          # Residual spread on the test window, used for simple prediction intervals in the API.
 201          "residual_std": float(np.std(test["target"].values - test_pred)),
 202      }
 203      (models_dir / "metadata.json").write_text(json.dumps(metadata, indent=2))
 204      (ROOT / "reports" / "metrics.json").write_text(json.dumps(
 205          {"validation": results, "tuning": tuning, "test": {best: test_m, "persistence_baseline":
baseline_m}}, indent=2))
 206      pd.DataFrame({"actual": test["target"], "predicted": test_pred}).to_parquet(
 207          ROOT / "reports" / "test_predictions.parquet")
 208      log.info("Saved model to %s", models_dir / "model.joblib")
 209
 210
 211  if __name__ == "__main__":
 212      main()
```

src/classify.py: peak-hour classifier

```
   1  """Stage 5 - Peak-hour classification: will the next hour be a high-consumption hour?
   2
   3  Turns the forecasting problem into a binary alert (useful for demand response):
   4  label = 1 when consumption exceeds the 75th percentile of the training period.
   5  A decision tree is trained with default settings and tuned with GridSearchCV
   6  (time-series folds, F1 scoring); a logistic regression is the linear reference.
```

```
  7  Evaluated with accuracy, precision, recall, F1, ROC-AUC and the confusion matrix.
  8  """
  9  import json
 10
 11  import joblib
 12  import mlflow
 13  import numpy as np
 14  import pandas as pd
 15  from sklearn.linear_model import LogisticRegression
 16  from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score, precision_recall_curve,
 17                               precision_score, recall_score, roc_auc_score, roc_curve)
 18  from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
 19  from sklearn.pipeline import make_pipeline
 20  from sklearn.preprocessing import StandardScaler
 21  from sklearn.tree import DecisionTreeClassifier
 22
 23  from src.features import feature_columns
 24  from src.train import split
 25  from src.utils import ROOT, get_logger, load_params, resolve
 26
 27  log = get_logger("classify")
 28
 29
 30  def scores(y_true, proba, threshold: float = 0.5) -> dict:
 31      pred = (proba >= threshold).astype(int)
 32      return {
 33          "accuracy": float(accuracy_score(y_true, pred)),
 34          "precision": float(precision_score(y_true, pred, zero_division=0)),
 35          "recall": float(recall_score(y_true, pred, zero_division=0)),
 36          "f1": float(f1_score(y_true, pred, zero_division=0)),
 37          "roc_auc": float(roc_auc_score(y_true, proba)),
 38          "confusion_matrix": confusion_matrix(y_true, pred).tolist(),  # [[TN, FP], [FN, TP]]
 39      }
 40
 41
 42  def main() -> None:
 43      params = load_params()
 44      cp = params["classification"]
 45      cols = feature_columns(params)
 46      df = pd.read_parquet(resolve(params["data"]["features"]))
 47      train, val, test = split(df, params)
 48      threshold_kw = float(train["target"].quantile(cp["threshold_quantile"]))
 49      label = lambda d: (d["target"] > threshold_kw).astype(int).values
 50      X_tr, y_tr, X_val, y_val = train[cols].values, label(train), val[cols].values, label(val)
 51      log.info("Peak threshold %.3f kW | positive rate train %.1f%% val %.1f%%",
 52               threshold_kw, y_tr.mean() * 100, y_val.mean() * 100)
 53
 54      mlflow.set_tracking_uri(params["mlflow"]["tracking_uri"])
 55      mlflow.set_experiment(params["mlflow"]["experiment"] + "-peak-classifier")
 56
 57      search = GridSearchCV(DecisionTreeClassifier(**cp["decision_tree"]), cp["grid"],
 58                            cv=TimeSeriesSplit(n_splits=params["tuning"]["cv_splits"]), scoring="f1",
n_jobs=-1)
 59      search.fit(X_tr, y_tr)
 60      models = {
 61          "logistic_regression": make_pipeline(StandardScaler(),
LogisticRegression(max_iter=2000)).fit(X_tr, y_tr),
 62          "decision_tree_default": DecisionTreeClassifier(**cp["decision_tree"]).fit(X_tr, y_tr),
 63          "decision_tree_tuned": search.best_estimator_,
 64      }
 65      val_scores = {}
 66      for name, model in models.items():
 67          s = scores(y_val, model.predict_proba(X_val)[:, 1])
 68          s["train_f1"] = float(f1_score(y_tr, model.predict(X_tr)))
 69          val_scores[name] = s
 70          with mlflow.start_run(run_name=name):
 71              mlflow.log_metrics({f"val_{k}": v for k, v in s.items() if k != "confusion_matrix"})
 72          log.info("%-22s val acc=%.3f prec=%.3f rec=%.3f F1=%.3f AUC=%.3f (train F1 %.3f)", name,
s["accuracy"],
 73                   s["precision"], s["recall"], s["f1"], s["roc_auc"], s["train_f1"])
 74
 75      best = max(val_scores, key=lambda n: val_scores[n]["f1"])
 76      full = pd.concat([train, val])
 77      final = (DecisionTreeClassifier(**cp["decision_tree"], **search.best_params_) if best ==
"decision_tree_tuned"
 78               else models[best].__class__(**models[best].get_params()))
 79      final.fit(full[cols].values, label(full))
 80      proba = final.predict_proba(test[cols].values)[:, 1]
 81      y_test = label(test)
```

```
    test_scores = scores(y_test, proba)
    fpr, tpr, _ = roc_curve(y_test, proba)
    prec, rec, _ = precision_recall_curve(y_test, proba)
    log.info("TEST %s: acc=%.3f prec=%.3f rec=%.3f F1=%.3f AUC=%.3f", best, test_scores["accuracy"],
             test_scores["precision"], test_scores["recall"], test_scores["f1"],
test_scores["roc_auc"])
    with mlflow.start_run(run_name=f"{best}_final"):
        mlflow.log_params({"model": best, "threshold_kw": threshold_kw, **search.best_params_})
        mlflow.log_metrics({f"test_{k}": v for k, v in test_scores.items() if k !=
"confusion_matrix"})

    # Reference: flag a peak whenever the regression forecast exceeds the threshold.
    reg = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet").reindex(test.index)
    rule = (reg["predicted"].values > threshold_kw).astype(float)
    rule_scores = {k: v for k, v in scores(y_test, rule).items() if k != "roc_auc"}
    log.info("TEST forecast>threshold rule: acc=%.3f prec=%.3f rec=%.3f F1=%.3f",
rule_scores["accuracy"],
             rule_scores["precision"], rule_scores["recall"], rule_scores["f1"])

    joblib.dump(final, ROOT / "models" / "peak_classifier.joblib")
    # Small sidecar the API needs at run time (reports/ is not shipped to the server).
    (ROOT / "models" / "peak_classifier.json").write_text(json.dumps(
        {"model": best, "threshold_kw": threshold_kw, "hyperparameters": search.best_params_},
indent=2))
    (ROOT / "reports" / "classification.json").write_text(json.dumps({
        "threshold_kw": threshold_kw,
        "threshold_quantile": cp["threshold_quantile"],
        "best_params": {k: v for k, v in search.best_params_.items()},
        "cv_f1": float(search.best_score_),
        "validation": val_scores,
        "selected": best,
        "test": test_scores,
        "forecast_threshold_rule_test": rule_scores,
        "positive_rate_test": float(y_test.mean()),
        # Down-sampled curves for plotting.
        "roc": {"fpr": fpr[:: max(1, len(fpr) // 200)].round(4).tolist(), "tpr": tpr[:: max(1,
len(tpr) // 200)].round(4).tolist()},
        "pr": {"precision": prec[:: max(1, len(prec) // 200)].round(4).tolist(), "recall": rec[::
max(1, len(rec) // 200)].round(4).tolist()},
    }, indent=2))
    log.info("Saved peak classifier (%s)", best)


if __name__ == "__main__":
    main()
```

src/monitor.py: drift monitoring

```
"""Stage 6 - Monitoring: data drift (PSI) and performance checks that decide on retraining."""
import json

import numpy as np
import pandas as pd

from src.train import metrics, split
from src.utils import ROOT, get_logger, load_params, resolve

log = get_logger("monitor")

MONITORED = ["lag_1", "roll_mean_24", "Voltage_lag1", "Global_intensity_lag1",
             "Sub_metering_3_lag1", "hour", "target"]


def psi(reference: np.ndarray, current: np.ndarray, bins: int = 10) -> float:
    """Population Stability Index: <0.1 stable, 0.1-0.2 moderate, >0.2 significant drift."""
    edges = np.unique(np.quantile(reference, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    ref = np.histogram(reference, edges)[0] / len(reference)
    cur = np.histogram(current, edges)[0] / len(current)
    ref, cur = np.clip(ref, 1e-6, None), np.clip(cur, 1e-6, None)
    return float(np.sum((cur - ref) * np.log(cur / ref)))


def main() -> None:
    params = load_params()
```

```
    df = pd.read_parquet(resolve(params["data"]["features"]))
    train, _, test = split(df, params)
    drift = {c: round(psi(train[c].values, test[c].values), 4) for c in MONITORED}
    drifted = [c for c, v in drift.items() if v > params["monitoring"]["psi_threshold"]]

    meta = json.loads((ROOT / "models" / "metadata.json").read_text())
    preds = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet")
    # Compare recent production-like window (last 30 days) with the overall test MAE.
    recent = preds.iloc[-24 * 30:]
    recent_mae = metrics(recent["actual"], recent["predicted"])["mae"]
    degradation = (recent_mae / meta["test_metrics"]["mae"] - 1) * 100
    retrain = bool(drifted) or degradation > params["monitoring"]["mae_degradation_pct"]

    report = {
        "psi": drift,
        "drifted_features": drifted,
        "recent_30d_mae": round(recent_mae, 4),
        "reference_test_mae": round(meta["test_metrics"]["mae"], 4),
        "mae_degradation_pct": round(degradation, 2),
        "retraining_recommended": retrain,
    }
    (ROOT / "reports" / "monitoring.json").write_text(json.dumps(report, indent=2))
    log.info("PSI: %s", drift)
    log.info("Recent MAE %.4f (%.1f%% vs reference) -> retrain=%s", recent_mae, degradation,
retrain)


if __name__ == "__main__":
    main()
```

src/evaluate.py: evaluation figures

```
"""Stage 7 - Evaluation: EDA, model diagnostics, classification and bias-slice figures for the
report and website."""
import json

import joblib
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.inspection import permutation_importance
from sklearn.metrics import mean_absolute_error
from sklearn.tree import DecisionTreeRegressor

from src.features import feature_columns
from src.train import split
from src.utils import ROOT, get_logger, load_params, resolve

log = get_logger("evaluate")
FIG = ROOT / "reports" / "figures"
plt.rcParams.update({"figure.dpi": 130, "axes.spines.top": False, "axes.spines.right": False,
                     "font.family": "serif", "font.size": 10})
BLUE, ORANGE, GREY = "#1f5fa8", "#e07b24", "#888888"
LABELS = {
    "persistence_baseline": "Persistence baseline", "seasonal_naive_24h": "Seasonal naive (24 h)",
    "linear_regression": "Linear regression", "ridge_regression": "Ridge regression",
    "decision_tree": "Decision tree", "knn": "K-nearest neighbours", "svm": "SVM (SVR)",
    "random_forest": "Random forest", "hist_gradient_boosting": "Gradient boosting",
}


def save(name: str) -> None:
    plt.tight_layout()
    plt.savefig(FIG / name, bbox_inches="tight")
    plt.close()


def eda(hourly: pd.DataFrame) -> None:
    y = hourly["Global_active_power"]
    daily = y.resample("D").mean()
    fig, ax = plt.subplots(figsize=(9, 3.2))
```

```
    ax.plot(daily.index, daily, color=BLUE, lw=0.7, label="Daily mean")
    ax.plot(daily.rolling(30).mean(), color=ORANGE, lw=1.8, label="30-day rolling mean")
    ax.set(title="Household active power, Dec 2006 - Nov 2010", ylabel="kW")
    ax.legend(frameon=False)
    save("fig_eda_timeseries.png")

    pivot = y.groupby([y.index.dayofweek, y.index.hour]).mean().unstack()
    pivot.index = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
    fig, ax = plt.subplots(figsize=(9, 3.2))
    sns.heatmap(pivot, cmap="YlOrRd", ax=ax, cbar_kws={"label": "kW"})
    ax.set(title="Average consumption by weekday and hour", xlabel="Hour of day", ylabel="")
    save("fig_eda_heatmap.png")

    fig, axes = plt.subplots(1, 2, figsize=(9, 3.2))
    y.groupby(y.index.month).mean().plot(kind="bar", color=BLUE, ax=axes[0])
    axes[0].set(title="Average by month", xlabel="Month", ylabel="kW")
    sub = hourly[["Sub_metering_1", "Sub_metering_2", "Sub_metering_3", "Sub_metering_other"]].sum()
    sub.index = ["Kitchen", "Laundry", "Water heater / AC", "Other"]
    axes[1].pie(sub, labels=sub.index, autopct="%1.0f%%", colors=[BLUE, ORANGE, "#5aa469", GREY])
    axes[1].set(title="Energy share by sub-meter")
    save("fig_eda_month_submeter.png")

    corr = hourly.corr()
    fig, ax = plt.subplots(figsize=(6.5, 5))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax, annot_kws={"size":
7})
    ax.set(title="Correlation between measured variables")
    save("fig_eda_correlation.png")


def model_figures(params: dict) -> None:
    m = json.loads((ROOT / "reports" / "metrics.json").read_text())["validation"]
    names = list(m)
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.2))
    colors = [GREY if "baseline" in n or "naive" in n else BLUE for n in names]
    best = json.loads((ROOT / "models" / "metadata.json").read_text())["model_name"]  # deployed
model
    colors[names.index(best)] = ORANGE
    labels = [LABELS.get(n, n) for n in names]
    axes[0].barh(labels, [m[n]["mae"] for n in names], color=colors)
    axes[0].set(title="Validation MAE (lower is better)", xlabel="kW")
    axes[0].invert_yaxis()
    axes[1].barh(labels, [m[n]["r2"] for n in names], color=colors)
    axes[1].set(title="Validation R² (higher is better)")
    axes[1].axvline(0, color="black", lw=0.6)
    axes[1].invert_yaxis()
    axes[1].tick_params(labelleft=False)
    save("fig_model_comparison.png")

    # Over/under-fitting: training vs validation error of every default model.
    ml = [n for n in names if "baseline" not in n and "naive" not in n]
    fig, ax = plt.subplots(figsize=(9, 3.4))
    x = np.arange(len(ml))
    ax.bar(x - 0.2, [m[n]["train_mae"] for n in ml], 0.4, color=GREY, label="Training MAE")
    ax.bar(x + 0.2, [m[n]["mae"] for n in ml], 0.4, color=BLUE, label="Validation MAE")
    ax.set_xticks(x, [LABELS[n].replace(" ", "\n", 1) for n in ml], fontsize=8)
    ax.set(title="Training vs validation error (default hyper-parameters)", ylabel="MAE (kW)")
    ax.legend(frameon=False)
    save("fig_train_vs_val.png")

    # Default vs GridSearchCV-tuned validation error.
    t = json.loads((ROOT / "reports" / "metrics.json").read_text())["tuning"]
    tn = list(t)
    fig, ax = plt.subplots(figsize=(9, 3.2))
    x = np.arange(len(tn))
    ax.bar(x - 0.2, [t[n]["default_val"]["mae"] for n in tn], 0.4, color=GREY, label="Default")
    ax.bar(x + 0.2, [t[n]["tuned_val"]["mae"] for n in tn], 0.4, color=ORANGE, label="Tuned
(GridSearchCV)")
    ax.set_xticks(x, [LABELS[n] for n in tn], fontsize=8)
    ax.set(title="Effect of hyper-parameter tuning on validation MAE", ylabel="MAE (kW)")
    ax.legend(frameon=False)
    save("fig_tuning.png")

    preds = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet")
    week = preds.iloc[-24 * 14:]
    fig, ax = plt.subplots(figsize=(9, 3.2))
    ax.plot(week.index, week["actual"], color=GREY, lw=1.2, label="Actual")
    ax.plot(week.index, week["predicted"], color=ORANGE, lw=1.2, label="Predicted")
    ax.set(title="Next-hour forecast vs actual - last 14 days of test set", ylabel="kW")
```

```
    ax.legend(frameon=False)
    save("fig_actual_vs_pred.png")

    resid = preds["actual"] - preds["predicted"]
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.2))
    axes[0].scatter(preds["actual"], preds["predicted"], s=3, alpha=0.3, color=BLUE)
    lim = [0, preds.max().max()]
    axes[0].plot(lim, lim, color="black", lw=0.8, ls="--")
    axes[0].set(title="Predicted vs actual (test)", xlabel="Actual kW", ylabel="Predicted kW")
    axes[1].hist(resid, bins=60, color=BLUE)
    axes[1].set(title="Residual distribution", xlabel="Actual - predicted (kW)")
    save("fig_residuals.png")

    err = preds.assign(err=resid.abs()).groupby(preds.index.hour)["err"].mean()
    fig, ax = plt.subplots(figsize=(9, 3))
    ax.bar(err.index, err.values, color=BLUE)
    ax.set(title="Mean absolute error by hour of day (test)", xlabel="Hour", ylabel="MAE (kW)")
    save("fig_error_by_hour.png")


def importance(params: dict) -> None:
    meta = json.loads((ROOT / "models" / "metadata.json").read_text())
    model = joblib.load(ROOT / "models" / "model.joblib")
    df = pd.read_parquet(resolve(params["data"]["features"]))
    _, _, test = split(df, params)
    r = permutation_importance(model, test[meta["features"]].values, test["target"].values,
                               n_repeats=5, random_state=0, scoring="neg_mean_absolute_error")
    imp = pd.Series(r.importances_mean, index=meta["features"]).sort_values().tail(15)
    imp.to_frame("importance").to_csv(ROOT / "reports" / "feature_importance.csv")
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.barh(imp.index, imp.values, color=BLUE)
    ax.set(title="Top 15 features (permutation importance)", xlabel="Increase in MAE when shuffled
(kW)")
    save("fig_feature_importance.png")


def monitoring_fig() -> None:
    mon = json.loads((ROOT / "reports" / "monitoring.json").read_text())
    s = pd.Series(mon["psi"]).sort_values()
    fig, ax = plt.subplots(figsize=(7, 3.2))
    ax.barh(s.index, s.values, color=[ORANGE if v > 0.2 else BLUE for v in s.values])
    ax.axvline(0.1, color=GREY, ls="--", lw=0.8)
    ax.axvline(0.2, color="red", ls="--", lw=0.8)
    ax.set(title="Data drift: PSI train vs test (dashed: 0.1 / 0.2 thresholds)", xlabel="PSI")
    save("fig_drift_psi.png")


def outlier_fig(hourly: pd.DataFrame) -> None:
    cols = ["Global_active_power", "Global_reactive_power", "Global_intensity"]
    fig, axes = plt.subplots(1, 3, figsize=(9, 3))
    for ax, c in zip(axes, cols):
        ax.boxplot(hourly[c], vert=True, widths=0.5, flierprops={"markersize": 2, "alpha": 0.3})
        ax.set(title=c.replace("_", " "), xticks=[])
    fig.suptitle("Outliers by the IQR rule (kept: they are genuine peak-usage hours)", fontsize=10)
    save("fig_outliers.png")


def validation_curve_fig(params: dict) -> None:
    """Decision-tree depth vs error: under-fitting on the left, over-fitting on the right."""
    cols = feature_columns(params)
    train, val, _ = split(pd.read_parquet(resolve(params["data"]["features"])), params)
    depths = [1, 2, 3, 4, 6, 8, 10, 12, 15, 20, 25]
    tr, va = [], []
    for d in depths:
        m = DecisionTreeRegressor(max_depth=d, random_state=42).fit(train[cols].values,
train["target"].values)
        tr.append(mean_absolute_error(train["target"], m.predict(train[cols].values)))
        va.append(mean_absolute_error(val["target"], m.predict(val[cols].values)))
    best = depths[int(np.argmin(va))]
    fig, ax = plt.subplots(figsize=(9, 3.2))
    ax.plot(depths, tr, "o-", color=GREY, label="Training MAE")
    ax.plot(depths, va, "o-", color=BLUE, label="Validation MAE")
    ax.axvline(best, color=ORANGE, ls="--", lw=1)
    ax.text(best + 0.3, max(va) * 0.95, f"best depth = {best}", color=ORANGE)
    ax.text(1.2, max(va) * 0.6, "under-fitting", color=GREY, style="italic")
    ax.text(18, max(va) * 0.6, "over-fitting", color=GREY, style="italic")
    ax.set(title="Validation curve: decision tree depth", xlabel="max_depth", ylabel="MAE (kW)")
    ax.legend(frameon=False)
    save("fig_validation_curve.png")
```

```
196
197
198  def classification_figs() -> None:
199      c = json.loads((ROOT / "reports" / "classification.json").read_text())
200      t = c["test"]
201      fig, axes = plt.subplots(1, 3, figsize=(10, 3.3))
202      cm = np.array(t["confusion_matrix"])
203      axes[0].imshow(cm, cmap="Blues")
204      for (i, j), v in np.ndenumerate(cm):
205          axes[0].text(j, i, f"{v:,}", ha="center", va="center", color="white" if v > cm.max() / 2
else "black")
206      axes[0].set(title="Confusion matrix (test)", xticks=[0, 1], yticks=[0, 1],
207                  xticklabels=["Normal", "Peak"], yticklabels=["Normal", "Peak"], xlabel="Predicted",
ylabel="Actual")
208      axes[1].plot(c["roc"]["fpr"], c["roc"]["tpr"], color=ORANGE, lw=1.8)
209      axes[1].plot([0, 1], [0, 1], color=GREY, ls="--", lw=0.8)
210      axes[1].set(title=f"ROC curve (AUC = {t['roc_auc']:.3f})", xlabel="False positive rate",
ylabel="True positive rate")
211      axes[2].plot(c["pr"]["recall"], c["pr"]["precision"], color=BLUE, lw=1.8)
212      axes[2].axhline(c["positive_rate_test"], color=GREY, ls="--", lw=0.8)
213      axes[2].set(title="Precision-recall curve", xlabel="Recall", ylabel="Precision")
214      save("fig_classification.png")
215
216      v = c["validation"]
217      names = list(v)
218      keys = ["accuracy", "precision", "recall", "f1", "roc_auc"]
219      fig, ax = plt.subplots(figsize=(9, 3.2))
220      x = np.arange(len(keys))
221      for i, (n, col) in enumerate(zip(names, [GREY, BLUE, ORANGE])):
222          ax.bar(x + (i - 1) * 0.27, [v[n][k] for k in keys], 0.27, color=col, label=n.replace("_", "
"))
223      ax.set_xticks(x, ["Accuracy", "Precision", "Recall", "F1", "ROC-AUC"])
224      ax.set(title="Peak-hour classifiers on the validation set", ylim=(0, 1))
225      ax.legend(frameon=False, fontsize=8, loc="lower right")
226      save("fig_classifier_comparison.png")
227
228
229  def fairness_slices() -> None:
230      """Bias check: is the forecast equally good for every kind of hour and season?"""
231      p = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet")
232      p["err"] = (p["actual"] - p["predicted"]).abs()
233      idx = p.index
234      slices = {
235          "Day type": np.where(idx.dayofweek >= 5, "Weekend", "Weekday"),
236          "Time of day": pd.cut(idx.hour, [-1, 5, 11, 17, 23], labels=["Night 0-5", "Morning 6-11",
237                                                                       "Afternoon 12-17", "Evening 18-
23"]),
238          "Season": pd.Categorical(np.where(idx.month.isin([6, 7, 8]), "Summer (Jun-Aug)", "Autumn
(Sep-Nov)"),
239                                   categories=["Summer (Jun-Aug)", "Autumn (Sep-Nov)"]),
240          "Usage level": pd.qcut(p["actual"], 3, labels=["Low use", "Medium use", "High use"]),
241      }
242      out = {}
243      for name, groups in slices.items():
244          g = p.groupby(pd.Series(groups, index=p.index), observed=True)["err"].agg(["mean", "count"])
245          out[name] = {str(k): {"mae": round(float(r["mean"]), 4), "hours": int(r["count"])} for k, r
in g.iterrows()}
246      overall = float(p["err"].mean())
247      (ROOT / "reports" / "fairness.json").write_text(json.dumps({"overall_mae": overall, "slices":
out}, indent=2))
248
249      fig, axes = plt.subplots(1, 4, figsize=(11, 3.2), sharey=True)
250      for ax, (name, groups) in zip(axes, out.items()):
251          ax.bar(list(groups), [v["mae"] for v in groups.values()], color=BLUE)
252          ax.axhline(overall, color=ORANGE, ls="--", lw=1)
253          ax.set_title(name)
254          ax.tick_params(axis="x", labelrotation=30, labelsize=7.5)
255      axes[0].set_ylabel("MAE (kW)")
256      fig.suptitle("Error across sub-groups of the test period (dashed = overall MAE)", fontsize=10)
257      save("fig_fairness.png")
258
259
260  def main() -> None:
261      params = load_params()
262      FIG.mkdir(parents=True, exist_ok=True)
263      hourly = pd.read_parquet(resolve(params["data"]["processed"]))
264      eda(hourly)
265      outlier_fig(hourly)
266      model_figures(params)
```

```
    validation_curve_fig(params)
    importance(params)
    classification_figs()
    fairness_slices()
    if (ROOT / "reports" / "monitoring.json").exists():
        monitoring_fig()
    log.info("Figures written to %s", FIG)


if __name__ == "__main__":
    main()
```

src/analysis.py: extended analysis

```
"""Stage 8 - Extended analysis for the report: every model on the test set, extra metrics, curves.

Produces reports/analysis.json and the figures in reports/analysis/:
  * test-set scores of all nine regressors (default and tuned), with accuracy-style metrics
  * per-model forecast, scatter and residual plots
  * error by forecast horizon (1-24 h ahead) for the deployed recursive forecaster
  * learning curves, cross-validation fold scores and hyper-parameter response curves
  * confusion matrices, ROC/PR overlays, classification report and threshold sweep
  * additional EDA (missing data, distributions, autocorrelation, profiles) and monthly drift
"""
import json
import time

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import (classification_report, confusion_matrix, explained_variance_score,
max_error,
                             median_absolute_error, precision_recall_curve, roc_auc_score,
roc_curve)
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit, cross_validate, learning_curve
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

from src.features import feature_columns
from src.monitor import MONITORED, psi
from src.predict import MIN_HISTORY_HOURS, forecast
from src.train import CANDIDATES, build, metrics, split
from src.utils import ROOT, get_logger, load_params, resolve

log = get_logger("analysis")
OUT = ROOT / "reports" / "analysis"
plt.rcParams.update({"figure.dpi": 130, "axes.spines.top": False, "axes.spines.right": False,
                     "font.family": "serif", "font.size": 9})
BLUE, ORANGE, GREY, GREEN, RED = "#1f5fa8", "#e07b24", "#888888", "#2f8f5b", "#b42318"
LABELS = {
    "persistence_baseline": "Persistence baseline", "seasonal_naive_24h": "Seasonal naive (24 h)",
    "linear_regression": "Linear regression", "ridge_regression": "Ridge regression",
    "decision_tree": "Decision tree", "knn": "K-nearest neighbours", "svm": "SVM (SVR)",
    "random_forest": "Random forest", "hist_gradient_boosting": "Gradient boosting",
}


def save(name: str) -> None:
    plt.tight_layout()
    plt.savefig(OUT / name, bbox_inches="tight")
    plt.close()


def extended_metrics(y, p) -> dict:
    """Standard regression metrics plus accuracy-style measures that are easy to explain."""
    y, p = np.asarray(y), np.asarray(p)
    err = np.abs(y - p)
    return {
        **metrics(y, p),
```

```
  60          "median_ae": float(median_absolute_error(y, p)),
  61          "max_error": float(max_error(y, p)),
  62          "explained_variance": float(explained_variance_score(y, p)),
  63          # 1 - WAPE: share of total consumption predicted correctly (common in energy forecasting).
  64          "accuracy_1_minus_wape": float(1 - err.sum() / np.abs(y).sum()),
  65          "within_0_25kw_pct": float((err <= 0.25).mean() * 100),
  66          "within_0_5kw_pct": float((err <= 0.5).mean() * 100),
  67      }
  68
  69
  70  # ---------------------------------------------------------------- regression on the test set
  71  def all_models_on_test(params, cols, train, val, test, tuning):
  72      full = pd.concat([train, val])
  73      X, y, Xt, yt = full[cols].values, full["target"].values, test[cols].values,
test["target"].values
  74      results, preds = {}, {}
  75      variants = [(n, None) for n in CANDIDATES] + [(n, t["best_params"]) for n, t in tuning.items()]
  76      for name, hp in variants:
  77          key = name + ("_tuned" if hp else "")
  78          t0 = time.perf_counter()
  79          model = build(name, params, cols, hp).fit(X, y)
  80          fit_s = time.perf_counter() - t0
  81          t0 = time.perf_counter()
  82          p = model.predict(Xt)
  83          pred_ms = (time.perf_counter() - t0) / len(Xt) * 1000
  84          results[key] = {**extended_metrics(yt, p), "fit_seconds": fit_s, "predict_ms_per_row":
pred_ms}
  85          if not hp:
  86              preds[name] = p
  87          log.info("test %-30s MAE=%.4f R2=%.4f acc(1-WAPE)=%.3f", key, results[key]["mae"],
results[key]["r2"],
  88                   results[key]["accuracy_1_minus_wape"])
  89      return results, preds
  90
  91
  92  def per_model_figures(test, preds):
  93      names = list(preds)
  94      week = slice(-24 * 7, None)
  95      fig, axes = plt.subplots(3, 3, figsize=(10, 7.5), sharey=True)
  96      for ax, n in zip(axes.flat, names):
  97          ax.plot(test.index[week], test["target"].values[week], color=GREY, lw=0.9, label="Actual")
  98          ax.plot(test.index[week], preds[n][week], color=ORANGE, lw=0.9, label="Predicted")
  99          ax.set_title(LABELS[n], fontsize=9)
 100          ax.tick_params(axis="x", labelsize=6, rotation=30)
 101      axes[0, 0].legend(frameon=False, fontsize=7)
 102      fig.suptitle("Next-hour forecasts of every model, last 7 days of the test set", fontsize=10)
 103      save("per_model_forecasts.png")
 104
 105      fig, axes = plt.subplots(3, 3, figsize=(10, 9))
 106      lim = [0, float(test["target"].max())]
 107      for ax, n in zip(axes.flat, names):
 108          ax.scatter(test["target"], preds[n], s=2, alpha=0.25, color=BLUE)
 109          ax.plot(lim, lim, color="black", lw=0.7, ls="--")
 110          ax.set(title=LABELS[n], xlabel="Actual kW", ylabel="Predicted kW")
 111      fig.suptitle("Predicted vs actual, test set (dashed line = perfect prediction)", fontsize=10)
 112      save("per_model_scatter.png")
 113
 114      fig, axes = plt.subplots(3, 3, figsize=(10, 7.5), sharex=True)
 115      for ax, n in zip(axes.flat, names):
 116          r = test["target"].values - preds[n]
 117          ax.hist(r, bins=60, range=(-3, 3), color=BLUE)
 118          ax.axvline(0, color="black", lw=0.7)
 119          ax.set_title(f"{LABELS[n]}  (mean {r.mean():+.3f})", fontsize=9)
 120      fig.suptitle("Residual distribution of every model, test set (actual - predicted, kW)",
fontsize=10)
 121      save("per_model_residuals.png")
 122
 123
 124  def test_metric_bars(results):
 125      names = [n for n in CANDIDATES]
 126      fig, axes = plt.subplots(2, 2, figsize=(10, 7))
 127      for ax, (k, title) in zip(axes.flat, [("mae", "MAE (kW)"), ("rmse", "RMSE (kW)"), ("r2", "R²"),
 128                                            ("accuracy_1_minus_wape", "Accuracy (1 - WAPE)")]):
 129          vals = [results[n][k] for n in names]
 130          cols = [GREY if "baseline" in n or "naive" in n else BLUE for n in names]
 131          cols[names.index(json.loads((ROOT / "models" / "metadata.json").read_text())["model_name"])]
= ORANGE
 132          ax.barh([LABELS[n] for n in names], vals, color=cols)
 133          ax.invert_yaxis()
```

```
        ax.set_title(f"Test {title}")
        for i, v in enumerate(vals):
            ax.text(v, i, f" {v:.3f}", va="center", fontsize=7)
    save("test_metrics_all_models.png")


# ---------------------------------------------------------------- forecasting horizon
def horizon_curve(params):
    hourly = pd.read_csv(ROOT / params["data"]["serving"], index_col="datetime", parse_dates=True)
    rng = np.random.default_rng(7)
    lo, hi = MIN_HISTORY_HOURS * 2, len(hourly) - 25
    origins = hourly.index[np.sort(rng.choice(np.arange(lo, hi), 60, replace=False))]
    err = np.zeros((len(origins), 24))
    naive = np.zeros((len(origins), 24))
    for i, o in enumerate(origins):
        hist = hourly[hourly.index < o].iloc[-MIN_HISTORY_HOURS * 2:]
        fc = forecast(hist, 24)
        actual = hourly["Global_active_power"].reindex(fc["timestamp"]).values
        err[i] = np.abs(actual - fc["predicted_kw"].values)
        naive[i] = np.abs(actual - hourly["Global_active_power"].reindex(fc["timestamp"] -
pd.Timedelta(hours=24)).values)
    mae, mae_naive = err.mean(axis=0), naive.mean(axis=0)
    fig, ax = plt.subplots(figsize=(9, 3.4))
    h = np.arange(1, 25)
    ax.plot(h, mae, "o-", color=ORANGE, label="VoltCast recursive forecast")
    ax.plot(h, mae_naive, "s--", color=GREY, label="Same hour yesterday (seasonal naive)")
    ax.set(title=f"Forecast error by horizon ({len(origins)} random origins in the test period)",
           xlabel="Hours ahead", ylabel="MAE (kW)", xticks=h)
    ax.legend(frameon=False)
    save("horizon_curve.png")
    return {"origins": len(origins), "mae_by_hour": mae.round(4).tolist(), "naive_mae_by_hour":
mae_naive.round(4).tolist()}


# ---------------------------------------------------------------- learning curves and CV
def learning_curves(params, cols, train):
    X, y = train[cols].values, train["target"].values
    cv = TimeSeriesSplit(n_splits=5)
    picks = [("linear_regression", None), ("decision_tree", None), ("random_forest",
{"n_estimators": 100}),
             ("hist_gradient_boosting", None)]
    fig, axes = plt.subplots(1, 4, figsize=(11, 3.2), sharey=True)
    out = {}
    for ax, (n, hp) in zip(axes, picks):
        sizes, tr, va = learning_curve(build(n, params, cols, hp), X, y, cv=cv, n_jobs=-1,
                                       train_sizes=np.linspace(0.2, 1.0, 5),
scoring="neg_mean_absolute_error")
        ax.plot(sizes, -tr.mean(axis=1), "o-", color=GREY, label="Training")
        ax.plot(sizes, -va.mean(axis=1), "o-", color=BLUE, label="Cross-validation")
        ax.fill_between(sizes, -va.mean(axis=1) - va.std(axis=1), -va.mean(axis=1) + va.std(axis=1),
color=BLUE, alpha=0.12)
        ax.set(title=LABELS[n], xlabel="Training rows")
        out[n] = {"sizes": sizes.tolist(), "train_mae": (-tr.mean(axis=1)).round(4).tolist(),
                  "cv_mae": (-va.mean(axis=1)).round(4).tolist()}
    axes[0].set_ylabel("MAE (kW)")
    axes[0].legend(frameon=False)
    fig.suptitle("Learning curves (time-series cross-validation)", fontsize=10)
    save("learning_curves.png")
    return out


def cv_fold_scores(params, cols, train, tuning):
    X, y = train[cols].values, train["target"].values
    out = {}
    for n, t in tuning.items():
        r = cross_validate(build(n, params, cols, t["best_params"]), X, y,
cv=TimeSeriesSplit(n_splits=5),
                           scoring="neg_mean_absolute_error", n_jobs=1 if n == "random_forest" else
-1)
        out[n] = (-r["test_score"]).round(4).tolist()
    fig, ax = plt.subplots(figsize=(9, 3.4))
    for i, (n, s) in enumerate(out.items()):
        ax.plot(range(1, 6), s, "o-", label=LABELS[n])
    ax.set(title="Mean absolute error in each time-series CV fold (tuned models)", xlabel="Fold
(later folds train on more history)",
           ylabel="MAE (kW)", xticks=range(1, 6))
    ax.legend(frameon=False, fontsize=8)
    save("cv_folds.png")
    return out
```

```
 205
 206
 207  def hyperparameter_curves(params, cols, train, val):
 208      X, y, Xv, yv = train[cols].values, train["target"].values, val[cols].values,
val["target"].values
 209      out = {}
 210      # Decision tree: full depth x leaf grid as a heat map of CV MAE.
 211      grid = params["tuning"]["grids"]["decision_tree"]
 212      gs = GridSearchCV(DecisionTreeRegressor(random_state=42), grid, cv=TimeSeriesSplit(5),
 213                        scoring="neg_mean_absolute_error", n_jobs=-1).fit(X, y)
 214      depths = [str(d) for d in grid["max_depth"]]
 215      leaves = grid["min_samples_leaf"]
 216      heat = np.zeros((len(depths), len(leaves)))
 217      for prm, s in zip(gs.cv_results_["params"], gs.cv_results_["mean_test_score"]):
 218          heat[depths.index(str(prm["max_depth"])), leaves.index(prm["min_samples_leaf"])] = -s
 219      fig, axes = plt.subplots(1, 3, figsize=(11, 3.4))
 220      im = axes[0].imshow(heat, cmap="viridis_r", aspect="auto")
 221      for (i, j), v in np.ndenumerate(heat):
 222          axes[0].text(j, i, f"{v:.3f}", ha="center", va="center", fontsize=6.5, color="white" if v >
heat.mean() else "black")
 223      axes[0].set(title="Decision tree: CV MAE", xticks=range(len(leaves)), xticklabels=leaves,
yticks=range(len(depths)),
 224                  yticklabels=depths, xlabel="min_samples_leaf", ylabel="max_depth")
 225      out["decision_tree_grid"] = {"max_depth": depths, "min_samples_leaf": leaves, "cv_mae":
heat.round(4).tolist()}
 226      # KNN: neighbours vs validation MAE.
 227      ks = [1, 3, 5, 10, 20, 40, 80]
 228      knn = [float(np.mean(np.abs(make_pipeline(StandardScaler(), KNeighborsRegressor(k)).fit(X,
y).predict(Xv) - yv))) for k in ks]
 229      axes[1].plot(ks, knn, "o-", color=BLUE)
 230      axes[1].set(title="KNN: number of neighbours", xlabel="k (log scale)", ylabel="Validation MAE
(kW)", xscale="log")
 231      out["knn_k"] = {"k": ks, "val_mae": np.round(knn, 4).tolist()}
 232      # Ridge: regularisation strength.
 233      alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000]
 234      rid = [float(np.mean(np.abs(make_pipeline(StandardScaler(), Ridge(alpha=a)).fit(X,
y).predict(Xv) - yv))) for a in alphas]
 235      axes[2].plot(alphas, rid, "o-", color=ORANGE)
 236      axes[2].set(title="Ridge: regularisation strength", xlabel="alpha (log scale)",
ylabel="Validation MAE (kW)", xscale="log")
 237      out["ridge_alpha"] = {"alpha": alphas, "val_mae": np.round(rid, 4).tolist()}
 238      save("hyperparameter_curves.png")
 239      return out
 240
 241
 242  # ---------------------------------------------------------------- classification
 243  def classification_analysis(params, cols, train, val, test):
 244      cp = params["classification"]
 245      thr = float(train["target"].quantile(cp["threshold_quantile"]))
 246      lab = lambda d: (d["target"] > thr).astype(int).values
 247      X, y, Xv, yv, Xt, yt = train[cols].values, lab(train), val[cols].values, lab(val),
test[cols].values, lab(test)
 248      tuned_params = json.loads((ROOT / "models" /
"peak_classifier.json").read_text())["hyperparameters"]
 249      models = {
 250          "Logistic regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
 251          "Decision tree (default)": DecisionTreeClassifier(random_state=42),
 252          "Decision tree (tuned)": DecisionTreeClassifier(random_state=42, **tuned_params),
 253      }
 254      full = pd.concat([train, val])
 255      out = {"threshold_kw": thr, "classifiers": {}}
 256      fig_cm, axes_cm = plt.subplots(1, 3, figsize=(10, 3.2))
 257      fig_roc, (ax_roc, ax_pr) = plt.subplots(1, 2, figsize=(10, 3.6))
 258      for ax, (name, m) in zip(axes_cm, models.items()):
 259          m.fit(full[cols].values, lab(full))
 260          proba = m.predict_proba(Xt)[:, 1]
 261          pred = (proba >= 0.5).astype(int)
 262          cm = confusion_matrix(yt, pred)
 263          rep = classification_report(yt, pred, target_names=["Normal hour", "Peak hour"],
output_dict=True)
 264          out["classifiers"][name] = {"confusion_matrix": cm.tolist(), "report": rep, "roc_auc":
float(roc_auc_score(yt, proba))}
 265          ax.imshow(cm, cmap="Blues")
 266          for (i, j), v in np.ndenumerate(cm):
 267              ax.text(j, i, f"{v:,}", ha="center", va="center", color="white" if v > cm.max() / 2 else
"black")
 268          ax.set(title=name, xticks=[0, 1], yticks=[0, 1], xticklabels=["Normal", "Peak"],
yticklabels=["Normal", "Peak"],
 269                 xlabel="Predicted", ylabel="Actual")
```

```
 270          fpr, tpr, _ = roc_curve(yt, proba)
 271          ax_roc.plot(fpr, tpr, label=f"{name} (AUC {out['classifiers'][name]['roc_auc']:.3f})")
 272          pr, rc, _ = precision_recall_curve(yt, proba)
 273          ax_pr.plot(rc, pr, label=name)
 274          if name == "Decision tree (tuned)":
 275              sweep = []
 276              for t in np.linspace(0.05, 0.95, 19):
 277                  pt = (proba >= t).astype(int)
 278                  tp, fp, fn = ((pt == 1) & (yt == 1)).sum(), ((pt == 1) & (yt == 0)).sum(), ((pt ==
0) & (yt == 1)).sum()
 279                  p_, r_ = tp / max(tp + fp, 1), tp / max(tp + fn, 1)
 280                  sweep.append({"threshold": round(float(t), 2), "precision": float(p_), "recall":
float(r_),
 281                                "f1": float(2 * p_ * r_ / max(p_ + r_, 1e-9)), "accuracy": float((pt
== yt).mean())})
 282              out["threshold_sweep"] = sweep
 283      fig_cm.suptitle("Confusion matrices on the test set (refit on train + validation)", fontsize=10)
 284      plt.figure(fig_cm.number)
 285      save("classifier_confusion_matrices.png")
 286      ax_roc.plot([0, 1], [0, 1], color=GREY, ls="--", lw=0.8)
 287      ax_roc.set(title="ROC curves (test)", xlabel="False positive rate", ylabel="True positive rate")
 288      ax_roc.legend(frameon=False, fontsize=7.5, loc="lower right")
 289      ax_pr.axhline(yt.mean(), color=GREY, ls="--", lw=0.8)
 290      ax_pr.set(title="Precision-recall curves (test)", xlabel="Recall", ylabel="Precision")
 291      ax_pr.legend(frameon=False, fontsize=7.5)
 292      plt.figure(fig_roc.number)
 293      save("classifier_roc_pr.png")
 294
 295      s = pd.DataFrame(out["threshold_sweep"])
 296      fig, ax = plt.subplots(figsize=(9, 3.2))
 297      for k, c in [("precision", BLUE), ("recall", ORANGE), ("f1", GREEN), ("accuracy", GREY)]:
 298          ax.plot(s["threshold"], s[k], "o-", color=c, ms=3, label=k.capitalize() if k != "f1" else
"F1")
 299      ax.axvline(0.5, color="black", ls=":", lw=0.8)
 300      ax.set(title="Tuned decision tree: effect of the decision threshold (test)", xlabel="Probability
threshold", ylim=(0, 1))
 301      ax.legend(frameon=False, ncol=4)
 302      save("classifier_threshold_sweep.png")
 303      return out
 304
 305
 306  # ---------------------------------------------------------------- data analysis
 307  def eda_extra(params):
 308      raw = pd.read_csv(resolve(params["data"]["raw_csv"]), parse_dates=["Datetime"],
index_col="Datetime",
 309                        usecols=["Datetime", "Global_active_power"])
 310      hourly = pd.read_parquet(resolve(params["data"]["processed"]))
 311      out = {}
 312      miss = raw["Global_active_power"].isna().resample("MS").mean() * 100
 313      fig, ax = plt.subplots(figsize=(9, 3))
 314      ax.bar(miss.index, miss.values, width=25, color=RED)
 315      ax.set(title="Share of missing one-minute readings per month (%)", ylabel="% missing")
 316      save("missing_by_month.png")
 317      out["missing_pct_by_month_max"] = {"month": str(miss.idxmax().date()), "pct":
round(float(miss.max()), 2)}
 318
 319      cols = ["Global_active_power", "Global_reactive_power", "Voltage", "Global_intensity",
 320              "Sub_metering_1", "Sub_metering_2", "Sub_metering_3", "Sub_metering_other"]
 321      fig, axes = plt.subplots(2, 4, figsize=(11, 5))
 322      for ax, c in zip(axes.flat, cols):
 323          ax.hist(hourly[c], bins=60, color=BLUE)
 324          ax.set_title(c.replace("_", " "), fontsize=8.5)
 325      fig.suptitle("Distribution of each hourly variable", fontsize=10)
 326      save("distributions.png")
 327      out["describe"] = hourly[cols].describe().round(3).to_dict()
 328
 329      y = hourly["Global_active_power"]
 330      lags = np.arange(1, 24 * 8 + 1)
 331      acf = [float(y.autocorr(l)) for l in lags]
 332      fig, ax = plt.subplots(figsize=(9, 3))
 333      ax.bar(lags, acf, width=0.8, color=BLUE)
 334      for d in range(1, 9):
 335          ax.axvline(24 * d, color=ORANGE, lw=0.6, ls="--")
 336      ax.set(title="Autocorrelation of hourly consumption (dashed: multiples of 24 h)", xlabel="Lag
(hours)", ylabel="Autocorrelation")
 337      save("autocorrelation.png")
 338      out["autocorr"] = {"lag_1": round(acf[0], 3), "lag_24": round(acf[23], 3), "lag_168":
round(acf[167], 3)}
 339
```

```
    week = y.groupby([y.index.dayofweek, y.index.hour]).mean().values
    fig, ax = plt.subplots(figsize=(9, 3))
    ax.plot(np.arange(168), week, color=BLUE)
    ax.set(title="Average weekly load profile (Monday 00:00 to Sunday 23:00)", xlabel="Hour of
week", ylabel="kW",
           xticks=np.arange(0, 169, 24), xticklabels=["Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun", ""])
    save("weekly_profile.png")

    sub = hourly[["Sub_metering_1", "Sub_metering_2", "Sub_metering_3", "Sub_metering_other"]]
    prof = sub.groupby(sub.index.hour).mean() / 1000
    fig, ax = plt.subplots(figsize=(9, 3.2))
    ax.stackplot(prof.index, prof.T.values, labels=["Kitchen", "Laundry", "Water heater / AC",
"Other"],
                 colors=[BLUE, ORANGE, GREEN, "#c9ced6"])
    ax.set(title="Average energy by sub-meter and hour of day", xlabel="Hour", ylabel="kWh per
hour")
    ax.legend(frameon=False, ncol=4, fontsize=8)
    save("submeter_by_hour.png")

    fig, ax = plt.subplots(figsize=(9, 3.2))
    data = [y[y.index.month == m].values for m in range(1, 13)]
    ax.boxplot(data, widths=0.6, flierprops={"markersize": 1.5, "alpha": 0.3})
    ax.set(title="Hourly consumption by month", xlabel="Month", ylabel="kW")
    save("monthly_boxplot.png")

    yearly = y.groupby([y.index.year, y.index.month]).mean().unstack(0)
    fig, ax = plt.subplots(figsize=(9, 3.2))
    for yr in yearly.columns:
        ax.plot(yearly.index, yearly[yr], "o-", ms=3, label=str(yr))
    ax.set(title="Monthly average consumption, year by year", xlabel="Month", ylabel="kW",
xticks=range(1, 13))
    ax.legend(frameon=False, ncol=5)
    save("year_over_year.png")
    return out


def drift_over_time(params, train, test):
    preds = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet")
    out = {"monthly_psi": {}, "monthly_mae": {}}
    months = sorted(set(test.index.to_period("M")))
    for m in months:
        part = test[test.index.to_period("M") == m]
        out["monthly_psi"][str(m)] = {c: round(psi(train[c].values, part[c].values), 4) for c in
MONITORED if c != "hour"}
        pm = preds[preds.index.to_period("M") == m]
        out["monthly_mae"][str(m)] = round(float((pm["actual"] - pm["predicted"]).abs().mean()), 4)
    df = pd.DataFrame(out["monthly_psi"]).T
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.4))
    for c in df.columns:
        axes[0].plot(range(len(df)), df[c], "o-", ms=3, label=c)
    axes[0].axhline(0.2, color=RED, ls="--", lw=0.8)
    axes[0].set(title="PSI against training data, month by month", xticks=range(len(df)),
xticklabels=df.index, ylabel="PSI")
    axes[0].legend(frameon=False, fontsize=6.5)
    axes[1].bar(list(out["monthly_mae"]), list(out["monthly_mae"].values()), color=BLUE)
    axes[1].set(title="Test MAE by month", ylabel="MAE (kW)")
    for a in axes:
        a.tick_params(axis="x", rotation=30)
    save("monthly_drift.png")
    return out


def main() -> None:
    params = load_params()
    OUT.mkdir(parents=True, exist_ok=True)
    cols = feature_columns(params)
    df = pd.read_parquet(resolve(params["data"]["features"]))
    train, val, test = split(df, params)
    tuning = json.loads((ROOT / "reports" / "metrics.json").read_text())["tuning"]

    test_results, preds = all_models_on_test(params, cols, train, val, test, tuning)
    per_model_figures(test, preds)
    test_metric_bars(test_results)
    analysis = {
        "test_all_models": test_results,
        "horizon": horizon_curve(params),
        "learning_curves": learning_curves(params, cols, train),
        "cv_folds": cv_fold_scores(params, cols, train, tuning),
```

```
        "hyperparameters": hyperparameter_curves(params, cols, train, val),
        "classification": classification_analysis(params, cols, train, val, test),
        "eda": eda_extra(params),
        "drift": drift_over_time(params, train, test),
    }
    (ROOT / "reports" / "analysis.json").write_text(json.dumps(analysis, indent=2))
    log.info("Analysis written: %d figures", len(list(OUT.glob("*.png"))))


if __name__ == "__main__":
    main()
```

src/predict.py: recursive forecasting

```
"""Inference helpers shared by the API and the dashboard."""
import json
from functools import lru_cache

import joblib
import numpy as np
import pandas as pd

from src.features import build_features
from src.utils import ROOT, load_params

MIN_HISTORY_HOURS = 169  # longest lag (168h) + 1


@lru_cache(maxsize=1)
def load_model():
    model = joblib.load(ROOT / "models" / "model.joblib")
    if "n_jobs" in model.get_params():
        model.set_params(n_jobs=1)  # forecasts predict one row at a time; worker start-up would dominate
    meta = json.loads((ROOT / "models" / "metadata.json").read_text())
    return model, meta


@lru_cache(maxsize=1)
def load_classifier():
    """Peak-hour classifier and its kW threshold (see src/classify.py)."""
    clf = joblib.load(ROOT / "models" / "peak_classifier.joblib")
    info = json.loads((ROOT / "models" / "peak_classifier.json").read_text())
    return clf, info["threshold_kw"]


def _next_row(history: pd.DataFrame, params: dict) -> pd.DataFrame:
    """Append a placeholder for the next hour and compute its feature row."""
    next_ts = history.index[-1] + pd.Timedelta(hours=1)
    ext = pd.concat([history, pd.DataFrame(index=[next_ts], columns=history.columns, dtype=float)])
    target = params["features"]["target"]
    ext[target] = ext[target].fillna(-1.0)  # placeholder so dropna() keeps the row; not used as a feature
    feats = build_features(ext, target, params["features"]["lags"], params["features"]["rolling_windows"])
    return feats.iloc[[-1]]


def forecast(history: pd.DataFrame, horizon: int = 1) -> pd.DataFrame:
    """Recursive multi-step forecast from an hourly history frame.

    Exogenous readings (voltage, sub-meters...) for future hours are unknown,
    so they are carried forward from the last observed hour.

    Peak-hour alerts: the classifier is a next-hour model, so it is used for
    step 1 only. Later hours are flagged when the forecast exceeds the peak
    threshold, which measured 76% accurate over hours 2-24 versus 58% for the
    classifier fed carried-forward readings.
    """
    params = load_params()
    model, meta = load_model()
    clf, peak_kw = load_classifier()
    if len(history) < MIN_HISTORY_HOURS:
        raise ValueError(f"Need at least {MIN_HISTORY_HOURS} hours of history, got {len(history)}")
    hist = history.sort_index().copy()
```

```
    target = params["features"]["target"]
    out = []
    for step in range(1, horizon + 1):
        row = _next_row(hist, params)
        x = row[meta["features"]].values
        yhat = float(max(model.predict(x)[0], 0.0))
        if step == 1:
            peak_prob = float(clf.predict_proba(x)[0, 1])
            alert, method = peak_prob >= 0.5, "classifier"
        else:
            peak_prob, alert, method = None, yhat > peak_kw, "forecast_threshold"
        # Uncertainty grows with the recursive horizon.
        band = 1.96 * meta["residual_std"] * np.sqrt(step)
        out.append({"timestamp": row.index[0], "predicted_kw": yhat,
                    "lower_kw": max(yhat - band, 0.0), "upper_kw": yhat + band,
                    "peak_probability": peak_prob, "peak_alert": bool(alert), "alert_method":
method})
        new = hist.iloc[[-1]].copy()
        new.index = row.index
        new[target] = yhat
        hist = pd.concat([hist, new])
    return pd.DataFrame(out)
```

src/export_web.py: web data export

```
"""Stage 7 - Export: lightweight artifacts for the hosted web app (React site in web/).

Writes the recent hourly history the API needs (CSV, so serving does not need
pyarrow) and a JSON bundle of precomputed charts for the static dashboard page.
"""
import json
import shutil

import pandas as pd

from src.predict import MIN_HISTORY_HOURS
from src.utils import ROOT, get_logger, load_params, resolve

log = get_logger("export_web")


def main() -> None:
    params = load_params()
    hourly = pd.read_parquet(resolve(params["data"]["processed"]))
    meta = json.loads((ROOT / "models" / "metadata.json").read_text())

    test_start = pd.Timestamp(meta["test_period"][0])
    recent = hourly[hourly.index >= test_start - pd.Timedelta(hours=MIN_HISTORY_HOURS * 2)]
    recent.round(4).to_csv(resolve(params["data"]["serving"]), index_label="datetime")

    y = hourly["Global_active_power"]
    daily = y.resample("D").mean().round(3)
    heat = y.groupby([y.index.dayofweek, y.index.hour]).mean().unstack().round(3)
    profile = y.groupby([y.index.dayofweek >= 5, y.index.hour]).mean().unstack(0).round(3)
    sub = (hourly[["Sub_metering_1", "Sub_metering_2", "Sub_metering_3",
"Sub_metering_other"]].sum() / 1000).round(0)
    preds = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet").iloc[-24 * 14:].round(3)
    imp = pd.read_csv(ROOT / "reports" / "feature_importance.csv",
index_col=0)["importance"].sort_values()
    bundle = {
        "stats": json.loads((ROOT / "reports" / "data_stats.json").read_text()),
        "metrics": json.loads((ROOT / "reports" / "metrics.json").read_text()),
        "monitoring": json.loads((ROOT / "reports" / "monitoring.json").read_text()),
        "model": {k: meta[k] for k in ["model_name", "trained_at", "test_period", "test_metrics",
                                       "baseline_test_metrics"]} | {"n_features":
len(meta["features"])},
        "daily": {"x": [d.strftime("%Y-%m-%d") for d in daily.index], "y": daily.tolist()},
        "heatmap": heat.values.tolist(),
        "profile": {"weekday": profile[False].tolist(), "weekend": profile[True].tolist()},
        "submeter": dict(zip(["Kitchen", "Laundry", "Water heater / AC", "Other"], sub.tolist())),
        "test_window": {"x": [t.isoformat() for t in preds.index], "actual":
preds["actual"].tolist(),
                        "predicted": preds["predicted"].tolist()},
        "importance": {"names": imp.index.tolist(), "values": imp.round(4).tolist()},
        "forecast_range": [str(recent.index[MIN_HISTORY_HOURS]), str(recent.index[-1])],
```

```
        "classification": json.loads((ROOT / "reports" / "classification.json").read_text()),
        "fairness": json.loads((ROOT / "reports" / "fairness.json").read_text()),
    }
    out = resolve(params["data"]["web_data"])
    out.write_text(json.dumps(bundle))
    img = ROOT / "web" / "public" / "img"
    img.mkdir(parents=True, exist_ok=True)
    shutil.copy(ROOT / "report" / "diagrams" / "fig_architecture.png", img / "architecture.png")
    log.info("Wrote %d serving rows and dashboard bundle (%.0f KB)", len(recent), out.stat().st_size
/ 1024)


if __name__ == "__main__":
    main()
```

serving/main.py: Flask REST API

```
"""Flask REST API serving the VoltCast models, with Prometheus metrics and prediction logging.

Endpoints (mounted under /api by app.py):
  GET  /health          liveness + loaded model
  GET  /model-info      model, training window and test metrics
  POST /predict         forecast from user-supplied hourly readings (JSON)
  GET  /forecast        forecast from a point in the stored test period, with actual values
  GET  /sample-request  ready-made body for POST /predict
  GET  /metrics         Prometheus metrics (scraped by Prometheus, shown in Grafana)
"""
import json
import logging
import os
import time
from datetime import datetime
from functools import lru_cache
from pathlib import Path

import pandas as pd
from flask import Blueprint, Response, g, jsonify, request
from prometheus_client import CONTENT_TYPE_LATEST, Counter, Gauge, Histogram, generate_latest
from pydantic import BaseModel, Field, ValidationError

from src.predict import MIN_HISTORY_HOURS, forecast, load_classifier, load_model
from src.utils import ROOT, load_params

api = Blueprint("api", __name__)

# ---------- Prometheus metrics ----------
REQUESTS = Counter("voltcast_requests_total", "HTTP requests", ["endpoint", "method", "status"])
LATENCY = Histogram("voltcast_request_latency_seconds", "Request latency", ["endpoint"],
                    buckets=(0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5))
PREDICTIONS = Counter("voltcast_predictions_total", "Hourly values forecast")
PREDICTED_KW = Histogram("voltcast_predicted_kw", "Distribution of forecast consumption (kW)",
                         buckets=(0.25, 0.5, 1, 1.5, 2, 3, 4, 6))
PEAK_ALERTS = Counter("voltcast_peak_alerts_total", "Forecast hours flagged as peak hours")
ERRORS = Counter("voltcast_prediction_errors_total", "Rejected or failed prediction requests",
["reason"])
MODEL_INFO = Gauge("voltcast_model_info", "Loaded model (value is always 1)", ["model",
"trained_at"])
TEST_MAE = Gauge("voltcast_model_test_mae_kw", "Test-set MAE of the loaded model")


# ---------- prediction log (one JSON line per request) ----------
def _prediction_logger() -> logging.Logger:
    logger = logging.getLogger("voltcast.predictions")
    if not logger.handlers:
        logger.setLevel(logging.INFO)
        logger.propagate = False
        logger.addHandler(logging.StreamHandler())
        # Serverless hosts (Vercel) have a read-only disk: log to stdout only there.
        if not os.environ.get("VERCEL"):
            log_dir = Path(os.environ.get("VOLTCAST_LOG_DIR", ROOT / "logs"))
            log_dir.mkdir(parents=True, exist_ok=True)
            logger.addHandler(logging.FileHandler(log_dir / "predictions.jsonl", encoding="utf-8"))
    return logger


```

```
 57  PRED_LOG = _prediction_logger()
 58
 59
 60  def log_predictions(source: str, fc: pd.DataFrame, extra: dict | None = None) -> None:
 61      PREDICTIONS.inc(len(fc))
 62      PEAK_ALERTS.inc(int(fc["peak_alert"].sum()))
 63      for v in fc["predicted_kw"]:
 64          PREDICTED_KW.observe(v)
 65      _, meta = load_model()
 66      PRED_LOG.info(json.dumps({
 67          "time": datetime.now().isoformat(timespec="seconds"), "source": source, "model":
meta["model_name"],
 68          "horizon": len(fc), "first_hour": str(fc["timestamp"].iloc[0]),
 69          "predicted_kw": [round(v, 3) for v in fc["predicted_kw"]],
 70          "peak_hours": int(fc["peak_alert"].sum()), **(extra or {}),
 71      }))
 72
 73
 74  @api.before_app_request
 75  def _start_timer():
 76      g.t0 = time.perf_counter()
 77
 78
 79  @api.after_app_request
 80  def _record(response):
 81      endpoint = request.url_rule.rule if request.url_rule else "unmatched"
 82      if endpoint.endswith("/metrics"):
 83          return response
 84      REQUESTS.labels(endpoint, request.method, response.status_code).inc()
 85      LATENCY.labels(endpoint).observe(time.perf_counter() - g.get("t0", time.perf_counter()))
 86      return response
 87
 88
 89  # ---------- input validation ----------
 90  class Reading(BaseModel):
 91      timestamp: datetime
 92      Global_active_power: float = Field(ge=0, description="Hourly mean active power, kW")
 93      Global_reactive_power: float = Field(ge=0)
 94      Voltage: float = Field(gt=0)
 95      Global_intensity: float = Field(ge=0)
 96      Sub_metering_1: float = Field(ge=0, description="Kitchen, Wh in the hour")
 97      Sub_metering_2: float = Field(ge=0, description="Laundry, Wh in the hour")
 98      Sub_metering_3: float = Field(ge=0, description="Water heater & AC, Wh in the hour")
 99
100
101  class ForecastRequest(BaseModel):
102      history: list[Reading]
103      horizon: int = Field(default=1, ge=1, le=48)
104
105
106  def error(status: int, message: str, reason: str):
107      ERRORS.labels(reason).inc()
108      return jsonify({"error": message}), status
109
110
111  def to_records(fc: pd.DataFrame) -> list[dict]:
112      # NaN is not valid JSON: missing values (e.g. no classifier probability) become null.
113      clean = fc.astype(object).where(fc.notna(), None)
114      return [{**r, "timestamp": r["timestamp"].isoformat()} for r in clean.to_dict(orient="records")]
115
116
117  @lru_cache(maxsize=1)
118  def serving_history() -> pd.DataFrame:
119      """Recent hourly readings shipped with the service (test period + two weeks of context)."""
120      params = load_params()
121      return pd.read_csv(ROOT / params["data"]["serving"], index_col="datetime", parse_dates=True)
122
123
124  # ---------- endpoints ----------
125  @api.get("/health")
126  def health():
127      _, meta = load_model()
128      return jsonify(status="ok", model=meta["model_name"], trained_at=meta["trained_at"])
129
130
131  @api.get("/model-info")
132  def model_info():
133      _, meta = load_model()
134      _, peak_kw = load_classifier()
```

```
 135      keys = ["model_name", "hyperparameters", "tuned", "trained_at", "mlflow_run_id", "train_period",
 136              "test_period", "test_metrics", "baseline_test_metrics"]
 137      return jsonify({k: meta[k] for k in keys} | {"n_features": len(meta["features"]),
"peak_threshold_kw": peak_kw})
 138
 139
 140  @api.post("/predict")
 141  def predict():
 142      try:
 143          req = ForecastRequest.model_validate(request.get_json(force=True, silent=False))
 144      except ValidationError as e:
 145          return error(422, json.loads(e.json(include_url=False)), "validation")
 146      except Exception:
 147          return error(400, "body must be JSON", "bad_json")
 148      if len(req.history) < MIN_HISTORY_HOURS:
 149          return error(422, f"history must contain at least {MIN_HISTORY_HOURS} hourly readings",
"short_history")
 150      hist = pd.DataFrame([r.model_dump() for r in req.history]).set_index("timestamp").sort_index()
 151      hist.index = pd.DatetimeIndex(hist.index).tz_localize(None)
 152      if hist.index.has_duplicates:
 153          return error(422, "history contains duplicate timestamps", "duplicates")
 154      hist = hist.asfreq("1h").interpolate(limit_direction="both")
 155      hist["Sub_metering_other"] = (hist["Global_active_power"] * 1000
 156                                    - hist[["Sub_metering_1", "Sub_metering_2",
"Sub_metering_3"]].sum(axis=1)).clip(lower=0)
 157      fc = forecast(hist, req.horizon)
 158      log_predictions("predict", fc)
 159      _, meta = load_model()
 160      return jsonify(model=meta["model_name"], horizon=req.horizon, forecast=to_records(fc),
 161                     predicted_energy_kwh=round(float(fc["predicted_kw"].sum()), 3))
 162
 163
 164  @api.get("/forecast")
 165  def forecast_at():
 166      """Forecast from a point in the stored test period and compare with what actually happened."""
 167      try:
 168          origin = pd.Timestamp(request.args["origin"]).tz_localize(None).floor("h")
 169          horizon = int(request.args.get("horizon", 24))
 170      except (KeyError, ValueError):
 171          return error(422, "query needs origin=YYYY-MM-DDTHH:MM and an integer horizon",
"validation")
 172      if not 1 <= horizon <= 48:
 173          return error(422, "horizon must be between 1 and 48", "validation")
 174      hourly = serving_history()
 175      history = hourly[hourly.index < origin]
 176      if len(history) < MIN_HISTORY_HOURS:
 177          return error(422, f"origin must be at least {MIN_HISTORY_HOURS} h after {hourly.index[0]}",
"short_history")
 178      fc = forecast(history.iloc[-MIN_HISTORY_HOURS * 2:], horizon)
 179      actual = hourly["Global_active_power"].reindex(fc["timestamp"])
 180      past = hourly["Global_active_power"].loc[origin - pd.Timedelta(hours=72): origin -
pd.Timedelta(hours=1)]
 181      log_predictions("forecast", fc, {"origin": str(origin)})
 182      return jsonify(
 183          forecast=[{**r, "actual_kw": None if pd.isna(a) else float(a)} for r, a in
zip(to_records(fc), actual.values)],
 184          history=[{"timestamp": t.isoformat(), "kw": float(v)} for t, v in past.items()],
 185          predicted_energy_kwh=round(float(fc["predicted_kw"].sum()), 3),
 186          range=[str(hourly.index[MIN_HISTORY_HOURS]), str(hourly.index[-1])],
 187      )
 188
 189
 190  @api.get("/sample-request")
 191  def sample_request():
 192      """Returns a ready-to-use request body built from the last hours of the dataset."""
 193      hourly = serving_history().iloc[-MIN_HISTORY_HOURS:]
 194      cols = list(Reading.model_fields)[1:]
 195      rows = hourly[cols].round(3).reset_index().rename(columns={"datetime": "timestamp"})
 196      rows["timestamp"] = rows["timestamp"].astype(str)
 197      return jsonify(history=rows.to_dict(orient="records"), horizon=24)
 198
 199
 200  @api.get("/metrics")
 201  def metrics():
 202      _, meta = load_model()
 203      MODEL_INFO.labels(meta["model_name"], meta["trained_at"]).set(1)
 204      TEST_MAE.set(meta["test_metrics"]["mae"])
 205      return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)
```

app.py: web entry point

```
"""Web entrypoint (Vercel / Docker / local): Flask API under /api, React site at /."""
from pathlib import Path

from flask import Flask, send_from_directory

from serving.main import api

PUBLIC = Path(__file__).parent / "public"

app = Flask(__name__, static_folder=None)
app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024  # reject oversized request bodies
app.register_blueprint(api, url_prefix="/api")


# On Vercel the CDN serves public/ directly; locally and in Docker Flask serves the built React site.
@app.get("/")
@app.get("/<path:path>")
def web(path: str = "index.html"):
    target = PUBLIC / path
    return send_from_directory(PUBLIC, path if target.is_file() else "index.html")


if __name__ == "__main__":
    app.run(port=8000, debug=False)
```

tests/test_pipeline.py: automated tests

```
import json

import numpy as np
import pandas as pd
import pytest

from app import app
from src.classify import scores
from src.features import build_features, feature_columns
from src.monitor import psi
from src.predict import MIN_HISTORY_HOURS
from src.preprocess import clean_and_resample
from src.train import metrics
from src.utils import load_params

PARAMS = load_params()
client = app.test_client()


def synthetic_hourly(hours=400):
    idx = pd.date_range("2010-01-01", periods=hours, freq="h")
    rng = np.random.default_rng(0)
    gap = 1 + 0.8 * np.sin(2 * np.pi * idx.hour / 24) + rng.normal(0, 0.1, hours)
    return pd.DataFrame({
        "Global_active_power": np.abs(gap), "Global_reactive_power": 0.1, "Voltage": 240.0,
        "Global_intensity": np.abs(gap) * 4.2, "Sub_metering_1": 1.0, "Sub_metering_2": 2.0,
        "Sub_metering_3": 5.0, "Sub_metering_other": 10.0,
    }, index=idx)


def test_resample_fills_gaps():
    idx = pd.date_range("2010-01-01", periods=180, freq="min")
    raw = pd.DataFrame({c: 1.0 for c in ["Global_active_power", "Global_reactive_power", "Voltage",
                                         "Global_intensity", "Sub_metering_1", "Sub_metering_2",
                                         "Sub_metering_3"]}, index=idx)
    raw.iloc[60:120] = np.nan  # one fully missing hour
    hourly = clean_and_resample(raw, "1h", 6)
    assert len(hourly) == 3
    assert not hourly.isna().any().any()


def test_features_have_no_leakage():
    h = synthetic_hourly()
```

```
  44      f = build_features(h, "Global_active_power", PARAMS["features"]["lags"],
PARAMS["features"]["rolling_windows"])
  45      assert list(f.columns[:-1]) == feature_columns(PARAMS)
  46      # lag_1 at time t must equal the target at t-1, never the target at t.
  47      t = f.index[10]
  48      assert f.loc[t, "lag_1"] == pytest.approx(h.loc[t - pd.Timedelta(hours=1),
"Global_active_power"])
  49      assert f.loc[t, "roll_mean_3"] == pytest.approx(
  50          h["Global_active_power"].loc[t - pd.Timedelta(hours=3): t - pd.Timedelta(hours=1)].mean())
  51
  52
  53  def test_metrics_perfect_prediction():
  54      m = metrics([1.0, 2.0, 3.0], [1.0, 2.0, 3.0])
  55      assert m["mae"] == 0 and m["rmse"] == 0 and m["r2"] == 1
  56
  57
  58  def test_psi_detects_shift():
  59      rng = np.random.default_rng(1)
  60      ref = rng.normal(0, 1, 5000)
  61      assert psi(ref, rng.normal(0, 1, 5000)) < 0.1
  62      assert psi(ref, rng.normal(1.5, 1, 5000)) > 0.2
  63
  64
  65  def test_classification_scores():
  66      s = scores(np.array([0, 0, 1, 1]), np.array([0.1, 0.6, 0.4, 0.9]))
  67      assert s["accuracy"] == 0.5 and s["precision"] == 0.5 and s["recall"] == 0.5
  68      assert s["confusion_matrix"] == [[1, 1], [1, 1]]
  69
  70
  71  def test_health_endpoint():
  72      r = client.get("/api/health")
  73      assert r.status_code == 200 and r.get_json()["status"] == "ok"
  74
  75
  76  def test_predict_endpoint_returns_horizon():
  77      body = client.get("/api/sample-request").get_json()
  78      body["horizon"] = 6
  79      r = client.post("/api/predict", json=body)
  80      assert r.status_code == 200, r.get_data(as_text=True)
  81      # Strict parse: browsers reject NaN/Infinity, so the API must never emit them.
  82      out = json.loads(r.get_data(as_text=True), parse_constant=lambda c: pytest.fail(f"invalid JSON
constant {c}"))
  83      assert len(out["forecast"]) == 6
  84      assert all(p["lower_kw"] <= p["predicted_kw"] <= p["upper_kw"] for p in out["forecast"])
  85      assert 0 <= out["forecast"][0]["peak_probability"] <= 1 and out["forecast"][0]["alert_method"]
== "classifier"
  86      assert all(p["alert_method"] == "forecast_threshold" for p in out["forecast"][1:])
  87
  88
  89  def test_predict_rejects_short_history():
  90      body = client.get("/api/sample-request").get_json()
  91      body["history"] = body["history"][: MIN_HISTORY_HOURS - 10]
  92      assert client.post("/api/predict", json=body).status_code == 422
  93
  94
  95  def test_predict_rejects_invalid_reading():
  96      body = client.get("/api/sample-request").get_json()
  97      body["history"][0]["Voltage"] = -5
  98      assert client.post("/api/predict", json=body).status_code == 422
  99
 100
 101  def test_prometheus_metrics_exposed():
 102      client.get("/api/forecast?origin=2010-11-20T08:00&horizon=3")
 103      text = client.get("/api/metrics").get_data(as_text=True)
 104      assert "voltcast_predictions_total" in text and "voltcast_request_latency_seconds" in text
 105
 106
 107  def test_model_beats_baseline():
 108      import json
 109      from src.utils import ROOT
 110      meta = json.loads((ROOT / "models" / "metadata.json").read_text())
 111      assert meta["test_metrics"]["mae"] < meta["baseline_test_metrics"]["mae"]
```

dashboard/app.py: Streamlit dashboard

```
"""Streamlit dashboard: explore data, generate forecasts, review model and monitoring results."""
import json
import sys
from pathlib import Path

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from src.predict import MIN_HISTORY_HOURS, forecast, load_model  # noqa: E402
from src.utils import ROOT, load_params  # noqa: E402

st.set_page_config(page_title="VoltCast", page_icon="⚡", layout="wide")
PARAMS = load_params()
TARIFF = 0.18  # EUR per kWh, used only for the cost estimate


@st.cache_data
def hourly() -> pd.DataFrame:
    return pd.read_parquet(ROOT / PARAMS["data"]["processed"])


@st.cache_data
def read_json(name: str) -> dict:
    return json.loads((ROOT / "reports" / name).read_text())


data = hourly()
_, meta = load_model()
metrics = read_json("metrics.json")
monitor = read_json("monitoring.json")

st.title("⚡ VoltCast")
st.subheader("Household Electricity Consumption Prediction System")
st.caption("MLOps project · UCI Individual Household Electric Power Consumption dataset "
           "(Sceaux, France, Dec 2006 – Nov 2010, 2.07 M one-minute readings)")

tab_fc, tab_data, tab_model, tab_mon, tab_ops = st.tabs(
    ["🔮 Forecast", "📊 Data explorer", "⏳ Model performance", "🩺 Monitoring", "⚙️ MLOps
pipeline"])

with tab_fc:
    c1, c2 = st.columns([1, 3])
    with c1:
        test_start = pd.Timestamp(meta["test_period"][0])
        min_d, max_d = (test_start + pd.Timedelta(days=8)).date(), data.index.max().date()
        day = st.date_input("Forecast from (test period)", value=max_d - pd.Timedelta(days=3),
                            min_value=min_d, max_value=max_d)
        hour = st.slider("Starting hour", 0, 23, 0)
        horizon = st.slider("Horizon (hours ahead)", 1, 48, 24)
        origin = pd.Timestamp(day) + pd.Timedelta(hours=hour)
    history = data[data.index < origin]
    fc = forecast(history.iloc[-MIN_HISTORY_HOURS * 2:], horizon)
    actual = data["Global_active_power"].reindex(fc["timestamp"])
    with c1:
        energy = fc["predicted_kw"].sum()
        st.metric("Predicted energy", f"{energy:.2f} kWh")
        st.metric("Estimated cost", f"€{energy * TARIFF:.2f}", help=f"At €{TARIFF}/kWh")
        st.metric("Peak hour", fc.loc[fc["predicted_kw"].idxmax(), "timestamp"].strftime("%a
%H:%M"),
                  f"{fc['predicted_kw'].max():.2f} kW", delta_color="off")
        if actual.notna().all():
            st.metric("MAE vs actual", f"{(actual.values - fc['predicted_kw']).abs().mean():.3f}
kW")
    with c2:
        past = data["Global_active_power"].loc[origin - pd.Timedelta(hours=72): origin -
pd.Timedelta(hours=1)]
        fig = go.Figure()
        fig.add_trace(go.Scatter(x=list(fc["timestamp"]) + list(fc["timestamp"])[::-1],
                                 y=list(fc["upper_kw"]) + list(fc["lower_kw"])[::-1],
                                 fill="toself", fillcolor="rgba(224,123,36,0.15)",
line=dict(width=0),
                                 name="95% interval", hoverinfo="skip"))
        fig.add_trace(go.Scatter(x=past.index, y=past, name="History", line=dict(color="#1f5fa8")))
```

```
 72          fig.add_trace(go.Scatter(x=fc["timestamp"], y=actual, name="Actual",
 73                                   line=dict(color="#888", dash="dot")))
 74          fig.add_trace(go.Scatter(x=fc["timestamp"], y=fc["predicted_kw"], name="Forecast",
 75                                   line=dict(color="#e07b24", width=3)))
 76          fig.add_vline(x=origin, line_dash="dash", line_color="black")
 77          fig.update_layout(height=430, yaxis_title="Active power (kW)", hovermode="x unified",
 78                            margin=dict(t=20, b=10), legend=dict(orientation="h", y=1.08))
 79          st.plotly_chart(fig, use_container_width=True)
 80          with st.expander("Forecast table"):
 81              st.dataframe(fc.assign(actual_kw=actual.values).round(3), use_container_width=True)
 82
 83  with tab_data:
 84      gran = st.radio("Granularity", ["Hourly", "Daily", "Weekly", "Monthly"], index=1,
horizontal=True)
 85      rule = {"Hourly": "h", "Daily": "D", "Weekly": "W", "Monthly": "MS"}[gran]
 86      series = data["Global_active_power"].resample(rule).mean()
 87      st.plotly_chart(px.line(series, labels={"value": "kW", "datetime": ""}, height=320)
 88                      .update_layout(showlegend=False, margin=dict(t=10)), use_container_width=True)
 89      c1, c2 = st.columns(2)
 90      y = data["Global_active_power"]
 91      prof = y.groupby([y.index.dayofweek >= 5, y.index.hour]).mean().unstack(0)
 92      prof.columns = ["Weekday", "Weekend"]
 93      c1.plotly_chart(px.line(prof, title="Daily load profile", labels={"value": "kW", "datetime":
"Hour"},
 94                              height=320), use_container_width=True)
 95      sub = data[["Sub_metering_1", "Sub_metering_2", "Sub_metering_3", "Sub_metering_other"]].sum() /
1000
 96      sub.index = ["Kitchen", "Laundry", "Water heater / AC", "Other"]
 97      c2.plotly_chart(px.pie(values=sub.values, names=sub.index, title="Energy share by sub-meter
(kWh)",
 98                             height=320), use_container_width=True)
 99      stats = read_json("data_stats.json")
100      st.info(f"Raw rows: {stats['raw_rows']:,} · missing: {stats['raw_missing_pct']}% · "
101              f"hourly rows after cleaning: {stats['hourly_rows']:,}")
102
103  with tab_model:
104      st.subheader(f"Production model: `{meta['model_name']}`")
105      tm, bm = meta["test_metrics"], meta["baseline_test_metrics"]
106      k = st.columns(4)
107      k[0].metric("Test MAE", f"{tm['mae']:.3f} kW", f"{(tm['mae'] / bm['mae'] - 1) * 100:.1f}% vs
baseline",
108                  delta_color="inverse")
109      k[1].metric("Test RMSE", f"{tm['rmse']:.3f} kW", f"{(tm['rmse'] / bm['rmse'] - 1) * 100:.1f}% vs
baseline",
110                  delta_color="inverse")
111      k[2].metric("Test R²", f"{tm['r2']:.3f}", f"{tm['r2'] - bm['r2']:+.3f} vs baseline")
112      k[3].metric("Features", len(meta["features"]))
113      val = pd.DataFrame(metrics["validation"]).T.round(4)
114      st.markdown("**Validation comparison (Jan – May 2010)**")
115      st.dataframe(val.style.highlight_min(subset=["mae", "rmse", "mape"], color="#d4edda")
116                   .highlight_max(subset=["r2"], color="#d4edda"), use_container_width=True)
117      c1, c2 = st.columns(2)
118      c1.image(str(ROOT / "reports/figures/fig_feature_importance.png"))
119      c2.image(str(ROOT / "reports/figures/fig_residuals.png"))
120      st.caption(f"MLflow run id: `{meta['mlflow_run_id']}` · trained {meta['trained_at']} · "
121                 f"train window {meta['train_period'][0]} → {meta['train_period'][1]}")
122
123  with tab_mon:
124      if monitor["retraining_recommended"]:
125          st.warning(f"Retraining recommended: drift in {', '.join(monitor['drifted_features']) or
'none'}; "
126                     f"recent MAE change {monitor['mae_degradation_pct']:+.1f}%")
127      else:
128          st.success("Model healthy: no significant drift or degradation")
129      c1, c2 = st.columns(2)
130      psi = pd.Series(monitor["psi"]).sort_values()
131      c1.plotly_chart(px.bar(psi, orientation="h", title="Population Stability Index (train vs test)",
132                             color=psi > PARAMS["monitoring"]["psi_threshold"],
133                             color_discrete_map={True: "#e07b24", False: "#1f5fa8"}, height=350)
134                      .update_layout(showlegend=False, xaxis_title="PSI", yaxis_title=""),
135                      use_container_width=True)
136      preds = pd.read_parquet(ROOT / "reports" / "test_predictions.parquet")
137      roll = (preds["actual"] - preds["predicted"]).abs().rolling(24 * 7).mean().dropna()
138      c2.plotly_chart(px.line(roll, title="Rolling 7-day MAE on test period", height=350)
139                      .add_hline(y=meta["test_metrics"]["mae"], line_dash="dash")
140                      .update_layout(showlegend=False, yaxis_title="MAE (kW)", xaxis_title=""),
141                      use_container_width=True)
142
143  with tab_ops:
```

```
     st.graphviz_chart("""
     digraph { rankdir=LR; node [shape=box style="rounded,filled" fillcolor="#eef3fa"
fontname=Helvetica];
       A [label="UCI dataset\\n(2.07M rows)"]; B [label="Ingestion &\\nschema check"];
       C [label="Cleaning &\\nhourly resample"]; D [label="Feature\\nengineering"];
       E [label="9 models + GridSearchCV\\n+ MLflow tracking"]; F [label="Model registry\\n(best
model)"];
       G [label="Flask REST API\\n(Docker / K8s / Vercel)"]; H [label="Streamlit\\ndashboard"];
       I [label="Prometheus + Grafana\\nPSI drift, logs"]; J [label="GitHub Actions\\nCI: dvc repro +
tests + build"];
       A->B->C->D->E->F->G->H; G->I; I->E [label="retrain trigger" style=dashed]; J->E
[style=dashed];
     }""")
     st.markdown("""
 | Stage | Tool | Artifact |
 |---|---|---|
 | Versioned pipeline | DVC (`dvc.yaml`, `dvc.lock`) | dataset CSV, processed data, features |
 | Experiment tracking | MLflow (SQLite backend) | runs, params, metrics, registered model |
 | Serving | Flask + gunicorn, Docker, Kubernetes, Vercel | `/api/predict`, `/api/forecast`,
`/api/health` |
 | CI/CD | GitHub Actions, pytest, Vercel | quality gates, test report, Docker image, deployment |
 | Monitoring | Prometheus, Grafana, PSI drift | `/api/metrics`, `logs/predictions.jsonl`,
`reports/monitoring.json` |
 """)
```

web/src/main.jsx: React entry point

```
 1  import React from "react";
 2  import { createRoot } from "react-dom/client";
 3  import App from "./App.jsx";
 4  import "./index.css";
 5
 6  createRoot(document.getElementById("root")).render(
 7    <React.StrictMode>
 8      <App />
 9    </React.StrictMode>
10  );
```

web/src/App.jsx: React application

```
 1  import { useEffect, useMemo, useState } from "react";
 2  import Plot, { COLORS } from "./components/Plot.jsx";
 3  import Forecast from "./components/Forecast.jsx";
 4  import ModelPerformance from "./components/ModelPerformance.jsx";
 5  import PeakAlerts from "./components/PeakAlerts.jsx";
 6  import ResponsibleAI from "./components/ResponsibleAI.jsx";
 7  import Section, { LABEL, hbar } from "./components/Section.jsx";
 8
 9  const REPO = "https://github.com/Kunal039/voltcast";
10  const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
11
12  function Kpis({ D }) {
13    const T = D.model.test_metrics, B = D.model.baseline_test_metrics;
14    const pct = (a, b) => ((1 - a / b) * 100).toFixed(1);
15    const items = [
16      ["Test MAE", `${T.mae.toFixed(3)} kW`, `${pct(T.mae, B.mae)}% better than baseline`],
17      ["Test RMSE", `${T.rmse.toFixed(3)} kW`, `${pct(T.rmse, B.rmse)}% better than baseline`],
18      ["Test R²", T.r2.toFixed(3), `+${(T.r2 - B.r2).toFixed(3)} vs baseline`],
19      ["Training data", `${(D.stats.raw_rows / 1e6).toFixed(2)} M`, "one-minute readings"],
20      ["Model", LABEL[D.model.model_name], `${D.model.n_features} engineered features`],
21    ];
22    return (
23      <div className="kpis">
24        {items.map(([l, v, d]) => (
25          <div className="kpi" key={l}><div className="label">{l}</div><div
className="value">{v}</div><div className="delta">{d}</div></div>
26        ))}
27      </div>
28    );
29  }
30
```

```
function DataExplorer({ D }) {
  const charts = useMemo(() => ({
    daily: [{ x: D.daily.x, y: D.daily.y, line: { color: COLORS.blue, width: 1 }, name: "Daily mean kW" }],
    heat: [{ type: "heatmap", z: D.heatmap, x: [...Array(24).keys()], y: DAYS, colorscale: "YlOrRd",
      reversescale: true, colorbar: { title: "kW", thickness: 10 } }],
    profile: [
      { y: D.profile.weekday, name: "Weekday", line: { color: COLORS.blue } },
      { y: D.profile.weekend, name: "Weekend", line: { color: COLORS.orange } },
    ],
  }), [D]);
  const s = D.stats;
  return (
    <Section title="Data explorer"
      sub={`${s.raw_rows.toLocaleString("en-US")} one-minute readings (${s.raw_missing_pct}% missing) from a house in Sceaux, France, cleaned into ${s.hourly_rows.toLocaleString("en-US")} hourly records.`}>
      <div className="card"><Plot data={charts.daily} title="Daily mean active power, Dec 2006 – Nov 2010" height={300}
        layout={{ xaxis: { type: "date", gridcolor: "#eef0f3" } }} /></div>
      <div className="grid2 gap">
        <div className="card"><Plot data={charts.heat} title="Average kW by weekday and hour"
          layout={{ hovermode: "closest", yaxis: { autorange: "reversed" } }} /></div>
        <div className="card"><Plot data={charts.profile} title="Daily load profile (kW by hour)" /></div>
      </div>
    </Section>
  );
}

function Monitoring({ D }) {
  const M = D.monitoring;
  const charts = useMemo(() => {
    const psi = Object.entries(M.psi).sort((a, b) => a[1] - b[1]);
    return {
      psi: [{ type: "bar", orientation: "h", x: psi.map((p) => p[1]), y: psi.map((p) => p[0]),
        marker: { color: psi.map((p) => (p[1] > 0.2 ? COLORS.orange : COLORS.blue)) } }],
      sub: [{ type: "pie", labels: Object.keys(D.submeter), values: Object.values(D.submeter), hole: 0.45,
        marker: { colors: [COLORS.blue, COLORS.orange, COLORS.green, "#c9ced6"] } }],
    };
  }, [D]); // eslint-disable-line react-hooks/exhaustive-deps
  return (
    <Section title="Monitoring" sub="Data drift between training and recent data is measured with the Population Stability Index (PSI > 0.2 = significant).">
      {M.retraining_recommended && (
        <div className="alert"><b>Retraining recommended.</b> Drift detected in {M.drifted_features.join(", ")}; recent
          30-day MAE is {M.mae_degradation_pct}% above the reference. The training data is winter-heavy while the test
          period is summer, so this is genuine seasonal drift.</div>
      )}
      <div className="grid2">
        <div className="card"><Plot data={charts.psi} title="PSI: training vs test data" layout={{ ...hbar,
          shapes: [{ type: "line", x0: 0.2, x1: 0.2, yref: "paper", y0: 0, y1: 1, line: { color: "#b42318", dash: "dash" } }] }} /></div>
        <div className="card"><Plot data={charts.sub} title="Energy share by sub-meter (kWh)" layout={{ hovermode: "closest" }} /></div>
      </div>
    </Section>
  );
}

function Pipeline() {
  const stack = [
    ["Pipeline & versioning", "dvc.yaml · dvc.lock · Git"], ["Tracking & registry", "MLflow"],
    ["Serving", "Flask REST API · gunicorn"], ["Deployment", "Docker · Kubernetes · Vercel"],
    ["CI / CT", "GitHub Actions · pytest"], ["Monitoring", "Prometheus · Grafana · PSI drift"],
  ];
  return (
    <Section title="MLOps pipeline" sub="Every stage is a tested Python module driven by one config file; GitHub Actions re-runs it on each push and weekly.">
      <div className="card">
        <img className="arch" src="/img/architecture.png"
          alt="Architecture: data ingestion, preprocessing, features, training with MLflow, registry, Flask API, web app, Prometheus and Grafana monitoring and CI/CT" />
        <div className="stack">{stack.map(([t, d]) => <div key={t}><b>{t}</b>{d}</div>)}</div>
      </div>
```

```
      </Section>
    );
  }

  const ENDPOINTS = [
    ["GET", "/api/health", "Liveness check and loaded model"],
    ["GET", "/api/model-info", "Model, hyper-parameters, training window, test metrics"],
    ["GET", "/api/forecast?origin=&horizon=", "Forecast from a point in the test period, with actual values"],
    ["POST", "/api/predict", "Forecast from your own 169+ hourly readings (JSON)"],
    ["GET", "/api/sample-request", "Ready-made body for POST /api/predict"],
    ["GET", "/api/metrics", "Prometheus metrics: requests, latency, forecasts, peak alerts"],
  ];

  function ApiUsage() {
    const host = window.location.origin;
    return (
      <Section id="api" title="REST API (Flask)" sub="The same models are served by a Flask REST API. Prometheus scrapes /api/metrics; Grafana shows the dashboard (see docker-compose.yml).">
        <pre>{`curl "${host}/api/forecast?origin=2010-11-20T08:00&horizon=6"
  curl -s ${host}/api/sample-request > req.json
  curl -X POST ${host}/api/predict -H "Content-Type: application/json" -d @req.json`}</pre>
        <table className="endpoints">
          <tbody>
            {ENDPOINTS.map(([m, path, d]) => <tr key={path}><td><code>{m}</code></td><td><code>{path}</code></td><td>{d}</td></tr>)}
          </tbody>
        </table>
      </Section>
    );
  }

  export default function App() {
    const [D, setD] = useState(null);
    const [error, setError] = useState("");

    useEffect(() => {
      fetch("/data/dashboard.json")
        .then((r) => (r.ok ? r.json() : Promise.reject(new Error(r.statusText))))
        .then(setD)
        .catch((e) => setError(e.message));
    }, []);

    return (
      <>
        <header>
          <div className="wrap">
            <h1>⚡ VoltCast</h1>
            <p className="tagline">Household Electricity Consumption Prediction System</p>
            <p>An end-to-end MLOps project that forecasts a household's hourly electricity use for the next 1–48 hours,
              trained on 2.07 million smart-meter readings from the UCI Individual Household Electric Power Consumption dataset.</p>
            <div className="links">
              <a href="#forecast">Try a forecast</a>
              <a href="#api">REST API</a>
              <a href={REPO}>GitHub repository</a>
            </div>
          </div>
        </header>
        <main className="wrap">
          {error && <div className="alert">Could not load dashboard data: {error}</div>}
          {!D ? <p className="sub loading">Loading dashboard…</p> : (
            <>
              <Kpis D={D} />
              <section id="forecast">
                <h2>Live forecast</h2>
                <p className="sub">Pick a moment in the held-out test period (Jun–Nov 2010). The deployed model forecasts
                  the following hours, and the chart shows what actually happened.</p>
                <Forecast range={D.forecast_range} />
              </section>
              <ModelPerformance D={D} />
              <PeakAlerts D={D} />
              <DataExplorer D={D} />
              <Monitoring D={D} />
              <ResponsibleAI D={D} />
              <Pipeline />
              <ApiUsage />
```

```
 169           </>
 170         )}
 171       </main>
 172       <footer>
 173         <div className="wrap">VoltCast · MLOps project · Chandigarh University · Kunal Mittal, Ansh
Kumar, Vishal Kumar Shrivastava ·
 174           Data: G. Hebrail &amp;amp; A. Berard, UCI Machine Learning Repository (CC BY 4.0)</div>
 175       </footer>
 176     </>
 177   );
 178 }
```

web/src/components/Plot.jsx: Plotly wrapper component

```
   1 import { useEffect, useRef } from "react";
   2 // Cartesian build covers every trace type used here (scatter, bar, heatmap, pie) at about a quarter
of the size.
   3 import Plotly from "plotly.js-cartesian-dist-min";
   4
   5 export const COLORS = { blue: "#1f5fa8", orange: "#e07b24", grey: "#9aa3af", green: "#2f8f5b" };
   6
   7 const BASE = {
   8   margin: { t: 36, r: 12, b: 64, l: 50 },
   9   paper_bgcolor: "rgba(0,0,0,0)",
  10   plot_bgcolor: "rgba(0,0,0,0)",
  11   font: { family: "Inter, sans-serif", size: 12, color: "#14181f" },
  12   hovermode: "x unified",
  13   legend: { orientation: "h", y: -0.16, x: 0 },
  14   xaxis: { gridcolor: "#eef0f3" },
  15   yaxis: { gridcolor: "#eef0f3", zeroline: false },
  16 };
  17
  18 /** Thin React wrapper around Plotly; `layout` is merged over the shared base style. */
  19 export default function Plot({ data, title = "", layout = {}, height = 340 }) {
  20   const ref = useRef(null);
  21
  22   useEffect(() => {
  23     // Plotly writes inferred axis types back into the layout, so each render gets a fresh copy.
  24     const full = { ...structuredClone(BASE), title: { text: title, font: { size: 14 }, x: 0 },
...layout };
  25     Plotly.react(ref.current, data, full, { displayModeBar: false, responsive: true });
  26   }, [data, title, layout]);
  27
  28   useEffect(() => {
  29     const el = ref.current;
  30     return () => Plotly.purge(el);
  31   }, []);
  32
  33   return <div ref={ref} style={{ width: "100%", height }} />;
  34 }
```

web/src/components/Section.jsx: shared section component

```
   1 export const LABEL = {
   2   persistence_baseline: "Persistence baseline", seasonal_naive_24h: "Seasonal naive (24 h)",
   3   linear_regression: "Linear regression", ridge_regression: "Ridge regression", decision_tree:
"Decision tree",
   4   knn: "K-nearest neighbours", svm: "SVM (SVR)", random_forest: "Random forest",
hist_gradient_boosting: "Gradient boosting",
   5 };
   6
   7 export const isBaseline = (n) => /baseline|naive/.test(n);
   8
   9 export const hbar = { margin: { t: 36, r: 12, b: 40, l: 160 }, hovermode: "closest",
  10   xaxis: { type: "linear", gridcolor: "#eef0f3" }, yaxis: { type: "category", automargin: true } };
  11
  12 export default function Section({ id, title, sub, children }) {
  13   return (
  14     <section id={id}>
  15       <h2>{title}</h2>
  16       {sub && <p className="sub">{sub}</p>}
```

```
      {children}
    </section>
  );
}
```

web/src/components/Forecast.jsx: live forecast component

```
import { useCallback, useEffect, useMemo, useState } from "react";
import Plot, { COLORS } from "./Plot.jsx";

const pad = (h) => String(h).padStart(2, "0");

export default function Forecast({ range }) {
  const [lo, hi] = range.map((s) => s.slice(0, 10));
  const [day, setDay] = useState("2010-11-20");
  const [hour, setHour] = useState(8);
  const [horizon, setHorizon] = useState(24);
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const origin = `${day}T${pad(hour)}:00`;

  const run = useCallback(async () => {
    setLoading(true);
    setError("");
    try {
      const r = await fetch(`/api/forecast?origin=${origin}&horizon=${horizon}`);
      const body = await r.json();
      if (!r.ok) throw new Error(body.detail || r.statusText);
      setResult({ ...body, origin });
    } catch (e) {
      setError(`Forecast failed: ${e.message}`);
    } finally {
      setLoading(false);
    }
  }, [origin, horizon]);

  // Show a forecast as soon as the page opens.
  useEffect(() => { run(); }, []); // eslint-disable-line react-hooks/exhaustive-deps

  const chart = useMemo(() => {
    if (!result) return null;
    const f = result.forecast;
    const xs = f.map((p) => p.timestamp);
    return {
      data: [
        { x: [...xs, ...[...xs].reverse()], y: [...f.map((p) => p.upper_kw), ...f.map((p) =>
p.lower_kw).reverse()],
          fill: "toself", fillcolor: "rgba(224,123,36,.15)", line: { width: 0 }, name: "95%
interval", hoverinfo: "skip" },
        { x: result.history.map((p) => p.timestamp), y: result.history.map((p) => p.kw), name:
"History",
          line: { color: COLORS.blue, width: 1.6 } },
        { x: xs, y: f.map((p) => p.actual_kw), name: "Actual", line: { color: "#6b7280", dash:
"dot", width: 1.6 } },
        { x: xs, y: f.map((p) => p.predicted_kw), name: "Forecast", line: { color: COLORS.orange,
width: 3 } },
        { x: f.filter((p) => p.peak_alert).map((p) => p.timestamp), y: f.filter((p) =>
p.peak_alert).map((p) => p.predicted_kw),
          name: "Peak-hour alert", mode: "markers", marker: { color: "#b42318", size: 9, symbol:
"triangle-up" },
          customdata: f.filter((p) => p.peak_alert).map((p) => (p.alert_method === "classifier"
            ? `classifier: ${(p.peak_probability * 100).toFixed(0)}% peak probability` : "forecast
above peak threshold")),
          hovertemplate: "Peak-hour alert<br>%{customdata}<extra></extra>" },
      ],
      layout: {
        margin: { t: 20, r: 12, b: 64, l: 50 },
        xaxis: { type: "date", gridcolor: "#eef0f3" },
        yaxis: { title: "kW", gridcolor: "#eef0f3" },
        shapes: [{ type: "line", x0: result.origin, x1: result.origin, yref: "paper", y0: 0, y1: 1,
          line: { dash: "dash", color: "#14181f" } }],
      },
    };
```

```
  61    }, [result]);
  62
  63    const stats = useMemo(() => {
  64      if (!result) return [];
  65      const f = result.forecast;
  66      const peak = f.reduce((a, b) => (a.predicted_kw > b.predicted_kw ? a : b));
  67      const paired = f.filter((p) => p.actual_kw !== null);
  68      const mae = paired.reduce((s, p) => s + Math.abs(p.actual_kw - p.predicted_kw), 0) /
paired.length;
  69      const when = new Date(peak.timestamp).toLocaleString("en-GB", { weekday: "short", hour: "2-
digit", minute: "2-digit" });
  70      return [
  71        [`${result.predicted_energy_kwh.toFixed(2)} kWh`, "predicted energy"],
  72        [`${when} · ${peak.predicted_kw.toFixed(2)} kW`, "predicted peak"],
  73        [paired.length ? `${mae.toFixed(3)} kW` : "–", "MAE vs actual"],
  74        [`${f.filter((p) => p.peak_alert).length} of ${f.length} hours`, "peak-hour alerts"],
  75      ];
  76    }, [result]);
  77
  78    return (
  79      <div className="card fc">
  80        <div>
  81          <label htmlFor="day">Forecast from</label>
  82          <input type="date" id="day" min={lo} max={hi} value={day} onChange={(e) =>
setDay(e.target.value)} />
  83          <label htmlFor="hour">Starting hour: <b>{pad(hour)}:00</b></label>
  84          <input type="range" id="hour" min="0" max="23" value={hour} onChange={(e) =>
setHour(+e.target.value)} />
  85          <label htmlFor="horizon">Hours ahead: <b>{horizon}</b></label>
  86          <input type="range" id="horizon" min="1" max="48" value={horizon} onChange={(e) =>
setHorizon(+e.target.value)} />
  87          <button onClick={run} disabled={loading}>{loading ? "Forecasting…" : "Run
forecast"}</button>
  88          {error && <div className="err">{error}</div>}
  89          <div className="stats">
  90            {stats.map(([v, s]) => <div className="stat" key={s}><b>{v}</b><span>{s}</span></div>)}
  91          </div>
  92        </div>
  93        {chart ? <Plot data={chart.data} layout={chart.layout} height={420} /> : <div
className="placeholder">Loading forecast…</div>}
  94      </div>
  95    );
  96  }
```

web/src/components/ModelPerformance.jsx: model performance component

```
   1  import { useMemo } from "react";
   2  import Plot, { COLORS } from "./Plot.jsx";
   3  import Section, { LABEL, hbar, isBaseline } from "./Section.jsx";
   4
   5  const grouped = { hovermode: "closest", barmode: "group", xaxis: { tickangle: -25 },
   6    margin: { t: 36, r: 12, b: 90, l: 50 }, legend: { orientation: "h", y: 1.1, x: 0 } };
   7
   8  export default function ModelPerformance({ D }) {
   9    const V = D.metrics.validation;
  10    const tuning = D.metrics.tuning;
  11    const names = Object.keys(V);
  12    const ml = names.filter((n) => !isBaseline(n));
  13    const best = D.model.model_name; // the deployed model (best syllabus algorithm on validation)
  14    const dt = tuning.decision_tree;
  15
  16    const charts = useMemo(() => {
  17      const tn = Object.keys(tuning);
  18      return {
  19        fit: [
  20          { type: "bar", x: ml.map((n) => LABEL[n]), y: ml.map((n) => V[n].train_mae), name: "Training
MAE", marker: { color: COLORS.grey } },
  21          { type: "bar", x: ml.map((n) => LABEL[n]), y: ml.map((n) => V[n].mae), name: "Validation
MAE", marker: { color: COLORS.blue } },
  22        ],
  23        tuning: [
  24          { type: "bar", x: tn.map((n) => LABEL[n]), y: tn.map((n) => tuning[n].default_val.mae),
name: "Default", marker: { color: COLORS.grey } },
  25          { type: "bar", x: tn.map((n) => LABEL[n]), y: tn.map((n) => tuning[n].tuned_val.mae), name:
"Tuned (GridSearchCV)", marker: { color: COLORS.orange } },
```

```
  26        ],
  27        test: [
  28          { x: D.test_window.x, y: D.test_window.actual, name: "Actual", line: { color: COLORS.grey,
width: 1.5 } },
  29          { x: D.test_window.x, y: D.test_window.predicted, name: "Predicted (next hour)", line: {
color: COLORS.orange, width: 1.5 } },
  30        ],
  31        imp: [{ type: "bar", orientation: "h", x: D.importance.values, y: D.importance.names, marker:
{ color: COLORS.blue } }],
  32      };
  33    }, [D]); // eslint-disable-line react-hooks/exhaustive-deps
  34
  35    return (
  36      <Section title="Model performance"
  37        sub="Nine models trained on 2006–2009 and compared on Jan–May 2010 in MLflow. Only syllabus
algorithms (linear and ridge regression, decision tree, KNN, SVM, random forest) may be deployed; gradient
boosting and two naive baselines are kept as reference points. The deployed model (highlighted) is refit
and scored once on the untouched test period.">
  38        <div className="grid2">
  39          <div className="card">
  40            <table>
  41              <thead><tr><th>Model</th><th>Train MAE</th><th>Val
MAE</th><th>RMSE</th><th>R²</th></tr></thead>
  42              <tbody>
  43                {names.map((n) => (
  44                  <tr key={n} className={n === best ? "best" : ""}>
  45
<td>{LABEL[n]}</td><td>{V[n].train_mae.toFixed(3)}</td><td>{V[n].mae.toFixed(3)}</td>
  46                    <td>{V[n].rmse.toFixed(3)}</td><td>{V[n].r2.toFixed(3)}</td>
  47                  </tr>
  48                ))}
  49              </tbody>
  50            </table>
  51          </div>
  52          <div className="card">
  53            <Plot data={charts.fit} title="Over-fitting check: training vs validation MAE (kW)"
height={340} layout={grouped} />
  54            <p className="note">The default decision tree has zero training error but the worst
validation error: it memorises the training set.</p>
  55          </div>
  56        </div>
  57        <div className="grid2 gap">
  58          <div className="card">
  59            <Plot data={charts.tuning} title="Tuning with GridSearchCV (time-series folds)"
height={340} layout={grouped} />
  60            <p className="note">Limiting the tree (max_depth {String(dt.best_params.max_depth)},
min_samples_leaf {dt.best_params.min_samples_leaf})
  61              cuts its validation MAE from {dt.default_val.mae.toFixed(3)} to
{dt.tuned_val.mae.toFixed(3)} kW.</p>
  62          </div>
  63          <div className="card"><Plot data={charts.imp} title="Top features (permutation importance)"
layout={hbar} /></div>
  64        </div>
  65        <div className="card gap">
  66          <Plot data={charts.test} title="Test set: last 14 days, next-hour forecasts" layout={{
xaxis: { type: "date", gridcolor: "#eef0f3" } }} />
  67        </div>
  68      </Section>
  69    );
  70  }
```

web/src/components/PeakAlerts.jsx: classifier component

```
   1  import { useMemo } from "react";
   2  import Plot, { COLORS } from "./Plot.jsx";
   3  import Section from "./Section.jsx";
   4
   5  export default function PeakAlerts({ D }) {
   6    const C = D.classification;
   7    const T = C.test;
   8    const [tn, fp] = T.confusion_matrix[0];
   9    const [fn, tp] = T.confusion_matrix[1];
  10
  11    const charts = useMemo(() => ({
  12      cm: [{
```

```
  13        type: "heatmap", z: [[fn, tp], [tn, fp]], x: ["Predicted normal", "Predicted peak"], y:
["Actual peak", "Actual normal"],
  14        colorscale: [[0, "#f1f5fb"], [1, "#7fa6d6"]], showscale: false, text: [[fn, tp], [tn,
fp]].map((r) => r.map((v) => v.toLocaleString("en-US"))),
  15        texttemplate: "%{text}", textfont: { size: 16, color: "#14181f" },
  16      }],
  17      roc: [
  18        { x: C.roc.fpr, y: C.roc.tpr, name: `Tuned tree, AUC ${T.roc_auc.toFixed(3)}`, line: { color:
COLORS.orange, width: 2.5 } },
  19        { x: [0, 1], y: [0, 1], name: "Random guess", line: { color: COLORS.grey, dash: "dash" } },
  20      ],
  21    }), [D]); // eslint-disable-line react-hooks/exhaustive-deps
  22
  23    const kpis = [["Accuracy", T.accuracy], ["Precision", T.precision], ["Recall", T.recall], ["F1-
score", T.f1], ["ROC-AUC", T.roc_auc]];
  24    return (
  25      <Section id="peaks" title="Peak-hour alerts (classification)"
  26        sub={`A second model flags hours above ${C.threshold_kw.toFixed(2)} kW (the busiest 25% of
training hours). A decision tree tuned with GridSearchCV beat its default version and logistic regression
on validation F1; these are its scores on the held-out test period.`}>
  27        <div className="kpis inline">
  28          {kpis.map(([l, v]) => (
  29            <div className="kpi" key={l}><div className="label">{l}</div><div className="value">{(v *
100).toFixed(1)}%</div></div>
  30          ))}
  31        </div>
  32        <div className="grid2 gap">
  33          <div className="card">
  34            <Plot data={charts.cm} title="Confusion matrix (test)" height={320}
  35              layout={{ hovermode: "closest", margin: { t: 36, r: 12, b: 50, l: 100 } }} />
  36          </div>
  37          <div className="card">
  38            <Plot data={charts.roc} title="ROC curve (test)" height={320}
  39              layout={{ hovermode: "closest", legend: { x: 0.35, y: 0.08, bgcolor:
"rgba(255,255,255,.8)" },
  40                xaxis: { title: "False positive rate", range: [0, 1] },
  41                yaxis: { title: "True positive rate", range: [0, 1.02] } }} />
  42          </div>
  43        </div>
  44        <p className="note">High recall, moderate precision: the alert catches {(T.recall *
100).toFixed(0)}% of real peak hours at the cost
  45          of some false alarms, the safer trade-off when the goal is to avoid unexpected peaks. For
comparison, flagging hours where the
  46          regression forecast exceeds the threshold is more accurate
({(C.forecast_threshold_rule_test.accuracy * 100).toFixed(1)}%) but catches only{" "}
  47          {(C.forecast_threshold_rule_test.recall * 100).toFixed(0)}% of peaks. The live forecast uses
the classifier for the next hour and that rule for
  48          later hours, where the classifier's inputs are no longer up to date.</p>
  49      </Section>
  50    );
  51  }
```

web/src/components/ResponsibleAI.jsx: responsible-AI component

```
   1  import { useMemo } from "react";
   2  import Plot, { COLORS } from "./Plot.jsx";
   3  import Section from "./Section.jsx";
   4
   5  const POINTS = [
   6    ["Fairness & bias", "Error is compared across groups of hours (chart). Forecasts are weakest in
evening peaks and high-use hours, and the model learned from one French household, so it must be re-
validated before use on other homes or climates."],
   7    ["Transparency", "Feature importance, a model card, and numbers anyone can reproduce with dvc
repro. Forecasts carry a 95% interval instead of a single figure."],
   8    ["Privacy", "Meter data can reveal when people are home. Only a public anonymised dataset is used,
and the prediction log stores forecasts, never the readings sent in."],
   9    ["Security", "Every reading is validated, request size is limited to 2 MB, and rejected requests
are counted in Prometheus so misuse shows up in Grafana."],
  10  ];
  11
  12  export default function ResponsibleAI({ D }) {
  13    const F = D.fairness;
  14    const { data, layout } = useMemo(() => {
  15      const groups = Object.entries(F.slices);
  16      return {
```

```
        data: groups.map(([name, g], i) => ({
          type: "bar", x: Object.keys(g), y: Object.values(g).map((v) => v.mae), name,
          xaxis: `x${i + 1}`, yaxis: `y${i + 1}`, marker: { color: COLORS.blue }, showlegend: false,
        })),
        layout: {
          hovermode: "closest",
          grid: { rows: 1, columns: groups.length, pattern: "independent", xgap: 0.12 },
          margin: { t: 40, r: 12, b: 90, l: 50 },
          annotations: groups.map(([name], i) => ({
            text: `<b>${name}</b>`, xref: `x${i + 1} domain`, yref: `y${i + 1} domain`, x: 0.5, y:
1.13, showarrow: false,
          })),
          shapes: groups.map((_, i) => ({
            type: "line", xref: `x${i + 1} domain`, yref: `y${i + 1}`, x0: 0, x1: 1, y0:
F.overall_mae, y1: F.overall_mae,
            line: { color: COLORS.orange, dash: "dash" },
          })),
          ...Object.fromEntries(groups.flatMap((_, i) => [
            [`xaxis${i ? i + 1 : ""}`, { tickangle: -30, gridcolor: "#eef0f3" }],
            [`yaxis${i ? i + 1 : ""}`, { range: [0, 0.5], gridcolor: "#eef0f3", showticklabels: i ===
0, title: i === 0 ? "MAE (kW)" : "" }],
          ])),
        },
      };
    }, [D]); // eslint-disable-line react-hooks/exhaustive-deps

    return (
      <Section id="ethics" title="Responsible AI" sub="Ethical considerations from the course:
fairness, transparency, privacy and security.">
        <div className="card">
          <Plot data={data} layout={layout} height={330} />
          <p className="note">Test MAE for each group of hours. Dashed line: overall MAE
({F.overall_mae.toFixed(3)} kW).</p>
        </div>
        <div className="stack gap">{POINTS.map(([t, d]) => <div key={t}><b>{t}</b>{d}</div>)}</div>
      </Section>
    );
  }
```

web/src/index.css: styles

```
  :root {
    --bg: #f6f7f9; --card: #ffffff; --ink: #14181f; --muted: #5d6675; --line: #e3e6eb;
    --orange: #e07b24; --green: #2f8f5b; --amber-bg: #fff6e6; --amber: #9a5b00;
    --radius: 14px; --shadow: 0 1px 2px rgba(16,24,40,.05), 0 1px 3px rgba(16,24,40,.06);
  }
  * { box-sizing: border-box; }
  body { margin: 0; background: var(--bg); color: var(--ink); font: 15px/1.55 Inter, system-ui, sans-
serif; }
  a { color: #1f5fa8; }
  .wrap { max-width: 1180px; margin: 0 auto; padding: 0 16px; }
  header { background: linear-gradient(135deg, #0f2f57 0%, #1f5fa8 60%, #2b74c4 100%); color: #fff;
padding: 44px 0 36px; }
  header h1 { margin: 0 0 8px; font-size: clamp(26px, 4vw, 38px); letter-spacing: -.02em; }
  header p { margin: 0; color: #d6e4f5; max-width: 760px; }
  header .tagline { color: #fff; font-size: 18px; font-weight: 600; margin: -4px 0 10px; }
  .links { margin-top: 18px; display: flex; flex-wrap: wrap; gap: 10px; }
  .links a { color: #fff; text-decoration: none; border: 1px solid rgba(255,255,255,.35); padding: 7px
14px; border-radius: 999px; font-size: 14px; }
  .links a:hover { background: rgba(255,255,255,.12); }
  .kpis { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px, 1fr)); gap: 12px;
margin-top: -22px; }
  .kpi { background: var(--card); border-radius: var(--radius); box-shadow: var(--shadow); padding:
16px 18px; }
  .kpi .label { color: var(--muted); font-size: 13px; }
  .kpi .value { font-size: 26px; font-weight: 700; letter-spacing: -.02em; }
  .kpi .delta { font-size: 12.5px; color: var(--green); font-weight: 600; }
  section { margin: 34px 0; }
  h2 { font-size: 21px; margin: 0 0 4px; letter-spacing: -.01em; }
  .sub { color: var(--muted); margin: 0 0 14px; font-size: 14px; }
  .loading { margin-top: 40px; }
  .card { background: var(--card); border-radius: var(--radius); box-shadow: var(--shadow); padding:
18px; min-width: 0; }
  .grid2 { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
  .gap { margin-top: 16px; }
```

```
  .fc { display: grid; grid-template-columns: 260px 1fr; gap: 18px; }
  .fc > *, .grid2 > * { min-width: 0; }
  .placeholder { display: grid; place-items: center; color: var(--muted); min-height: 420px; }
  label { display: block; font-size: 13px; color: var(--muted); margin: 12px 0 4px; }
  input { width: 100%; padding: 9px 10px; border: 1px solid var(--line); border-radius: 9px; font:
inherit; background: #fff; }
  input[type=range] { padding: 0; accent-color: var(--orange); }
  button { margin-top: 16px; width: 100%; padding: 11px; border: 0; border-radius: 9px; background:
var(--orange); color: #fff; font: 600 15px Inter, sans-serif; cursor: pointer; }
  button:disabled { opacity: .6; cursor: wait; }
  .stats { margin-top: 16px; display: grid; gap: 10px; }
  .stat { border-top: 1px solid var(--line); padding-top: 8px; }
  .stat b { display: block; font-size: 20px; }
  .stat span { color: var(--muted); font-size: 12.5px; }
  table { width: 100%; border-collapse: collapse; font-size: 14px; }
  th, td { text-align: right; padding: 8px 10px; border-bottom: 1px solid var(--line); }
  th:first-child, td:first-child { text-align: left; }
  th { color: var(--muted); font-weight: 600; font-size: 12.5px; text-transform: uppercase; letter-
spacing: .04em; }
  tr.best td { background: #eef6f0; font-weight: 600; }
  .alert { background: var(--amber-bg); color: var(--amber); border-radius: 10px; padding: 12px 14px;
margin-bottom: 14px; font-size: 14px; }
  .arch { width: 100%; height: auto; display: block; }
  .stack { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px, 1fr)); gap: 12px;
margin-top: 14px; }
  .stack div { border: 1px solid var(--line); border-radius: 10px; padding: 12px; font-size: 14px; }
  .stack b { display: block; margin-bottom: 2px; }
  pre { background: #10151d; color: #dfe6ef; padding: 14px; border-radius: 10px; overflow-x: auto;
font: 12.5px/1.5 "JetBrains Mono", monospace; }
  .err { color: #b42318; font-size: 13px; margin-top: 8px; }
  footer { color: var(--muted); font-size: 13px; padding: 26px 0 40px; border-top: 1px solid var(---
line); margin-top: 40px; }
  @media (max-width: 820px) { .grid2, .fc { grid-template-columns: 1fr; } }
  .kpis.inline { margin-top: 0; }
  .note { color: var(--muted); font-size: 13px; margin: 8px 0 0; }
  .endpoints td { text-align: left; vertical-align: top; }
  .endpoints code { white-space: nowrap; }
```

Dockerfile: container image

```
  # Stage 1: build the React web app into /public
  FROM node:20-alpine AS web
  WORKDIR /web
  COPY web/package.json web/package-lock.json ./
  RUN npm ci
  COPY web/ ./
  RUN npm run build

  # Stage 2: Python runtime with the Flask API, Streamlit dashboard and MLflow UI
  FROM python:3.12-slim
  WORKDIR /app
  ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 VOLTCAST_LOG_DIR=/app/logs

  COPY requirements.txt requirements-dev.txt ./
  RUN pip install --no-cache-dir -r requirements-dev.txt

  COPY configs/ configs/
  COPY src/ src/
  COPY serving/ serving/
  COPY app.py .
  COPY dashboard/ dashboard/
  COPY .streamlit/ .streamlit/
  COPY models/ models/
  COPY reports/ reports/
  COPY data/processed/hourly.parquet data/processed/hourly.parquet
  COPY data/serving/ data/serving/
  COPY --from=web /public public/

  EXPOSE 8000 8501 5000
  HEALTHCHECK CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/api/health')" || exit 1
  # One worker keeps Prometheus counters in a single process; threads handle concurrent requests.
  CMD ["gunicorn", "--workers", "1", "--threads", "4", "--bind", "0.0.0.0:8000", "app:app"]
```

docker-compose.yml: multi-service stack

```
  1  # Full local stack: API + web app, Streamlit dashboard, MLflow UI, Prometheus and Grafana.
  2  #   docker compose up --build
  3  #   web + API  http://localhost:8000      Streamlit  http://localhost:8501
  4  #   MLflow     http://localhost:5000      Prometheus http://localhost:9090
  5  #   Grafana    http://localhost:3000  (dashboard "VoltCast - Model Monitoring", anonymous viewer
access)
  6  services:
  7    api:
  8      build: .
  9      image: voltcast:latest
 10      ports: ["8000:8000"]
 11      volumes: ["./logs:/app/logs"]          # prediction log: logs/predictions.jsonl
 12
 13    dashboard:
 14      image: voltcast:latest
 15      command: streamlit run dashboard/app.py --server.port 8501 --server.address 0.0.0.0
 16      ports: ["8501:8501"]
 17      depends_on: [api]
 18
 19    mlflow:
 20      image: voltcast:latest
 21      command: mlflow ui --backend-store-uri sqlite:///mlflow.db --host 0.0.0.0 --port 5000
 22      volumes: ["./mlflow.db:/app/mlflow.db", "./mlruns:/app/mlruns"]
 23      ports: ["5000:5000"]
 24      depends_on: [api]
 25
 26    prometheus:
 27      image: prom/prometheus:v2.53.2
 28      volumes: ["./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
 29      ports: ["9090:9090"]
 30      depends_on: [api]
 31
 32    grafana:
 33      image: grafana/grafana:11.2.0
 34      environment:
 35        GF_AUTH_ANONYMOUS_ENABLED: "true"
 36        GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
 37        GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/voltcast.json
 38      volumes:
 39        - ./monitoring/grafana/provisioning:/etc/grafana/provisioning:ro
 40        - ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards:ro
 41      ports: ["3000:3000"]
 42      depends_on: [prometheus]
```

monitoring/prometheus.yml: Prometheus scrape configuration

```
  1  global:
  2    scrape_interval: 15s
  3
  4  scrape_configs:
  5    - job_name: voltcast-api
  6      metrics_path: /api/metrics
  7      static_configs:
  8        - targets: ["api:8000"]
```

monitoring/grafana/provisioning/datasources/prometheus.yml: Grafana data source

```
  1  apiVersion: 1
  2  datasources:
  3    - name: Prometheus
  4      uid: prometheus
  5      type: prometheus
  6      access: proxy
  7      url: http://prometheus:9090
  8      isDefault: true
```

k8s/deployment.yaml: Kubernetes manifests

```yaml
# Kubernetes deployment of the VoltCast API (same image as docker-compose).
# Build and load the image first, e.g. with minikube:
#   docker build -t voltcast:latest . && minikube image load voltcast:latest
#   kubectl apply -f k8s/
apiVersion: apps/v1
kind: Deployment
metadata:
  name: voltcast-api
  labels: {app: voltcast-api}
spec:
  replicas: 2
  selector:
    matchLabels: {app: voltcast-api}
  template:
    metadata:
      labels: {app: voltcast-api}
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path: /api/metrics
        prometheus.io/port: "8000"
    spec:
      containers:
        - name: api
          image: voltcast:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          resources:
            requests: {cpu: 250m, memory: 512Mi}
            limits: {cpu: "1", memory: 1Gi}
          readinessProbe:
            httpGet: {path: /api/health, port: 8000}
            initialDelaySeconds: 10
            periodSeconds: 10
          livenessProbe:
            httpGet: {path: /api/health, port: 8000}
            initialDelaySeconds: 20
            periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: voltcast-api
spec:
  type: NodePort
  selector: {app: voltcast-api}
  ports:
    - port: 80
      targetPort: 8000
      nodePort: 30080
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: voltcast-api
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: voltcast-api}
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 70}
```

.github/workflows/ci.yml: CI/CD workflow

```yaml
name: CI / CT pipeline

on:
  push:
    branches: [main]
```

```
  pull_request:
  schedule:
    - cron: "0 2 * * 1"   # weekly continuous-training run
  workflow_dispatch:

jobs:
  train-test-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements-dev.txt

      - name: Reproduce the DVC pipeline (ingest -> train -> classify -> monitor -> evaluate ->
export)
        run: dvc repro --force
      - name: Show metrics tracked by DVC
        run: dvc metrics show

      - name: Quality gate - forecaster must beat the persistence baseline
        run: |
          python -c "import json,sys; m=json.load(open('models/metadata.json')); \
          sys.exit(0 if m['test_metrics']['mae'] < m['baseline_test_metrics']['mae'] else 1)"
      - name: Quality gate - peak classifier must beat chance (ROC-AUC > 0.75)
        run: |
          python -c "import json,sys; c=json.load(open('reports/classification.json')); \
          sys.exit(0 if c['test']['roc_auc'] > 0.75 else 1)"

      - name: Unit and API tests
        run: pytest -q

      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Build React web app (into public/, used by Docker and Vercel)
        run: cd web && npm ci && npm run build

      - name: Validate Docker Compose, Prometheus/Grafana and Kubernetes configs
        run: |
          docker compose config -q
          python -c "import yaml,glob; [list(yaml.safe_load_all(open(f))) for f in
glob.glob('k8s/*.yaml') + glob.glob('monitoring/**/*.yml', recursive=True)]"
      - name: Build Docker image
        run: docker build -t voltcast:${{ github.sha }} .

      - uses: actions/upload-artifact@v4
        with:
          name: model-and-reports
          path: |
            models/
            reports/
            dvc.lock
```

vercel.json: Vercel build configuration

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": "cd web && npm ci && npm run build",
  "functions": {
    "app.py": {
      "includeFiles": "public/**"
    }
  }
}
```

requirements.txt: serving dependencies

```
# Runtime dependencies of the prediction service (what Vercel / Docker serving installs).
```

```
  2  # scikit-learn is pinned to the version the model was trained with, so the pickled model loads
safely.
  3  scikit-learn==1.9.1
  4  numpy>=2.0
  5  pandas>=2.2
  6  joblib>=1.4
  7  pyyaml>=6
  8  flask>=3.0
  9  prometheus_client>=0.20
 10  pydantic>=2.6
```

requirements-dev.txt: development dependencies

```
  1  # Everything needed to run the full pipeline, dashboard and tests locally or in CI.
  2  -r requirements.txt
  3  pyarrow>=15
  4  requests>=2.31
  5  mlflow>=2.15
  6  matplotlib>=3.8
  7  seaborn>=0.13
  8  gunicorn>=22; sys_platform != "win32"
  9  streamlit>=1.35
 10  plotly>=5.20
 11  pytest>=8
 12  dvc>=3.50
```

USER MANUAL

Hosted version (no installation): open https://voltcast-forecast.vercel.app for the web application and live forecasts. The API is at https://voltcast-forecast.vercel.app/api (for example /api/health). Source code: https://github.com/Kunal039/voltcast.

Prerequisites: Python 3.10 or newer, Node.js 20, about 1 GB free disk space, and internet access for the first run (dataset download, ~20 MB). Docker Desktop is needed for the monitoring stack.

Step 1: Get the project.

git clone https://github.com/Kunal039/voltcast.git then open a terminal inside the folder.

Step 2: Install dependencies.

pip install -r requirements-dev.txt

Step 3: Reproduce the pipeline.

dvc repro (downloads the dataset as CSV, preprocesses, builds features, trains and tunes all models, trains the peak classifier, runs monitoring and creates all figures; about 4 minutes). dvc metrics show prints the tracked metrics.

Step 4: Run the tests.

pytest -q (11 tests should pass)

Step 5: Inspect experiments in MLflow.

mlflow ui --backend-store-uri sqlite:///mlflow.db then open http://localhost:5000 to compare runs and see the registered model voltcast-forecaster.

Step 6: Start the web app and Flask API.

cd web && npm install && npm run build && cd .. then python app.py and open http://localhost:8000 (Figure M.1). Choose a date, starting hour and horizon and click Run forecast. The chart shows history, forecast, actual values, the 95% interval and peak-hour alerts.

Step 7: Call the API.

curl http://localhost:8000/api/sample-request > req.json then curl -X POST -H "Content-Type: application/json" -d @req.json http://localhost:8000/api/predict

Step 8: Run the full stack with monitoring.

docker compose up --build starts the API and web app (8000), Streamlit (8501), MLflow (5000), Prometheus (9090) and Grafana (3000). Open Grafana and select the dashboard "VoltCast – Model Monitoring" (Figures M.4–M.5).

Step 9: Deploy on Kubernetes (optional).

docker build -t voltcast:latest . then load the image into the cluster (e.g. minikube image load voltcast:latest) and run kubectl apply -f k8s/

Step 10: Retrain with new settings.

Edit configs/params.yaml (e.g. a hyper-parameter grid or split dates) and run dvc repro. Only affected stages re-run; a new MLflow run and model version are created.

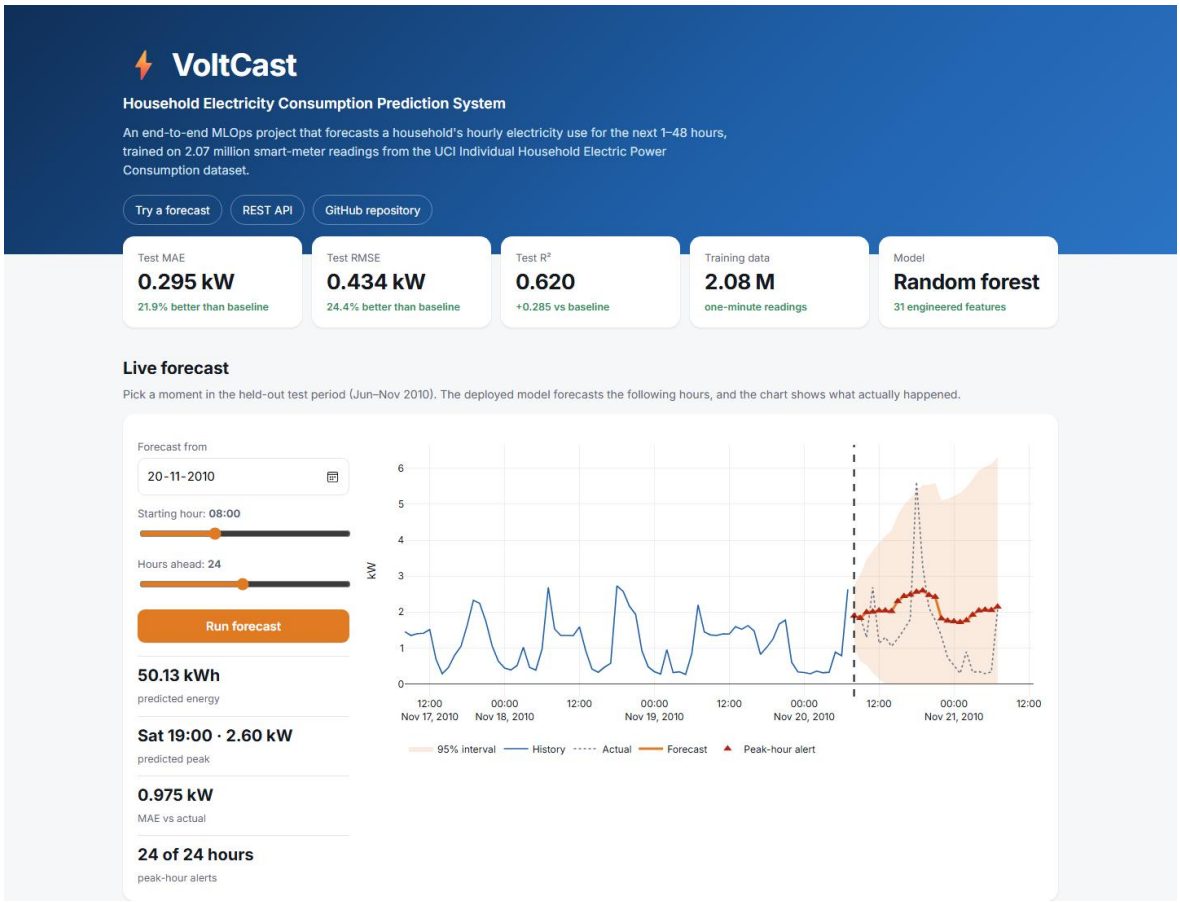


Figure M.1*: Web application – live forecast*



Figure M.2: *Web application – model performance, over-fitting and tuning*

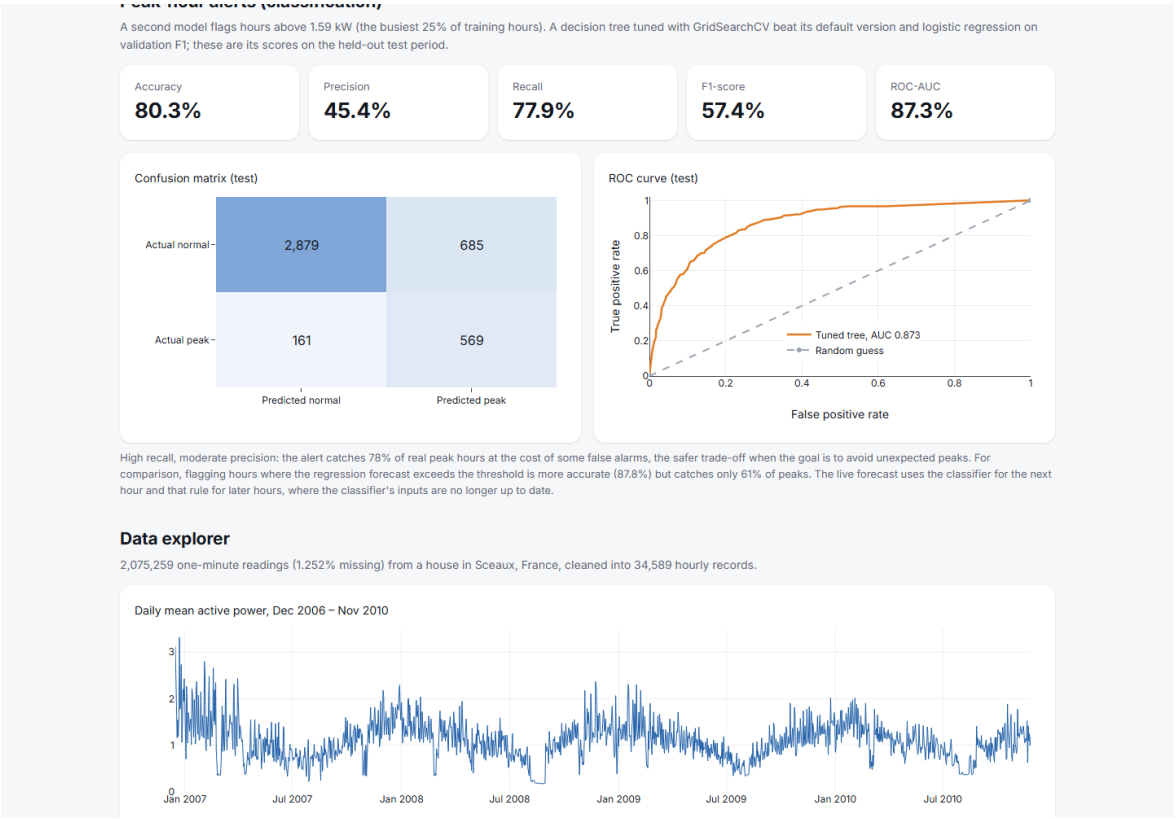


Figure M.3*: Web application – peak-hour classifier metrics*



Figure M.4*: Grafana monitoring dashboard*

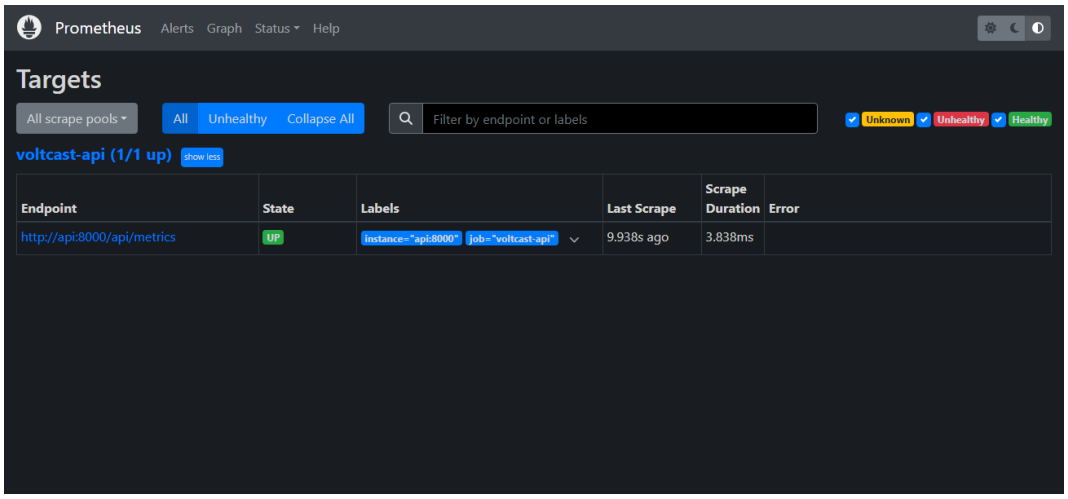


Figure M.5*: Prometheus targets page*

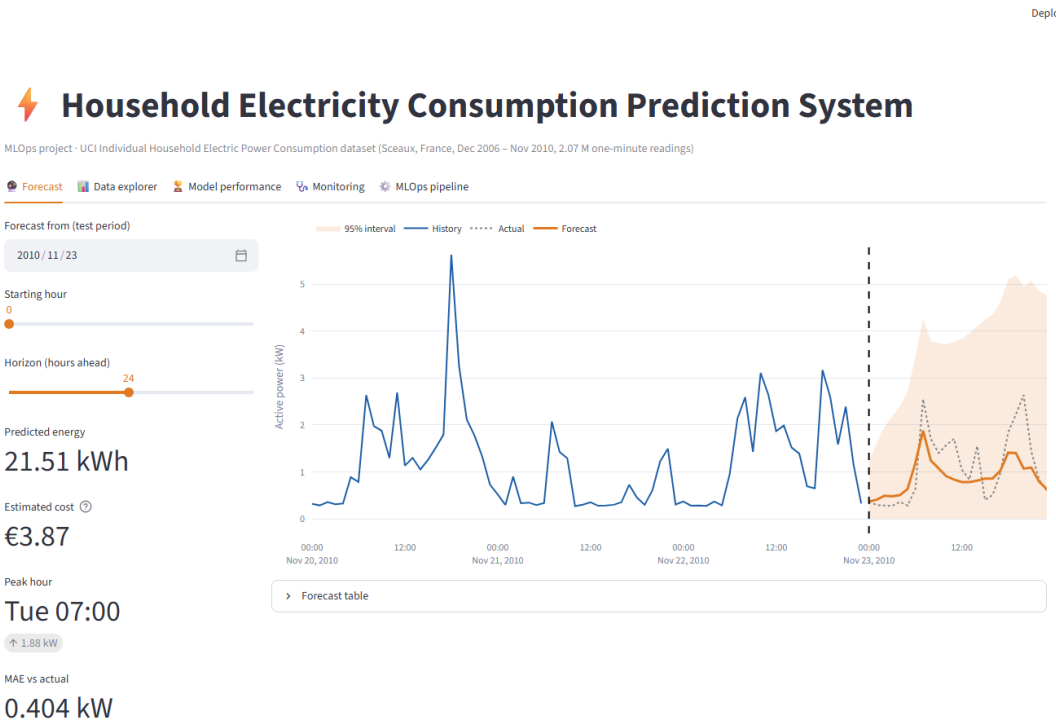


Figure M.6*: Streamlit dashboard (Docker / local) – forecast tab*